



Security Audit Report

GenLayer Token & Vesting

Prepared by:

Arcadia

Prepared for:

GenLayer Labs Corp.

Original report: August 27th, 2026

Latest revision: September 28th, 2026

<https://arcadia.agency/>

Email: audits@arcdiamgroup.com

Telegram: <https://t.me/thearcdiagroup>

Auditor Information

Company: Arcadia

Address: 955 W John Carpenter Frwy, Ste. 100, Irving, Texas 75039

Website: <https://arcadia.agency/>

Email: audits@arcdiamgroup.com

About Us: Arcadia is a software security and product development company building and securing software since 2017.

Client Information

Company Name: GenLayer Labs Corp.

Website: <https://genlayer.com/>

Documentation: <https://docs.genlayer.com/>

Project Name: GenLayer Token & Vesting

Current Token: GENOrbit, inheriting GENCore

Current Vesting: Vesting, VestingFactory, and the helpers listed in Section [1.1.2](#)

Revision History

Date	Reason	Version
08/26/2026	Initial Audit Scope	Source snapshot (see §1.1)
08/28/2026	Initial Version	
09/4/2026	Preliminary Findings Delivery	Source snapshot (see §1.1)
09/08/2026	Remediation Review	Source snapshot at 66af77611
09/10/2026	Feedback Provided to Customer on Remediation	Source snapshot at 66af77611
09/12/2026	Remediation Review	Source snapshot at f6230f7e8
09/16/2026	GTV-39 remediation review; GTV-36 reopened	Recovery snapshot 0760dd53e850 , following f6230f7e8
09/16/2026	GTV-36 acknowledged and accepted by GenLayer as a design trade-off	Recovery snapshot 0760dd53e850
09/24/2026	Token and vesting delta review: Orbit token, standalone vesting, and scheduled rewards/loss accounting	Snapshot 404cd5ef8fa4 ; baseline 0760dd53e850 . Corresponding commit: 62e309242f61 (confirmed by GenLayer).
09/25/2026	TGE audit: exact-change verification and manual review of all 13 scoped implementation files	Same snapshot 404cd5ef8fa4 ; 47 passing tests.
09/26/2026	TGE audit, second pass over the same scope: GTV-40 raised (High)	Same snapshot 404cd5ef8fa4 ; five reproduction tests.
09/28/2026	GTV-40 remediation review: remediated, with a deployment notice	Fix commit cb764fc5c , directly above 62e309242f61 ; six fix-verification tests.

September 25 and 26 audit. All 13 files in the agreed token generation event (TGE) scope were manually reviewed, including unchanged dependencies (Section 1.1.2). One new finding was confirmed: GTV-40 (High), where the L1 token lets the L2 staking address mint and that address can be occupied on L1 (Section 4.8). All 47 distinct tests passed; 31 vesting/recovery tests also passed under Paris (Section 5.9), and five further tests reproduce GTV-40 (Section 5.10). GenLayer fixed GTV-40 in **cb764fc5c**; it is remediated, with two deployment steps recorded in the finding. GTV-39 remains remediated, and

GTV-36 remains acknowledged and accepted. Separate bridge deployments and the later staking migration remain outside the scope.

Contents

1	Introduction	8
1.1	Audit Scope	8
1.1.1	Original audit scope	8
1.1.2	Current TGE scope and September follow-up reviews	10
1.2	Out of Scope	11
1.3	Review Team	12
1.4	Project Background	12
1.5	Methodology	13
1.6	Threat Modeling	14
1.7	Key Security Concerns	16
2	Report Summary	17
3	Audit Findings	20
3.1	[GTV-01] An inflation message with a zero epoch target rewinds the mint counter and re-arms every minted epoch	20
3.2	[GTV-03] An unbounded schedule parameter overflows and permanently bricks a vesting contract	22
3.3	[GTV-04] A creator that cannot receive value makes a vesting un-revocable and its principal unclaimable	24
3.4	[GTV-05] A write-once share floor strands the creator's claw-back behind the just-revoked beneficiary	25
3.5	[GTV-06] The post-revocation reserve covers self-stake but not delegation principal	28
3.6	[GTV-07] An ownership handover leaves the deployer holding unlimited mint authority	29
3.7	[GTV-08] A partially-matured delegation claim books a phantom loss and orphans the remaining tranche	31
3.8	[GTV-09] The bridge fee is uncapped net-new supply and its reclaim path is unimplemented	34
3.9	[GTV-10] A genesis-epoch delegation exit books no profit or loss	36
3.10	[GTV-11] Revocation splits recovered gains but migrates prior losses in full, shorting the beneficiary	37
3.11	[GTV-12] Stopping then revoking traps the stopped flag and disables the beneficiary's settlement	39
3.12	[GTV-13] A two-step operator transfer defeats the distinct-operator check	40
3.13	[GTV-14] A one-wei squat blocks a beneficiary's grant and the clear is not atomic	42
3.14	[GTV-15] Role rotation grants without revoking the previous holder	43
3.15	[GTV-16] Rotating a role leaves the factory's registry describing the wrong parties	45

3.16	[GTV-17] The creator can decouple the vested numerator from <code>totalAmount</code> and then dilute every ratio-denominated protection	46
3.17	[GTV-18] Revoking a manual-unlock vesting before it is unlocked zeroes the beneficiary permanently	48
3.18	[GTV-19] The GEN implementation contract is left initializable	49
3.19	[GTV-20] The inflation curve reverts permanently past a finite epoch	50
3.20	[GTV-21] The compounding root uses a 360-day year while the decay horizon uses 365	51
3.21	[GTV-22] Inflation has no L1 clock check, and the inputs for one are present but unused	52
3.22	[GTV-23] An unguarded signed cast in <code>divInt</code> can wrap negative	53
3.23	[GTV-24] A superseded inflation message is indistinguishable from a live one	54
3.24	[GTV-25] The validator address sync is permissionless, so a third party picks the moment a registry migration takes effect	56
3.25	[GTV-26] Informational suggestions	57
3.26	[GTV-27] A delegation exit taken while a same-epoch deposit is still unpriced over-attributes principal, books a phantom loss and strands the deposit	61
3.27	[GTV-28] The revoker's capped partial exit is rejected by staking's minimum-stake rule, so the clawback can never start	63
3.28	[GTV-29] A pending operator transfer survives revocation and the incoming operator completes it on the wallet directly	65
3.29	[GTV-30] Self-stake settlement books returns principal-first, so realized profit or loss can be deferred across the revocation boundary	66
3.30	[GTV-31] The L1 inflation cursor advances before the L2 delivery is known to succeed, and a single failed delivery deadlocks inflation with no recovery path	68
3.31	[GTV-32] The registry rotation added in <code>c382be97</code> can be blocked by a one-wei squat on the target address	70
4	Additional Findings in Later Builds	71
4.1	[GTV-33] A permissionless staking claim desynchronises the self-stake ledger and books principal as reward	72
4.2	[GTV-34] Legacy proxies cannot be revoked until the beneficiary co-signs the migration	74
4.3	[GTV-35] The accounting migration cannot detect a missing validator, and a miss is permanent	75
4.4	[GTV-36] Revoker recovery can exit the beneficiary's vested share	77
4.5	[GTV-37] A router pin is irrevocable and a wrong one writes the position off	79
4.6	[GTV-38] Design notes on the new settlement model	80

4.7	[GTV-39] Revocation walks every validator wallet, so the beneficiary controls whether it fits in a block	81
4.8	[GTV-40] The L2 staking address can mint on the L1 token, and on Orbit that address can be occupied on L1	83
5	Testing and Verification	87
5.1	Original audit invariant register	88
5.1.1	The six violated properties	89
5.1.2	Where a passing property is not reassurance	90
5.1.3	The uncovered properties	90
5.2	Original audit test suite composition	91
5.3	Original audit invariant campaigns	92
5.4	Original audit inflation fuzzing	92
5.5	Original audit reproduction and verification tests	93
5.6	Limits of the original testing	93
5.7	September 16 recovery follow-up: GTV-39 and GTV-36	93
5.8	September 24 token and vesting delta review	94
5.9	September 25 audit of the agreed TGE scope	95
5.10	GTV-40 reproduction and fix verification	97
6	Contact Information	98
7	Disclaimer	98

1 Introduction

GenLayer Labs engaged Arcadia to audit the **GEN token** and its **vesting system**. This report records the original audit, subsequent remediation reviews, the September 24, 2026 differential review, and the September 25 audit of the agreed TGE scope. GenLayer’s original delivery targeted a zkSync Elastic Chain; the current token and vesting deployment targets Arbitrum Orbit. Historical findings retain the source versions and behavior against which they were raised.

The system spans two domains. GEN is an upgradeable ERC-20 on Ethereum (L1) and the *native base token* of the GenLayer rollup, so the vesting contracts hold native value. The current token implementation is **GENOrbit**, inheriting **GENCore**. For the TGE, vesting can operate before staking infrastructure exists, and token inflation remains unconfigured. The original zkSync transport and percentage-based executor payment are historical; the current implementation is described below.

The audit evaluates the security and correctness of these contracts, with emphasis on the four areas where token-and-vesting systems most often fail: supply integrity across the trust boundary, entitlement arithmetic, distribution of value between parties with opposing interests, and availability of funds under adversarial or mistaken operation.

1.1 Audit Scope

The original audit covered the five delivered files below, pinned by content hash. The September 24 follow-up assessed changes against the September 14 final-recovery package. On September 25, all 13 files in the agreed current TGE scope were manually reviewed. That scope is listed separately in Section 1.1.2; it covers implementation files and excludes interface-only files.

1.1.1 Original audit scope

Description	File	SLOC
Token, inflation & bridge	src/GEN.sol	638
Vesting escrow	src/Vesting.sol	1,274
Vesting deployer	src/VestingFactory.sol	333
Vesting interface	src/IVesting.sol	256
Factory interface	src/IVestingFactory.sol	12
Total		2,513

The original file counts and SLOC, including the two interfaces, are retained as historical delivery records. They are not the current TGE scope or its SLOC total.

Original source provenance: Arcadia received the files in two zip files. Their hashes are recorded below.

File	SHA-256	Bytes
GEN.sol	3bdee9e64a4624b5cff7489bca6d109c9e0131112a6b6e785aaed6e2f4a086ae	19,901
Vesting.sol	5c744e735ddb6dd67c12e786e6be7d9d68956048d1898c4b10a681b28074e1d	47,880
VestingFactory.sol	0b2cd147f783ca21c3162e1e998d64b0b71dfd69b2839ccea58c3078ede65d5f	10,619
IVesting.sol	593afdb7b0d495cdd86106b804124006d9858bb9c38360f69167fefb3c762431	8,461
IVestingFactory.sol	f4fb8471c7409f53bab3a4639eb02e2c5c89bfdc76c818bcf20571f605485b3b	393

Earlier reference and remediation reviews. Arcadia requested the L2 sources needed to understand the original trust assumptions. These counterparties were read for context and were not audited. The reference package `genlayer-consensus-token-vesting-reference-c382be97` also contained newer versions of the audited contracts. Findings were checked against that version and, after report delivery, against build `66af77611`. Each finding's Remediation Status preserves those results.

In the comparison with `c382be97`, GTV-01, GTV-07, GTV-15, and GTV-19 were already fixed, while GTV-03, GTV-04, GTV-06, and GTV-12 remained present. These are historical comparison results; current dispositions are in the summary. The `INFLATION_INITIAL` parameter also moved from 15% to 9%, so the original year-one figure in GTV-21 describes the earlier curve.

September 16 recovery review: Subsequent reviews covered `f6230f7e8` and the recovery package `genlayer-vesting-gtv39-final-recovery-20260914-0760dd53e850.zip`. The latter was reviewed on September 16, 2026, as a follow-up to `f6230f7e8`, focusing on GTV-39, the changed vesting recovery paths, and their effect on GTV-36. This was a targeted source review with executable probes, not a full audit of the package or its deployment. GTV-39 is remediated in that snapshot. GTV-36 was reopened because its vested-stake protection was removed; GenLayer subsequently acknowledged and accepted the resulting staking interruption and potential foregone rewards as a deliberate design trade-off.

Original dependency reconstruction: The original `GEN.sol` imported four files that were not delivered: two interfaces (`IGEN`, `IGenLayerStaking`) and two math libraries (`FixedPoint`, `LogExpMath`). Arcadia reconstructed the interfaces from their call sites and vendored the libraries from Balancer V3 to compile and test. These dependencies were outside the original scope. Enum ordering and the fixed-point domain were therefore assumptions of that original harness. The September 20 package supplies its own dependencies; the

September 24 and 25 test-build distinctions are recorded in Sections 5.8 and 5.9.

1.1.2 Current TGE scope and September follow-up reviews

The September 24 and 25 reviews used the September 20 package `genlayer-to-ken-vesting-delta-review-20260920-404cd5ef8fa4.zip` and the September 14 final-recovery snapshot 0760dd53e850 as the comparison baseline. The September 25 audit first verified the extracted deliveries against their manifests and checksums, then confirmed that applying the supplied diff reproduced the current files.

Reviewed content snapshot (SHA-256):

404cd5ef8fa4726f1657e2a1e565a9c70caf93c70437f28024db7904bbd868a6

Corresponding pushed commit, confirmed by GenLayer:

62e309242f61068cf3575e68522cdb971a123d88

The content snapshot remains the reference for the reviewed package; the commit correspondence is recorded from GenLayer's confirmation.

GenLayer confirmed that the current scope is limited to the token and vesting implementations intended for TGE. The following **13 implementation files** define that scope. Paths are relative to `source/contracts/` in the package.

Component	File
Token implementation	<code>GENOrbit.sol</code>
Inherited token logic	<code>bridge/GENCore.sol</code>
Token delivery storage	<code>bridge/GENInflationDeliveryStorage.sol</code>
Token delivery logic	<code>bridge/InflationDelivery.sol</code>
Vesting escrow	<code>vesting/Vesting.sol</code>
Vesting factory	<code>vesting/VestingFactory.sol</code>
Vesting storage	<code>vesting/VestingStorage.sol</code>
Migration helper	<code>vesting/VestingMigrationModule.sol</code>
Operator helper	<code>vesting/VestingOperatorModule.sol</code>
Recovery helper	<code>vesting/VestingRecoveryLens.sol</code>
Operator registry	<code>vesting/VestingOperatorRegistry.sol</code>
Registry binding logic	<code>utils/ProtocolRegistry.sol</code>
Public-key utilities	<code>utils/PubKeyUtils.sol</code>

`GENOrbit` inherits `GENCore`; they form one token implementation behind the token proxy. `GENCore` is not deployed separately. The `Vesting` constructor

deploys its migration module, operator module and recovery lens, and the factory's beacon setup creates the operator registry. Their code and the inherited or internal library logic above belong to the deployed token and vesting system.

Interface-only files are compilation dependencies and are excluded from the current audit file list and audited SLOC. The original `GEN.sol` remains historical code, not the Orbit deployment target. The token imports `ArbitrumGENBridge` for its external-call ABI and types; this does not deploy that adapter or add its separate implementation to the TGE scope.

Changes from the September 14 delivery. Two scoped files were added: `GENOrbit.sol` and `GENCore.sol`, with the latter extracting shared logic from the previous `GEN.sol`. Three files have executable changes: `Vesting.sol`, `VestingFactory.sol`, and `VestingOperatorModule.sol`. `VestingStorage.sol` has comment-only changes. The other seven scoped files are byte-identical to the baseline.

Review coverage. September 24 was a differential review of new and changed implementations, following unchanged dependencies where needed. It recorded 30 passing local tests (Section 5.8). The September 25 audit manually reviewed **all 13 scoped implementation files and their interactions**, including the seven unchanged files. It added 20 tests and reran 27 regressions; all 47 passed (Section 5.9). A second pass over the same scope on September 26 raised one new finding, GTV-40 (Section 4.8), which GenLayer fixed in commit `cb764fc5c`. GTV-39 remains remediated and GTV-36 remains acknowledged and accepted.

Documentation: <https://docs.genlayer.com/>. NatSpec and code comments are treated as claims to validate against the implementation, divergence from this is treated as notable drift.

1.2 Out of Scope

The following components are outside the current TGE scope. Their assumptions and effects on the in-scope token and vesting code were considered where relevant:

- **The L2 counterparties.** `GenLayerStaking`, `ValidatorWallet`, and `AddressManager`. Arcadia read reference copies from the archive above but did not audit them, so they enter the report only as *trust assumptions*, where a finding's severity depends on their runtime behavior.
- **Separate bridge deployments.** `ArbitrumGENBridge`, `ArbitrumInflationEscrow`, and `ArbitrumInflationFeeVault` are not part of the confirmed TGE deployment scope. Other standalone legacy bridge contracts are also outside it. Bridge-facing logic and activation guards inside `GENOrbit` and its inherited implementation remain in scope.

- **Rollup infrastructure.** The canonical Arbitrum/Nitro bridge and the later staking migration were not audited end to end. The zkSync BridgeHub, SharedBridge/L1AssetRouter, and inclusion-proof assumptions belong to the historical audit. The canonical bridge used to fund native-GEN grants is a deployment dependency, not a component audited by this report.
- **Third-party libraries.** OpenZeppelin and the Balancer-derived Fixed-Point/LogExpMath libraries were not independently audited. The package pins OpenZeppelin Contracts and Contracts Upgradeable to **5.1.0**. The September 24 harness used **5.4.0**; the September 25 harness used local **5.1.0** source copies with recorded hashes. The latter did not independently verify npm archive integrity or reproduce the full dependency-locked Hardhat build (Section 5.9).
- **Deployment and operations.** No deployment scripts accompanied the original five-file delivery. Later packages supplied deployment material. The September 25 tests exercised factory initialization through a transparent proxy and grant creation through beacon proxies, but did not validate a live deployment, Safe execution, the complete production deployment sequence or an upgrade rehearsal. The token ERC1967 proxy, factory transparent proxy, and vesting beacon/proxies rely on third-party OpenZeppelin implementations.
- **Key management and governance configuration**, including any multisig or timelock intended to hold owner authority.
- **Off-chain components:** indexers, executor and watcher bots, SDKs, and front-ends.

1.3 Review Team

Our Engagement-Specific Team

- **Van Cam Pham, PhD** - Lead Security Engineer
- **Rasikh Morani** - Co-Founder & Secondary Review

1.4 Project Background

Historical token design. The original GEN implementation used zkSync message proofs and paid the executor a percentage deducted from scheduled inflation. Its initial inflation-rate parameter was 15%, declining toward 4%. Those descriptions and the related findings apply to the earlier snapshots.

Current token implementation. GENOrbit and GENCore implement the UUPS-upgradeable token named “GenLayer” (symbol GEN). The initial allocation remains 4.2 billion GEN, with `initialMint` callable once and role-gated minting retained. The scheduled inflation curve uses a **9% initial-rate parameter and a 4% terminal rate**. The implementation supports Arbitrum

delivery with full scheduled principal and a bounded, separately minted gas-based keeper payment. The initial bridge-fee mint has its own bound; it is separate from the keeper-payment cap. At TGE, this transport and inflation are unconfigured. Deployment of the counterpart contracts and subsequent protocol activation fall outside the TGE scope.

The vesting system. `VestingFactory` deploys one `Vesting` escrow per beneficiary as a beacon proxy, funded at creation with native GEN. Each escrow enforces a cliff, an optional percentage unlocked at the cliff, and linear periods afterwards. A beneficiary withdraws vested tokens; a creator funds the grant, edits its schedule and receives the unvested allocation on revocation; an optional revoker can stop or revoke it. The current vesting floor protects accrued entitlement against schedule edits, but scales with realized staking rewards and losses. It is no longer an absolute amount that can never decrease.

Historical staking accounting. Earlier versions applied accumulated rewards and losses outside the vesting schedule when calculating withdrawals. That description remains relevant to the historical findings but does not describe snapshot `404cd5ef8fa4`.

Current staking accounting and deployment. Before revocation, realized rewards and losses adjust the vesting base and follow the grant's schedule; claimed rewards are not automatically fully withdrawable. Revocation snapshots the adjusted base and economic vested entitlement, and subsequent settlement follows the post-revocation allocation rules. For TGE, grants may be deployed with a zero `AddressManager` and support funding, withdrawals and revocation without staking. Later connection uses the factory's one-time manager choice and requires a sealed registry. New staking remains unavailable until those conditions are met.

1.5 Methodology

The following methodology describes the original audit. Later reviews and the September 25 TGE audit are identified separately by snapshot and date.

Project investigation: Reconstructed the system from source and documentation, the two domains and their bridge, the inflation and supply design, the vesting schedule and roles, staking-while-vesting, and the upgrade and storage layout, since only `NatSpec` accompanied the code. This produced the architecture document the rest of the work used.

Threat modeling: Built an explicit model of what the contracts hold, who can reach it, and by which path, covering assets, trust boundaries, and attack surface. It decided where the review spent its time and is given in full below.

Invariants assessment: Arcadia derived machine-checkable properties from the original implementation: supply conservation, the then-monotonic vested floor, entitlement limits, value conservation through staking, and others. The original register and its results are in Section [5.1](#).

Manual code review: Read the code line by line against that target list, focused on supply and message decoding, entitlement arithmetic, revocation and reserve accounting, staking profit-and-loss reconciliation, access control, and the upgrade path, always asking how the three roles could order calls to shift or strand value.

In-house auditing harness: Arcadia’s proprietary tool ranked, breadth-first, the intersection of what an unprivileged caller can reach and what carries value. Its output is a lead, not a verdict: no finding shipped without an engineer reproducing it.

Property-based and fuzz testing: A Foundry harness ran the register as stateful invariant campaigns, fuzzed the inflation mathematics across its domain, and reproduced every Medium-and-above finding. The L2 counterparties, unavailable at runtime, were modeled twice, once honest and once hostile, to bound the damage when an assumption fails.

Independent second pass: A separate review ran blind to the first, and the two were reconciled against executed evidence, with a test settling any disagreement.

September 25 TGE audit: Verified the exact delivered changes before reviewing all 13 scoped files, including the extracted token logic, unchanged helpers and cross-module calls. The review traced access control, initialization and storage, minting and bridge-facing calls, scheduled staking accounting, revocation and recovery. Targeted tests exercised these paths and their failure cases. This audit did not repeat the original stateful invariant campaign or independently audit the external staking and bridge implementations.

1.6 Threat Modeling

Original audit model. The model below and Figure 1 describe the original zkSync-based delivery. They are retained to explain the historical findings and the original test suite, not as a diagram of the current Orbit deployment. The model categorizes 40 threats and ends in 31 machine-checkable properties.

Assets: Seven things carry value or authority, on Ethereum, the total GEN supply and the inflation in transit to L2. On GenLayer, the native GEN escrowed in each vesting, the creator’s claim on the unvested remainder, and the staked principal with its gains and losses. Two span both chains: the role and upgrade authority over everything else, and the plain ability to get funds out once they are in. That last asset earns its place, because several findings freeze or destroy value with no one profiting, and a model that only asks who can steal would have missed them.

Trust boundaries. These contracts sit on both sides of a bridge, and neither side can inspect the other. Five assumptions the code depends on and cannot check. T1: ZKSync’s `proveL2MessageInclusion` is sound, so a `true` return means the message really came from the pinned L2 sender. T2, **the L2 staking**

contract is honest, reporting truthful epochs, shares, and validator views, returning the value it owes, and applying the same executor carve L1 applies (the L2 Staking Contract was explicitly not in scope). T3: AddressManager resolves to the canonical staking and consensus addresses. T4: a ValidatorWallet forwards its whole balance to its owner on claim. T5: the owner keys on both chains are held by their intended parties.

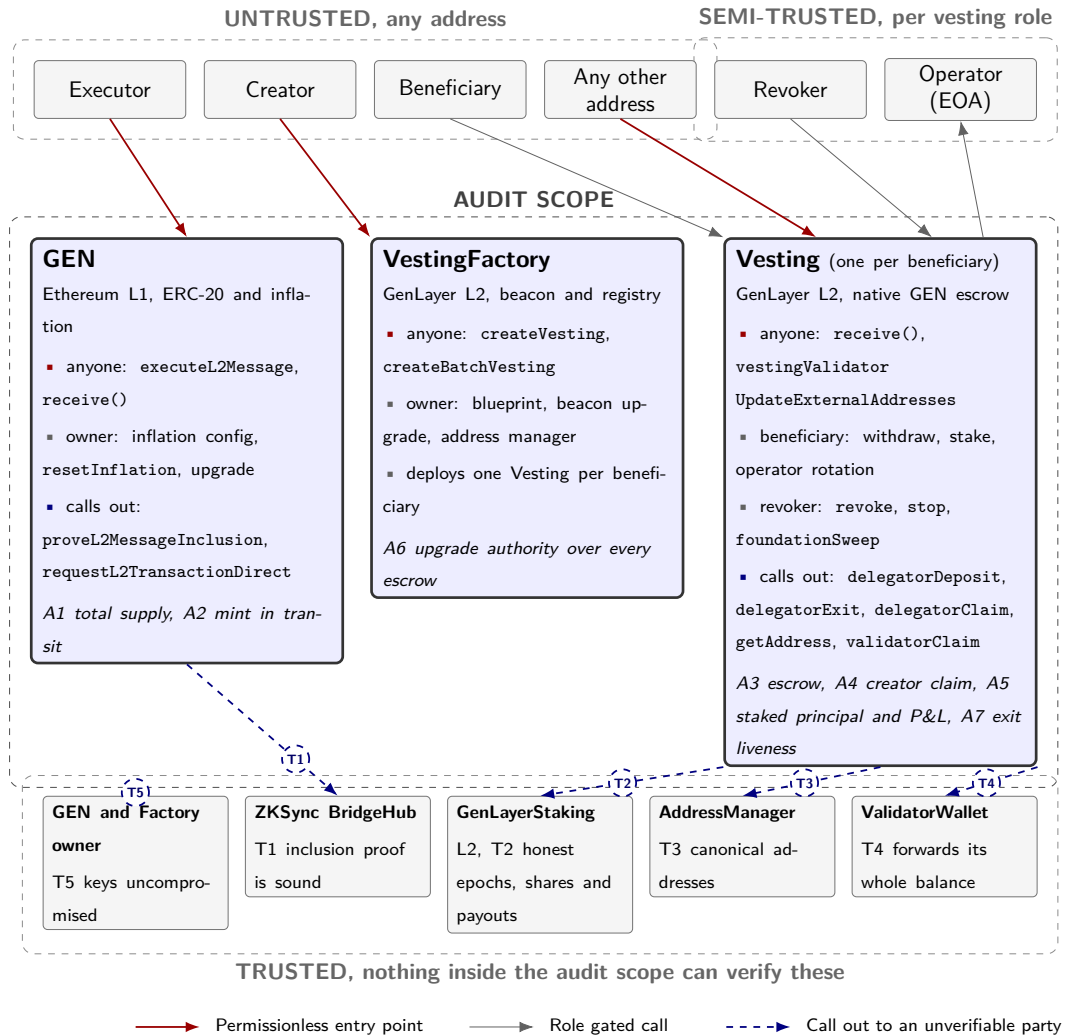


Figure 1: Original audit threat model (historical zkSync-based delivery). Red marks what any address can reach, grey what a role gates, and blue dashed what the original audited code hands off to a counterparty. T1 to T5 identify its trust assumptions; A1 to A7 identify its assets. The current TGE scope and September 24 changes are described separately in the text.

These five assumptions are the factors that most heavily decide severity, so the report treats them as open questions. Where a finding's impact depends on one, the report states the question that settles it and leaves the answer to the

client, who can read the L2 source. The test suite models each counterparty twice, once honest and once hostile.

Original attack surface: The three contracts expose roughly seventy-five external functions, most of them views or role-gated. The interesting subset is small: `executeL2Message` mints supply and pays its caller; `createVesting` and `createBatchVesting` take funds from anyone; `vestingValidatorUpdateExternalAddresses` can be called by any party against any escrow; and the payable fallbacks on GEN and every vesting let anyone add balance to contracts that infer value from balance deltas. That subset got disproportionate attention, and the report's most serious finding lives there.

Current TGE assessment, September 24–25. The review covered token minting and authority, transport activation and caller authentication, separate keeper issuance, scheduled rewards/loss accounting, revocation allocation, and delayed staking connection. The September 25 audit also traced shared reentrancy protection, operator ownership and reservation cleanup across the factory, grants and helper modules. At TGE, the token and standalone vesting must remain usable while their later bridge/staking configuration is absent. The token's bridge-facing code is in scope; it relies on the configured adapter for authenticated bridge context and a bounded keeper quote. The separate adapter, custody contracts, canonical bridge and complete staking migration are outside the confirmed TGE scope. The supplied staking implementation still uses zkSync messaging, so later Orbit activation requires further integration work and end-to-end validation.

1.7 Key Security Concerns

The following nine concerns framed the original audit; current changes and their scope are described above:

- **Supply integrity** across the bridge: whether the inclusion proof, pinned sender, replay marker, and monotonic epoch counter can be disabled or rewound.
- **Executor economics:** whether the permissionless curve is bounded and exact, and whether the L1 and L2 sides can drift out of step.
- **Inflation mathematics:** overflow, underflow, truncation, and domain violations across the fixed-point curve, and the per-epoch loop's gas ceiling.
- **Entitlement correctness:** whether the schedule, floor ratchet, and revocation snapshots can over- or under-grant the beneficiary.
- **Distributional correctness:** whether value can shift between beneficiary, creator, and revoker by ordering, by staking outcomes that straddle revocation, or by asymmetric reserves. This held the material risk.
- **Staking reconciliation:** whether the balance-delta inference of profit

and loss can be polluted, losses go unbooked, or principal be stranded or double-counted.

- **Availability:** whether any reachable state stops funds from ever leaving, including one entered by a mistaken call or a counterparty that cannot receive payment.
- **Authority and upgradeability:** the dual `Ownable` and `AccessControl` roots, the beacon over every escrow at once, the registry resolved on every call, and whether authority transfers completely.
- **Validator operator safety:** whether operator keys rotate safely, uniqueness constraints hold, and operator control can be seized after revocation.

2 Report Summary

The original audit and early reference review identified **31 findings** in the GEN token and vesting contracts. Reviews of subsequent builds added **8**, for **39** in total. The September 24, 2026 differential review and the September 25 and 26 audit of all 13 agreed TGE implementation files used snapshot `404cd5ef8fa4`. That audit raised **one new High finding, GTV-40**, since remediated in `cb764fc5c`. The historical findings fall into three main areas.

The first was supply integrity: inflation messages or initialization calls could rewind the counter used to prevent repeated minting. The second was availability: schedule overflow, rejecting counterparties, reserves and incompatible state flags could prevent funds from leaving a grant. The third was allocation between beneficiary, creator and revoker: staking outcomes across revocation, asymmetric reserves and unrecorded accounting could shift value between them. These describe defects in the historical snapshots. Their current dispositions are recorded below and in each finding.

Every finding of Medium severity or above in the original audited snapshot is backed by an executed Foundry reproduction. The original suite has **125 passing tests**. The September 16 recovery review of `0760dd53e850` recorded **16 passing targeted tests** (Section 5.7). The September 24 differential review recorded **30 passing local tests** (Section 5.8).

The latest audit, on September 25, manually reviewed all 13 scoped files and recorded **47 distinct passing tests**: 20 new tests and 27 rerun regressions. The 31 vesting/recovery tests also passed under Paris; these repeat tests already included in the 47, rather than adding to that count (Section 5.9). Five further tests reproduce GTV-40 and six verify its fix (Section 5.10). These are separate review stages, not a rerun of the original suite against each package.

Severity Level	Count	Remediated	Ack.	Open	Finding IDs
Critical	1	1	0	0	GTV-01
High	10	10	0	0	GTV-03 – GTV-08, GTV-27, GTV-33, GTV-34, GTV-40
Medium	15	15	0	0	GTV-09 – GTV-17, GTV-28 – GTV-31, GTV-35, GTV-39
Low	11	10	1	0	GTV-18 – GTV-25, GTV-32, GTV-36, GTV-37
Informational	2	1	1	0	GTV-26, GTV-38
Total	39	37	2	0	

Status legend. Findings were raised against the audited snapshot for GTV-01 to GTV-26, c382be97 for GTV-27 to GTV-32, 66af77611 for GTV-33 to GTV-38, f6230f7e8 for GTV-39, and 404cd5ef8fa4 for GTV-40. The table carries forward their dispositions through the September 25 and 26 TGE audit of 404cd5ef8fa4 and the September 28 review of the GTV-40 fix in cb764fc5c; it does not imply that every historical finding was freshly retested. Each finding identifies the snapshot at which its remediation or acceptance was recorded. *Remediated* means the defect was verified closed. *Acknowledged* includes informational items for which Arcadia does not require changes and risks explicitly accepted by GenLayer; GTV-36 is recorded as *Acknowledged and accepted* in this column. Acceptance records the client’s decision to retain the behavior and its associated risk. *Open* means a finding awaits remediation or acceptance. *Reopened* records that a later change reintroduced the reported impact; it counts as Open until remediated or accepted. Each finding’s **Remediation Status** preserves the earlier review history.

Current Status. 25 of the 31 audit findings were closed in 66af77611. Reviewing that build raised six more (GTV-33 to GTV-38, Section 4). GenLayer then shared f6230f7e8, which closed all six and addressed the outstanding GTV-09, GTV-21, GTV-22, GTV-25, GTV-31, and seven of the nine items in GTV-26. GTV-26 keeps two informational items that were not applied, donations swept to the creator and GEN’s two authority roots; neither affects security and Arcadia does not ask for them, so the finding is closed as acknowledged.

The recovery snapshot 0760dd53e850 **remediated GTV-39**: revocation no longer walks or calls validator wallets, new wallet admission is capped, and revoker recovery operates on one selected position. The same change reintroduced GTV-36’s forced-unstaking impact by removing the vested-stake floor from revoker exits. GenLayer explicitly accepted full unstaking after revocation, including the vested portion, and the resulting staking interruption and potential foregone rewards during unbonding and re-entry. The revoker does not acquire the beneficiary’s vested entitlement. GTV-36 is therefore **Acknowledged and accepted**, with severity remaining Low.

The September 25 and 26 audit of 404cd5ef8fa4 found that **GTV-39 remains remediated** and **GTV-36 remains acknowledged and ac-**

cepted. All ten GTV-39 regression tests passed. In the current accounting, pre-revocation rewards and losses adjust the scheduled vesting base; revocation uses that adjusted base and economic vested entitlement. The audit also raised **GTV-40 (High, likelihood Low)**: the L1 token lets the L2 staking address mint, and on the new Orbit rollup that address can be occupied on L1 (Section 4.8). GenLayer fixed it in `cb764fc5c`, which stops granting token roles to that address; two deployment steps remain, listed in the finding. The totals are **37 remediated, two acknowledged, and none open**.

Severity model. Severity combines impact with likelihood, where likelihood accounts for exploitability, preconditions, and the trust level required. A finding whose likelihood depends on a compromised or misused trusted role is rated on impact, with its preconditions stated in the finding.

Historical launch gate, closed in `f6230f7e8`. `GEN.sol` shipped an owner-only `resetInflation` that re-armed a second 4.2 billion genesis mint, reproduced in GTV-02 (removed from the finding list), and the contract’s own NatSpec marked it “REMOVE BEFORE MAINNET.” Production `GEN.sol` in `f6230f7e8` carries no such selector. It survives only in `GENTestnet`, a separate subclass the deployment script builds behind an explicit `enableTestnetReset` flag, and even there the function reverts on chain id 1 and once inflation delivery is activated. That build also supplied the deployment script this engagement previously lacked. These details record that historical remediation.

Current TGE deployment target. Build `GENOrbit`, which inherits `GENCore`, and deploy that token implementation behind its ERC1967 proxy. `GENCore` is an abstract base, not a separate deployment. Vesting includes `Vesting`, `VestingFactory`, their deployed helpers and the inherited/shared logic listed in Section 1.1.2. The TGE setup leaves inflation unconfigured and initializes standalone vesting with a zero `AddressManager`. Separate bridge contracts and the later staking migration are outside the confirmed TGE scope. The supplied staking code still uses `zkSync` messaging; subsequent Orbit activation requires further migration and end-to-end integration validation. Before that activation, the L2 escrow and fee vault must be deployed from a key never used on L1, and the token must hold no role for the staking address (GTV-40). Closing the report’s findings does not by itself approve that later deployment.

Historical trust assumptions. The L2 staking contract, validator wallet and `AddressManager` were absent from the original five-file delivery. Later reference sources were read where findings depended on them; those counterparties remain outside the TGE scope. The historical findings retain their stated assumptions and remediation evidence.

3 Audit Findings

3.1 [GTV-01] An inflation message with a zero epoch target rewinds the mint counter and re-arms every minted epoch

Risk Level

Severity: Critical, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/GEN.sol`: 407–415 (guard exemption), 422 (unconditional write), 276–301 (INFLATION arm)

Description

Inflation is realized through `_inflation`, which advances `lastInflationEpoch`. That counter is the only defense that stops an already-minted epoch range from being minted a second time, because the replay map keys on caller-supplied coordinates that a second message can simply vary. When the target epoch is zero, `_inflation` skips both forward-progress guards, and then writes the counter anyway:

```
1 // _epochTo == 0 is the inflationInit handshake (no epochs to realize):
2 if (_epochTo > 0) {
3     if (_lastInflationEpoch >= _epochTo) revert InflationNotReadyToBeRealized();
4     if (maxInflationEpochs < _epochTo - _lastInflationEpoch)
5         revert CanOnlyTriggerInflationMaxEpochsInFuture();
6 }
7 // ...
8 lastInflationEpoch = _epochTo; // line 422: runs for _epochTo == 0 too
```

The exemption keys on the value, not the caller, and `_inflation` is shared between the owner-only `inflationInit` and the permissionless `executeL2Message`. The INFLATION arm never rejects a zero `epochTo`, even though the BURN arm immediately beside it rejects a zero amount at line 305.

Attack Scenario

Two paths reach the same effect. The first is through the bridge: a proven L2 message that carries `epochTo == 0` rewinds the counter to zero, after which the epochs already minted can be minted again under fresh replay coordinates. This path depends on whether the L2 staking contract can emit such a message (see the deciding question below). The second path needs nothing external. `executeL2Message` never checks whether inflation has been initialized, so inflation can advance first, and the owner can then call `inflationInit(0, ...)`, the value the NatSpec documents as the handshake, which routes into the same `_inflation(0, ...)` and rewinds the counter.

Impact

Reproduced with a bridge mock: 32,314,905 GEN minted for epochs 1 through 20, then the identical 32,314,905 GEN minted again after one zero-epoch mes-

sage, taking supply from 32.3M to 64.6M for the same twenty epochs. Even with no second mint, the rewind is a permanent denial of service, because every later message measures its range from zero, exceeds `maxInflationEpochs`, and reverts. The audit's independent review withdrew a "cleared" verdict from the first review pass: a test there confirmed the counter blocks replay, but that only proves the counter matters, not that it cannot be moved backward.

Question for the L2 source

Can `GenLayerStaking` ever emit an `INFLATION` message with `epochTo == 0`? The answer settles the bridge path. The owner path is reachable regardless, so the fix is warranted either way.

Proof of Concept

`test_CRITICAL_epochZeroRewindsCounter_enablingDoubleMint` advances inflation legitimately to epoch 20, then sends one `INFLATION` message with `epochTo == 0`, rewinding `lastInflationEpoch` to zero and re-minting the same twenty epochs.

```
1 // test/IndependentGenCritical.t.sol ::
   test_CRITICAL_epochZeroRewindsCounter_enablingDoubleMint
2 // setUp: GEN behind proxy, zkSync + staking wired to MockBridgeHubC, incentive 1%
3 // _msg(e,gp,gl) = abi.encode(IGEN.GenAction.INFLATION, e, gp, gl)
4
5 // --- 1. legitimate inflation for epochs (0, 20] ---
6 vm.prank(executor);
7 gen.executeL2Message(1, 0, 0, _msg(20, 1, 1), emptyProof, 1, 1);
8 assertEq(gen.lastInflationEpoch(), 20, "setup: counter advanced");
9 uint256 supplyAfterFirst = gen.totalSupply();
10
11 // --- 2. the reset: an INFLATION message with epochTo == 0 ---
12 vm.prank(executor);
13 gen.executeL2Message(2, 0, 0, _msg(0, 1, 1), emptyProof, 1, 1);
14 assertEq(gen.lastInflationEpoch(), 0, "CRITICAL: counter rewound to zero");
15
16 // --- 3. epochs (0, 20] are now mintable AGAIN ---
17 vm.prank(executor);
18 gen.executeL2Message(3, 0, 0, _msg(20, 1, 1), emptyProof, 1, 1);
19 uint256 secondMint = gen.totalSupply() - supplyAfterFirst;
20 assertEq(gen.lastInflationEpoch(), 20, "counter re-advanced");
21 assertApproxEqRel(secondMint, supplyAfterFirst, 0.01e18, "epochs 1-20 minted a SECOND time");
```

The counter returns to zero after the zero-epoch message and the second mint matches the first to within 1%, doubling supply for the same twenty epochs; companion tests confirm the rewind is also a permanent DoS (`CanOnlyTriggerInflationMaxEpochsInFuture`) and that the BURN arm rejects the same zero the `INFLATION` arm accepts.

Recommendation

Guard the write: `if (_epochTo > 0) { lastInflationEpoch = _epochTo; }`. This closes both paths in one line. As defense in depth, reject a zero `epochTo` in the `INFLATION` arm to match the BURN arm, and add the L1 clock check described in GTV-22.

Remediation Status

Partially Remediated in the reference tree c382be97, verified by Arcadia against GEN.sol SHA-256 b02f1bfd...59e1 (Section 1.1). The INFLATION arm there adds `if (epochTo == 0) revert InflationNotReadyToBeRealized();` together with `if (!inflationInitialized) revert InflationNotInitialized();`, which closes the bridge path and the owner-ordering path respectively.

One element of the recommendation above is not addressed: the write at `_inflation` remains unconditional, so the guard is enforced only at the two entry points. Arcadia advises applying the one-line change there as well, so a future caller of `_inflation` cannot reopen the issue. The finding stays *Open* against the audited snapshot.

Remediated in 66af77611, which applies that one-line change: the write reads `if (_epochTo > 0) lastInflationEpoch = _epochTo;`, so no path can rewind the cursor.

3.2 [GTV-03] An unbounded schedule parameter overflows and permanently bricks a vesting contract

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 254 (`startDate + cliffDuration`), 1118–1134 (setters), 161–206 (`initialize`)

Description

`_scheduledVestedAt` computes the cliff end as `startDate + cliffDuration` under checked arithmetic. Neither `initialize` nor any schedule setter bounds either input. A single creator call to `setCliffDuration(type(uint256).max)` succeeds, because `_accrueVestedFloor` runs against the still-valid old value before the new value is written. From the next block on, the addition overflows and every path that reads the schedule reverts with an arithmetic panic.

Impact

Reproduced on a 1,000e18 escrow with 550e18 already vested. After one poisoning call, `vestedAmount`, `withdrawableAmount`, `unvestedAmount`, `vestingWithdraw`, `revoke`, and `stopVesting` all panic, and so do all five repair setters, because each one accrues the floor before writing. Since `revoke` can never succeed, the revoker's entire recovery toolkit, which is gated on the revoked flag, becomes unreachable. The full 1,000e18 is locked with no on-chain recovery short of a beacon upgrade. Nobody profits from this: the funds become permanently unreachable rather than stolen. The monotonic floor ratchet was designed so a creator cannot claw back accrued vesting, but it does not stop

the creator from making the funds unreachable to everyone.

Proof of Concept

`test_E_scheduleOverflow_permanentlyBricksTheEscrow` sets `cliffDuration` to `type(uint256).max` with one creator call, then shows every read, withdrawal, and repair path panics with the exact arithmetic-overflow selector while the full escrow stays locked.

```

1 // test/IndependentReview.t.sol :: test_E_scheduleOverflow_permanentlyBricksTheEscrow
2 // setUp: 1000e18 escrow, initialize(startDate=now, cliffDuration=30 days, ...), warp +200 days
3 uint256 vestedBefore = v.vestedAmount();
4 assertGt(vestedBefore, 0, "setup: real value has vested");
5
6 // ONE unvalidated creator call. It SUCCEEDS: _accrueVestedFloor() runs against the
7 // still-valid old cliffDuration, and only then is the poison value written.
8 vm.prank(creator);
9 v.setCliffDuration(type(uint256).max);
10
11 // Every read now panics with the SPECIFIC arithmetic-overflow panic (0x11).
12 vm.expectRevert(stdError.arithmeticError); v.vestedAmount();
13 vm.expectRevert(stdError.arithmeticError); v.withdrawableAmount();
14 vm.expectRevert(stdError.arithmeticError); v.unvestedAmount();
15
16 vm.prank(revoker); vm.expectRevert(stdError.arithmeticError); v.revoke();
17 vm.prank(revoker); vm.expectRevert(stdError.arithmeticError); v.stopVesting();
18
19 // The only repair paths revert, because each accrues the floor BEFORE writing.
20 vm.prank(creator); vm.expectRevert(stdError.arithmeticError); v.setCliffDuration(0);
21
22 // 1000e18 GEN is locked permanently; only a beacon upgrade can recover it.
23 assertEq(address(v).balance, TOTAL, "E: full escrow locked forever");

```

After the single `setCliffDuration(type(uint256).max)` call, every read, withdrawal, and repair path reverts with `stdError.arithmeticError` (panic 0x11), and `address(v).balance == TOTAL` holds, confirming the full 1,000e18 is permanently locked.

Recommendation

Use a saturating addition in `_scheduledVestedAt`, for example return zero when `startDate > type(uint256).max - cliffDuration`, or bound `startDate` and `cliffDuration` to sane maxima in `initialize` and in both setters.

Remediation Status

Remediated in 66af77611. `_validateScheduleBounds` reverts `InvalidScheduleBounds` when `_startDate > type(uint256).max - _cliffDuration`, and is enforced at `initialize` and at both schedule setters, so the overflowing value can never be stored. The finding stays *Open* against the audited snapshot.

3.3 [GTV-04] A creator that cannot receive value makes a vesting un-revocable and its principal unclaimable

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- src/Vesting.sol: 848–851 (revoke), 472–477 (_bookProfitOrLoss), 936–941 (foundationSweep), 1194–1199 (setCreator)

Description

Three functions push native value to `creator` and revert the whole call if the push fails:

```
1 (bool success, ) = payable(creator).call{ value: amount }("");  
2 if (!success) revert TransferFailed();
```

The threat model treats `creator` as untrusted, yet the code requires it to accept a native transfer. A creator address with no `receive` function, such as a treasury or timelock contract, therefore blocks every path that pays it. `setCreator` has no revoked guard, so this state can also be entered on purpose after revocation.

Impact

Reproduced across three cases. A rejecting creator makes `revoke` revert, so the vesting can never be revoked. After a revocation that paid the creator nothing, a pending staking reward makes both the beneficiary's `vestingDelegatorClaim` and the revoker's `revokerDelegatorClaim` revert, stranding 150e18 with no caller able to retrieve it, because both claim paths run through `_bookProfitOrLoss`. `foundationSweep` is bricked the same way. The exposure is specific to the creator: a rejecting beneficiary only harms itself, since `vestingWithdraw` pays the beneficiary and no counterparty depends on it.

Proof of Concept

`test_A2_rejectingCreator_bricksPostRevokeClaim` stakes into a vesting whose `creator` rejects value, revokes, then shows both the beneficiary's and the revoker's claim reverting because each routes its pro-rata reward push through `_bookProfitOrLoss`.

```

1 // test/IndependentReview.t.sol :: test_A2_rejectingCreator_bricksPostRevokeClaim
2 // setUp: 200e18 grant, RejectingCreator, 30 days => 50% vested
3 vm.prank(beneficiary); v.vestingDelegatorJoin(VAL, 150e18);
4 vm.warp(block.timestamp + 30 days);
5 vm.prank(beneficiary); v.vestingDelegatorExit(VAL, 150e18);
6
7 vm.prank(revoker); v.revoke();
8 assertTrue(v.revoked(), "revoke succeeds (no creator payment was due)");
9
10 // Staking returns principal 150e18 + 40e18 reward. Creator's share = 40e18 * 1/2 = 20e18.
11 stk.setPayout(190e18);
12
13 vm.prank(beneficiary);
14 vm.expectRevert(IVesting.TransferFailed.selector);
15 v.vestingDelegatorClaim(VAL);
16
17 // The revoker's recovery path is bricked by the identical helper.
18 vm.prank(revoker);
19 vm.expectRevert(IVesting.TransferFailed.selector);
20 v.revokerDelegatorClaim(VAL);
21
22 assertEquals(v.pendingExitDeposited(VAL), 150e18, "principal still owed, unclaimable");
23 assertEquals(address(v).balance, 50e18, "nothing recovered");

```

The companion `test_A1_control_eoaCreatorRevokesFine` confirms an EOA creator settles the same flow normally.

Recommendation

Replace the creator push with a pull: accrue a `creatorOwed` balance and add a `claimCreatorFunds` function, or at least credit the owed amount on a failed push instead of reverting. Add a `revoked` guard to `setCreator` to match `setBeneficiary`.

Remediation Status

Remediated in 66af77611. Creator payouts became a pull: `_creditCreator` credits `creatorClaimable` and the creator withdraws through `claimCreatorFunds` to a recipient of their choosing. The three former push sites in `revoke`, `foundationSweep` and `_bookProfitOrLoss` now credit instead of transferring, so a creator that cannot receive value no longer blocks revocation or settlement. The finding stays *Open* against the audited snapshot.

3.4 [GTV-05] A write-once share floor strands the creator's claw-back behind the just-revoked beneficiary

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 963–971 and 1033–1041 (share floors), 836–846 and 921–934 (idle reserve)

Description

Two mechanisms reserve value for the beneficiary, and they are never netted against each other. `revoke` and `foundationSweep` reserve the beneficiary's absolute entitlement out of idle balance. Separately, `revokerDelegatorExit` and `revokerValidatorExit` reserve a proportional floor of $\text{shares} \times \text{vestedAtRevocation} / \text{totalAmountAtRevocation}$ on each staking target. That floor is snapshotted once and never lowered, and it ignores `totalWithdrawn`, so once the beneficiary has taken their vested share in cash the residual pool is economically the creator's but the floor still withholds a fraction of it.

Impact

Reproduced in the worst case: a 1,000e18 grant fully staked, 50% vested. `revoke` pays the creator nothing because idle balance is zero, the revoker can exit only the unvested half, and a second `revokerDelegatorExit` reverts because the sticky floor now equals the remaining shares. The creator recovers zero of a 500e18 clawback. The principal is not destroyed: the beneficiary can release it, since `vestingDelegatorExit` has no revoked guard, and it then routes to the creator. But release depends on cooperation from the party that was just revoked, and while the principal sits stranded the beneficiary keeps earning its share of all yield on it. A companion test measured 50e18 of a 100e18 yield extracted this way.

Proof of Concept

`test_D_creatorClawbackFullyStrandedBehindShareFloor` stakes a 1,000e18 grant fully, vests half, then walks the revoker through `revoke`, `exit`, `claim`, and `sweep` to show the creator recovers nothing of a 500e18 clawback while the write-once floor locks out the second exit.

```

1 // test/IndependentReview.t.sol :: test_D_creatorClawbackFullyStrandedBehindShareFloor
2 // setUp: 1000e18 grant, ERC1967 proxy over Vesting, 30-day linear schedule
3 // Beneficiary stakes the ENTIRE allocation, leaving zero idle.
4 vm.prank(beneficiary); v.vestingDelegatorJoin(VAL, TOTAL);
5 assertEq(address(v).balance, 0, "setup: nothing idle");
6
7 // Half vests, then the revoker revokes. Creator's rightful clawback = 500e18.
8 vm.warp(block.timestamp + 30 days);
9 vm.prank(revoker); v.revoke();
10 assertEq(v.vestedAtRevocation(), 500e18);
11 assertEq(creator.balance, 0, "idle was 0, so revoke paid the creator nothing");
12
13 // Revoker tries to claw back. The floor caps the exit at the UNVESTED half.
14 vm.prank(revoker); v.revokerDelegatorExit(VAL, TOTAL);
15 assertEq(v.delegationVestedFloor(VAL), 500e18, "floor snapshotted write-once at V/T");
16
17 stk.setPayout(500e18);
18 vm.prank(revoker); v.revokerDelegatorClaim(VAL);
19 assertEq(address(v).balance, 500e18, "500e18 recovered into the vesting");
20
21 // Sweep reserves the beneficiary's ABSOLUTE entitlement again; creator gets nothing.
22 vm.prank(revoker); v.foundationSweep();
23 assertEq(creator.balance, 0, "creator swept 0");
24
25 // Beneficiary takes their full vested share in cash.
26 vm.prank(beneficiary); v.vestingWithdraw(500e18);
27 assertEq(beneficiary.balance, 500e18);
28 assertEq(address(v).balance, 0);
29
30 // Remaining 500e18 is economically the creator's, but the sticky floor equals
31 // the remaining share count, so the second exit reverts.
32 vm.prank(revoker);
33 vm.expectRevert(IVesting.ZeroAmount.selector);
34 v.revokerDelegatorExit(VAL, 500e18);
35
36 assertEq(creator.balance, 0, "D: creator recovered NOTHING of a 500e18 clawback");
37 assertEq(v.depositedPerValidator(VAL), 500e18, "stranded principal still tracked");

```

The creator ends with a zero balance against a 500e18 clawback, the second `revokerDelegatorExit` reverts with `IVesting.ZeroAmount` because `delegationVestedFloor(VAL)` still equals the 500e18 of remaining shares, and `depositedPerValidator(VAL)` confirms that principal sits stranded in staking.

Recommendation

Compute the floor against the beneficiary's remaining unsatisfied entitlement, net of `totalWithdrawn` and of the idle already reserved, rather than the raw ratio. Alternatively drop the per-target floors and rely on `foundationSweep`'s absolute reserve together with the delegation reserve added in GTV-06.

Remediation Status

Remediated in 66af77611. The share floor is recomputed on each exit rather than written once, and `_requiredTargetBacking` subtracts the idle balance and any outstanding principal beyond the target from the beneficiary's remaining entitlement, which itself nets `totalWithdrawn`. Once the beneficiary has taken that entitlement in cash the floor releases and the creator's clawback is recoverable. The finding stays *Open* against the audited snapshot.

3.5 [GTV-06] The post-revocation reserve covers self-stake but not delegation principal

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 282–289 (`_outstandingValidatorPrincipal`), 312–313 (the reserve)

Description

After revocation, `_withdrawableAmount` reserves outstanding staked principal so a returned partial cannot be withdrawn before a later slash on the remainder is realized. That reserve sums `validatorDeposited`, which is self-stake only. Outstanding delegation principal, tracked in `depositedPerValidator` and `pendingExitDeposited`, is not reserved, even though a delegation carries the same slashing risk and the beneficiary's entitlement is frozen at `vestedAtRevocation` with no future vesting to absorb a late loss.

Impact

Reproduced against a control. On economically identical positions, post-revocation `withdrawableAmount` is zero via self-stake, correctly reserved, but the full 100e18 via delegation. In the full scenario, on a 200e18 grant with a 50% slash, the beneficiary nets 100e18 and the creator 50e18, where the fair pro-rata split is 75e18 each. 25e18 of loss is shifted onto the creator.

Proof of Concept

`test_poc_beneficiaryDodgesSlashShare` puts a 200e18 grant with 100e18 delegated, revokes at 50% vested, and shows the beneficiary withdrawing the full frozen entitlement before a 50% slash is realized on the unreserved delegation principal.

```

1 // test/VestingRevokeReserve.t.sol :: test_poc_beneficiaryDodgesSlashShare
2 // setUp: 200e18 grant, 2x30d periods => 50% vested at revoke, 100e18 delegated to VAL
3 vm.prank(beneficiary); v.vestingDelegatorJoin(VAL, 100e18);
4 vm.deal(address(stk), 1000e18); // staking reserve to pay out
5 vm.warp(block.timestamp + 30 days);
6 vm.prank(revoker); v.revoke();
7
8 // 1) Beneficiary withdraws the FULL vested entitlement before the loss is realized.
9 vm.prank(beneficiary); v.vestingWithdraw(100e18);
10 assertEq(beneficiary.balance, 100e18, "beneficiary took full vested share");
11
12 // 2) The still-staked delegation is slashed 50%.
13 stk.setPayoutBps(5000);
14 vm.prank(beneficiary); v.vestingDelegatorExit(VAL, 100e18);
15 vm.prank(beneficiary); v.vestingDelegatorClaim(VAL);
16 assertEq(v.postRevocationBeneficiaryLosses(), 25e18, "loss booked pro-rata");
17 assertEq(v.withdrawableAmount(), 0, "nothing left to claw back from beneficiary");
18
19 // 3) Creator sweeps what remains.
20 uint256 cBefore = creator.balance;
21 vm.prank(revoker); v.foundationSweep();
22 uint256 creatorGot = creator.balance - cBefore;
23 assertEq(beneficiary.balance, 100e18, "beneficiary kept full pre-slash entitlement");
24 assertEq(creatorGot, 50e18, "creator absorbed the beneficiary's 25e18 loss share");

```

The control pair confirms an identical self-stake position is reserved (`test_selfStake_isReserved` holds `withdrawableAmount` at 0) while the delegation is not (`test_delegation_isNotReserved` leaves the full 100e18 withdrawable).

Recommendation

Include outstanding delegation principal in the reserve. This requires tracking delegation targets in an enumerable structure, as validator wallets already are, or maintaining a running `totalDelegationOutstanding` scalar updated on join and on the principal consumed at claim.

Remediation Status

Remediated in 66af77611. The post-revocation reserve calls `_outstandingStakingPrincipal`, which returns `totalDelegationOutstanding` plus the self-stake total, so delegation principal is reserved alongside self-stake. The finding stays *Open* against the audited snapshot.

3.6 [GTV-07] An ownership handover leaves the deployer holding unlimited mint authority

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in c382be97

Code Location

- `src/GEN.sol`: 148–159 (initialize), 475–487 (role grants)

Description

`initialize` sets the Ownable owner and grants `DEFAULT_ADMIN_ROLE` and

MINTER_ROLE to the same address. These are two separate authority roots with no relationship. `transferOwnership` moves only the Ownable root, and GEN calls `_revokeRole` nowhere, so a handover to a multisig leaves the deploying key with an uncapped mint and the ability to grant minting to anyone. `renounceOwnership` would not disarm it either.

Impact

Reproduced: after `transferOwnership` to a fresh owner, the deployer minted 4.2e27 GEN. The related rotation problem was also reproduced: `setGenLayerStaking` grants MINTER_ROLE and BURNER_ROLE on every call without revoking the previous holder, so a rotated-away address kept an unlimited mint and minted 1,000,000e18. The rotation case is mitigated in production because `genLayerStaking` and `timelockFactory` are L2 addresses that cannot be the L1 `msg.sender` absent an address collision, which is why it is rated lower on its own in GTV-15; the handover case has no such mitigation.

Proof of Concept

`test_P7_rotationLeavesStaleMinter_andHandoverLeavesMintKey` transfers ownership to a fresh multisig and shows the deployer still mints without limit.

```
1 // test/AssessmentProbes.t.sol :: test_P7_rotationLeavesStaleMinter_andHandoverLeavesMintKey
2 // setUp: GEN deployed behind ERC1967Proxy, initialize() grants deployer MINTER + admin
3
4 // Ownership handover moves ONLY the Ownable root; the deployer keeps MINTER + admin.
5 address multisig = address(0x0501);
6 gen.transferOwnership(multisig);
7 assertEq(gen.owner(), multisig, "ownership moved");
8
9 address deployer = address(this);
10 assertTrue(gen.hasRole(gen.MINTER_ROLE(), deployer), "P7b: deployer STILL holds MINTER");
11 assertTrue(gen.hasRole(gen.DEFAULT_ADMIN_ROLE(), deployer), "P7b: and role-admin");
12 gen.mint(deployer, 4.2e27);
13 assertEq(gen.balanceOf(deployer), 4.2e27, "P7b: post-handover unlimited mint still live");
```

Recommendation

Move the AccessControl roles in the same procedure as ownership, and add a revoke path. Consider collapsing to a single authority root by gating the setters and `_authorizeUpgrade` on DEFAULT_ADMIN_ROLE and dropping Ownable, or at minimum adopt Ownable2Step to match VestingFactory. Script the handover so ownership transfer and role migration are one atomic operation.

Remediation Status

Remediated in the reference tree c382be97, verified against GEN.sol SHA-256 b02f1bfd...59e1. `transferOwnership` is overridden to grant DEFAULT_ADMIN_ROLE and MINTER_ROLE to the incoming owner and revoke them from the outgoing one in the same call, and `renounceOwnership` now reverts with `OwnershipRenunciationDisabled`, so authority cannot be abandoned while privileged configuration is live. The minter role is deliberately retained when the outgoing owner is also a configured staking or timelock endpoint, which is

correct. The finding stays *Open* against the audited snapshot.

3.7 [GTV-08] A partially-matured delegation claim books a phantom loss and orphans the remaining tranche

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 432–458 (`vestingDelegatorClaim`), 990–1013 (`revokerDelegatorClaim`), compare 596–606 (`_claimValidator`)

Description

`pendingExitDeposited` accumulates across exits with `+=`. The claim reads the whole accumulated figure, compares it against only what the claim returned, and then zeroes the tracker unconditionally. If only part of the earmarked principal has matured, the claim books a loss that never happened and clears the tracker, so the later-maturing tranche short-circuits at `deposited == 0` and can never be pulled. The validator path handles this correctly 140 lines later, taking the minimum of returned and outstanding and keeping the residual; the delegation path was never given the same treatment.

Impact

Reproduced: two 100e18 exits, first matures, a 100e18 loss is booked that did not occur, the tracker is cleared, and the second 100e18 is permanently stranded in staking while the beneficiary's 100e18 vested entitlement is wiped to zero withdrawable. Both reviewers who reached this independently noted the orphaning half is certain regardless of tranche timing: any divergence that leaves the tracker at zero while staking still owes a withdrawal makes those funds unreachable.

Premise confirmed against the L2 source

This finding was originally rated conditional. GenLayer subsequently supplied a reference tree containing the staking implementation, and it confirms the condition. `DelegatorUtils.delegatorClaim` loops the withdrawal array and pays a tranche only when `commitWithdrawal.priced` holds and `commitWithdrawal.epoch <= _epoch - delegatorUnbondingPeriod`; every other tranche is copied back as still pending. `delegatorExit` pushes a new claim whenever the commit epoch differs, so exits in different epochs are genuinely separate tranches, and the default unbonding period of seven epochs makes the window between maturities wide. Staggered maturity is therefore the ordinary case, not a hypothetical, and the severity is raised accordingly.

A second failure in the opposite direction

The same missing reconciliation fails the other way when a claim returns nothing. The early exit at `Vesting.sol:446-450` reads a zero return as an unmatured claim and books no loss, which is correct for a premature call. But the reference staking states that a priced zero payout is final: a tranche whose output has been driven to zero is consumed and its shares are retired, so it can never pay again. Vesting keeps `pendingExitDeposited` non-zero, never books the loss, and no retry can correct it because staking has nothing left to settle.

Reproduced on a 1,000e18 grant with 400e18 vested and 100e18 delegated, then fully slashed. `accumulatedLosses` stays at zero, the tracker still claims 100e18 is owed, the beneficiary withdraws the full 400e18 as though nothing was lost, and `revoke` leaves the creator with 500e18. With the loss booked the beneficiary is entitled to 300e18 and the creator sweeps 600e18, so the entire slashed principal moves from creator to beneficiary. A control test confirms the identical slash delivered as a partial return books correctly, isolating the defect to the zero-return path.

Escalation after revocation

`revokerDelegatorExit` writes into the same tracker and `vestingDelegatorExit` carries no revoked guard, so once a vesting is revoked the revoker can create the staggered state deliberately. The resulting phantom loss migrates into `postRevocationBeneficiaryLosses` and reduces the beneficiary's frozen entitlement, which is the outcome the vested-floor machinery exists to prevent.

Proof of Concept

`test_M3_staggeredMaturity_spuriousLoss_and_permanentStranding` splits one delegation into two 100e18 exits, matures only the first, and shows the claim booking a phantom loss, clearing the tracker, and permanently stranding the second tranche.

```

1 // test/VestingConditional.t.sol ::
    test_M3_staggeredMaturity_spuriousLoss_and_permanentStranding
2 // setUp: 200e18 delegator join to V1, then stk.setEpoch(1)
3
4 // two exits of 100e18 each -> pendingExitDeposited accumulates to 200e18
5 vm.prank(beneficiary); v.vestingDelegatorExit(V1, 100e18);
6 vm.prank(beneficiary); v.vestingDelegatorExit(V1, 100e18);
7 assertEq(v.pendingExitDeposited(V1), 200e18, "both exits earmarked");
8
9 // only the FIRST tranche has matured
10 stk.setNextPayout(100e18);
11 vm.prank(beneficiary); v.vestingDelegatorClaim(V1);
12
13 assertEq(v.accumulatedLosses(), 100e18, "M-3: SPURIOUS loss booked");
14 assertEq(v.pendingExitDeposited(V1), 0, "M-3: tracker wrongly cleared");
15 assertEq(stk.pending(address(v), V1), 100e18, "staking still owes 100e18");
16
17 // the later-maturing tranche can never be pulled: claim short-circuits at deposited==0
18 uint256 balBefore = address(v).balance;
19 vm.prank(beneficiary); v.vestingDelegatorClaim(V1);
20 assertEq(address(v).balance, balBefore, "M-3: second tranche PERMANENTLY stranded");
21
22 vm.warp(block.timestamp + 30 days); // 100e18 vested
23 assertEq(v.withdrawableAmount(), 0, "vested 100e18 wiped out by the phantom 100e18 loss");

```

The companion `test_pricedZeroTranche_lossNeverBooked_creatorAbsorbsIt` shows the opposite failure where a priced-zero payout books no loss, and `test_control_partialReturnIsBooked` confirms the same slash delivered as a partial return books correctly.

Recommendation

Mirror the *semantics* of `_claimValidator`, which already handles both cases correctly, rather than its control flow: that function writes `validatorDeposited` twice and reads back its own first write across three external calls, which is safe only because the calls are static and every caller is guarded, and it should not be copied in that shape. Consume `min(returned, deposited)`, retain the residual in `pendingExitDeposited`, and book a loss only once the delegation is provably settled. The staking read facet exposes a delegator equivalent of `validatorWithdrawalLen`, which `IVestingStaking` does not currently declare; declaring it would let the delegation path use the same settled test the validator path uses. One change fixes both failures, since the phantom loss and the unbooked loss stem from the same missing reconciliation.

Remediation Status

Remediated in 66af77611. `_claimDelegation` enumerates the withdrawal tranches before and after the claim, matches them by commit epoch, and settles only the principal of the tranches that actually cleared, subtracting that amount from `pendingExitDeposited` and booking profit or loss against it. The unconditional zeroing and the full-`deposited` loss are both gone. The finding stays *Open* against the audited snapshot.

3.8 [GTV-09] The bridge fee is uncapped net-new supply and its reclaim path is unimplemented

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in f6230f7e8

Code Location

- `src/GEN.sol`: 441 (`fees`), 451, 546 (`_sendToL2` mint), 303–307 (BURN arm), 391–398 (header comment), 17–25 and 66–81 (declared, unused)

Description

`fees = baseCost * 16 / 10` is minted as net-new supply on every trigger, where `baseCost` depends on the gas parameters carried inside the L2 message, with no bound in absolute terms or relative to the delivered inflation. Two identical single-epoch inflations differing only in message gas price minted amounts differing by exactly 1.6 times the base-cost difference. Separately, the reclaim the header comment describes is not implemented: `_sendToL2` mints and lets the bridge pull the whole amount, so GEN ends every inflation at a zero balance, and a later BURN reverts unless an external actor has already finalized the L2 to L1 refund, because GEN never calls `finalizeWithdrawal` and the `ISharedBridge`, `IMailbox`, and `ZKSYNC_SHARED_BRIDGE` declarations are unused.

Impact

On every trigger the unused L2 gas plus the 60% buffer is unbacked supply, and the offsetting burn depends on two uncoordinated off-chain steps with no on-chain sequencing. At sane gas parameters the per-trigger amount is small next to the epoch inflation, so the defect is the missing cap and the missing on-chain reclaim rather than a large single loss.

Proof of Concept

`test_V3_feesScaleUnboundedlyWithMessageGasParams` runs two otherwise identical single-epoch inflations that differ only in the gas price carried inside the L2 message, and measures the resulting net-new GEN supply.

```

1 // test/VerificationPass.t.sol :: test_V3_feesScaleUnboundedlyWithMessageGasParams
2 // --- low gas price ---
3 (GEN g1,) = _fresh();
4 g1.executeL2Message(1, 0, 0,
5     abi.encode(IGEN.GenAction.INFLATION, uint256(1), uint256(1e9), uint256(1_000_000)),
6     emptyProof, 1e9, 1_000_000);
7 uint256 mintedLow = g1.totalSupply();
8
9 // --- same epoch, 1000x gas price ---
10 (GEN g2,) = _fresh();
11 g2.executeL2Message(1, 0, 0,
12     abi.encode(IGEN.GenAction.INFLATION, uint256(1), uint256(1e12), uint256(1_000_000)),
13     emptyProof, 1e12, 1_000_000);
14 uint256 mintedHigh = g2.totalSupply();
15
16 // The inflation component is identical (same epoch); the whole delta is fee mint.
17 assertGt(mintedHigh, mintedLow, "V3: fee mint scales with message-supplied gas price");
18 // baseCost delta = (1e12-1e9)*1e6, fees = 1.6x that.
19 assertEq(mintedHigh - mintedLow, ((1e12 - 1e9) * 1_000_000 * 16) / 10,
20     "V3: delta is exactly 1.6x the baseCost delta - unbounded and uncapped");

```

The test passes: the extra net-new supply is 1.5984e18, exactly $((1e12 - 1e9) * 1000000 * 16) / 10$, confirming fees is minted as uncapped supply driven by message-supplied gas params; the reclaim half is exercised by the companion BURN-arm tests showing GEN ends every inflation at a zero balance with no on-chain `finalizeWithdrawal`.

Recommendation

Cap fees relative to the delivered inflation, so a gas quote driven by message-supplied parameters cannot mint an unbounded buffer:

```

1 uint256 fees = (_l2BaseGasCost * 16) / 10;
2
3 // NEW: bound the buffer against the value actually being delivered.
4 uint256 feeCap = (amountToStaking * maxFeeBps) / 10000;
5 if (fees > feeCap) revert InflationFeeExceedsCap();

```

Then either wire the finalization into the BURN arm so the round trip closes on-chain, or remove the misleading interfaces and comment and document the two-step operational dependency with monitoring. If the reclaim stays off-chain, track what is owed so the outstanding amount is derivable from state rather than only from the operator's records:

```

1 // Accrue on every send; decrease in the BURN arm as excess returns.
2 uint256 public outstandingFeeBuffer;

```

Remediation Status

Remediated in f6230f7e8, which closes all three parts. The fee is capped: `_inflation` derives `feeBudget` from the inflation being delivered at `MAX_INFLATION_FEE_BPS` of 1,000 and reverts `InflationFeeExceedsCap` above it. The reclaim path is implemented: `finalizeStakingBurnWithdrawal` and `finalizeFeeBurnWithdrawal` prove the L2 withdrawal message, finalize it into a dedicated `GENBurnCustody`, check the exact credit and burn it, with receipt ids nullified against replay. The refund now lands where the contract can act on it: `refundRecipient` is an attested L2 escrow, and principal the delivery

did not consume is re-sent through `submitInflationReconciliation`. The finding stays *Open* against the audited snapshot.

3.9 [GTV-10] A genesis-epoch delegation exit books no profit or loss

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 400–422 (`vestingDelegatorExit`), compare 568–575 (`vestingValidatorExit`)

Description

When the staking epoch is zero, an exit settles synchronously and the principal returns to the vesting balance in the same call. `vestingDelegatorExit` decrements its tracker but skips the pending bookkeeping and never measures the balance delta, so `accumulatedRewards` and `accumulatedLosses` are untouched, and a later claim early-returns at `deposited == 0` without reconciling. The validator path auto-claims at epoch zero to book its profit or loss; the delegation path does not.

Impact

On a 1,000e18 grant with 40% vested and 600e18 delegated, a 50% slash realized through a genesis-epoch exit leaves `accumulatedLosses` at zero, so the beneficiary withdraws the full 400e18 loss-free and the creator receives 300e18 instead of 600e18, shifting 300e18, or 30% of the grant, from creator to beneficiary. A control test books the same slash correctly at epoch one, isolating the defect to the genesis path. The mirror case shorts the beneficiary: genesis-epoch yield is never credited and is swept to the creator.

Deciding question for the L2 source

Does epoch zero carry real yield or slashing? If the genesis window has strictly no profit or loss, the impact drops to informational.

Proof of Concept

`test_P3_epochZeroExitBooksNoPnL_creatorAbsorbsTheLoss` realizes a 50% slash through a genesis-epoch delegator exit and shows the loss is never booked, shifting 300e18 from creator to beneficiary.

```

1 // test/AssessmentProbes.t.sol :: test_P3_epochZeroExitBooksNoPnL_creatorAbsorbsTheLoss
2 Vesting v = _deploy(1000e18, 5, 0);
3 stk.setEpoch(0); // genesis window: synchronous exits
4
5 vm.prank(beneficiary); v.vestingDelegatorJoin(VAL, 600e18);
6 vm.warp(block.timestamp + 60 days);
7 assertEq(v.vestedAmount(), 400e18, "setup: 40% vested");
8
9 // A 50% slash is realized through an epoch-0 (synchronous) exit.
10 stk.setYieldBps(-5000);
11 vm.prank(beneficiary); v.vestingDelegatorExit(VAL, 600e18);
12
13 assertEq(v.accumulatedLosses(), 0, "P3: the 300e18 loss was NEVER booked");
14 assertEq(v.withdrawableAmount(), 400e18, "P3: withdrawable ignores the slash");
15 vm.prank(beneficiary); v.vestingWithdraw(400e18);
16
17 vm.prank(revoker); v.revoke();
18 assertEq(creator.balance, 300e18, "creator receives only 300e18");
19 assertEq(beneficiary.balance, 400e18, "beneficiary took 400e18 instead of 100e18");

```

The companion control `test_P4_control_epochOneSlashIsBookedCorrectly` books the identical slash as 300e18 of `accumulatedLosses` at epoch one.

Recommendation

Measure the balance delta around the synchronous exit and call `_bookProfitOrLoss` when the epoch is zero, mirroring the validator path.

Remediation Status

Remediated in 66af77611. Both genesis paths settle: `_exitDelegation` books profit or loss inline when the epoch is zero, and `_claimValidatorEpochZero` measures the synchronous refund before settling principal. The finding stays *Open* against the audited snapshot.

3.10 [GTV-11] Revocation splits recovered gains but migrates prior losses in full, shorting the beneficiary

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 826–833 (loss migration in `revoke`), 465–489 (`_bookProfitOrLoss`)

Description

A loss realized before revocation is borne entirely by the beneficiary, held in `accumulatedLosses` and migrated at full weight into the post-revocation bucket by `revoke`. A gain realized after revocation is split between beneficiary and creator on the vested ratio. When a loss and an equal recovery straddle the revocation, the two rules do not cancel.

Impact

A 50e18 loss before revocation and an equal 50e18 gain after it, a net-zero staking result, leaves the beneficiary 25e18 short while the creator collects 25e18 in cash. Each rule is documented as intended, so this is an emergent consequence at the boundary rather than a coding slip.

Proof of Concept

`test_NEW_revokeStraddle_lossThenEqualRecovery_shortsBeneficiary` realizes a 50e18 loss before `revoke` and an equal 50e18 gain after, then shows the loss migrates in full while the recovery is split.

```

1 // test/VestingConditional.t.sol ::
2   test_NEW_revokeStraddle_lossThenEqualRecovery_shortsBeneficiary
3 // setUp: _deploy() 200e18 grant, warp 30 days -> 100e18 (50%) vested at revoke
4 // --- pre-revoke: realize a 50e18 loss on V1 ---
5 vm.prank(beneficiary); v.vestingDelegatorJoin(V1, 100e18);
6 stk.setEpoch(1);
7 vm.prank(beneficiary); v.vestingDelegatorExit(V1, 100e18);
8 stk.setNextPayout(50e18); // 50% slash
9 vm.prank(beneficiary); v.vestingDelegatorClaim(V1);
10 assertEq(v.accumulatedLosses(), 50e18, "pre-revoke loss borne 100% by beneficiary");
11
12 // --- stake V2, then revoke ---
13 vm.prank(beneficiary); v.vestingDelegatorJoin(V2, 50e18);
14 vm.prank(revoker); v.revoke();
15 assertEq(v.postRevocationBeneficiaryLosses(), 50e18, "loss migrated at FULL weight");
16
17 // --- post-revoke: an equal 50e18 gain on V2 ---
18 vm.prank(beneficiary); v.vestingDelegatorExit(V2, 50e18);
19 uint256 creatorBefore = creator.balance;
20 stk.setNextPayout(100e18); // 50e18 principal + 50e18 profit
21 vm.prank(beneficiary); v.vestingDelegatorClaim(V2);
22 assertEq(v.postRevocationBeneficiaryRewards(), 25e18, "beneficiary got only HALF the recovery");
23
24 assertEq(creator.balance - creatorBefore, 25e18, "creator pocketed the other half");

```

Net staking P&L is zero (-50e18 then +50e18), yet `postRevocationBeneficiaryLosses` holds 50e18 against a `postRevocationBeneficiaryRewards` of 25e18.

Recommendation

Treat pre-revocation profit or loss symmetrically with post-revocation profit or loss at the migration point, or net realized results across the revocation boundary before applying the split, so a loss and an equal recovery cancel.

Remediation Status

Remediated in 66af77611. `revoke` nets pre-revocation profit against loss before migrating it and records a signed `postRevocationNetCarry`. `_bookProfitOrLoss` then applies a later delta against that carry one for one before splitting any remainder on the vested ratio, so a loss and an equal recovery straddling revocation cancel. The finding stays *Open* against the audited snapshot.

3.11 [GTV-12] Stopping then revoking traps the stopped flag and disables the beneficiary's settlement

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 904–906 (`resumeVesting`), 438 and 585 (claim guards), 572 (epoch-zero auto-claim), 990 and 1058 (revoker claims, no guard)

Description

`revoke` never clears `vestingStopped`, and `resumeVesting` reverts once the contract is revoked (`Vesting.sol`:904–906). Both beneficiary claim paths revert while stopped (438, 585), so with both flags set no transition can clear the stop and the beneficiary can never settle a staked position. The revoker claim paths carry no stopped guard (990, 1058), so only the revoker can settle it.

Impact

With 600e18 awaiting settlement, the beneficiary's claim and `resumeVesting` both revert while the revoker settles the same position. This is a liveness lever, not a direct loss: idle balance stays withdrawable, but the revoker can hold the beneficiary's staked entitlement hostage indefinitely.

Proof of Concept

`test_P5_stopThenRevoke_permanentlyDisablesBeneficiarySettlement` stops then revokes with 600e18 awaiting settlement, shows `vestingStopped` is never cleared, that both the beneficiary claim and `resumeVesting` revert, and that only the revoker can settle the position.

```

1 // test/AssessmentProbes.t.sol ::
    test_P5_stopThenRevoke_permanentlyDisablesBeneficiarySettlement
2 // setUp: 1000e18 grant, proxy at v, ProbeStaking stk, ProbeAM am
3
4 vm.prank(beneficiary); v.vestingDelegatorJoin(VAL, 600e18);
5 vm.warp(block.timestamp + 30 days);
6 vm.prank(beneficiary); v.vestingDelegatorExit(VAL, 600e18);
7 assertEq(v.pendingExitDeposited(VAL), 600e18, "setup: 600e18 awaiting settlement");
8
9 // Revoker stops, then revokes.
10 vm.prank(revoker); v.stopVesting();
11 vm.prank(revoker); v.revoke();
12 assertTrue(v.vestingStopped(), "P5: vestingStopped is NEVER cleared by revoke()");
13
14 // No transition can clear it: resumeVesting is gated on !revoked.
15 vm.prank(revoker);
16 vm.expectRevert(IVesting.AlreadyRevoked.selector);
17 v.resumeVesting();
18
19 // The beneficiary can therefore never settle their own staked position.
20 vm.prank(beneficiary);
21 vm.expectRevert(IVesting.VestingAlreadyStopped.selector);
22 v.vestingDelegatorClaim(VAL);
23
24 // Only the revoker can, since the revoker claim paths carry no 'vestingStopped' guard.
25 stk.setPayout(600e18);
26 vm.prank(revoker); v.revokerDelegatorClaim(VAL);
27 assertEq(v.pendingExitDeposited(VAL), 0, "P5: only the revoker could settle it");

```

The beneficiary's `vestingDelegatorClaim` reverts with `VestingAlreadyStopped` and `resumeVesting` reverts with `AlreadyRevoked`, while `revokerDelegatorClaim` drains `pendingExitDeposited(VAL)` from 600e18 to 0, so only the revoker settles the trapped position.

Recommendation

Clear `vestingStopped` inside `revoke`, or drop the stopped guard from the claim paths once the contract is revoked, since the post-revocation split already governs how gains are shared and the freeze rationale no longer applies.

Remediation Status

Remediated in 66af77611. `revoke` clears `vestingStopped` and `vestingStoppedAt`, so the state in which neither party can settle is unreachable. The finding stays *Open* against the audited snapshot.

3.12 [GTV-13] A two-step operator transfer defeats the distinct-operator check

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 656–667 (`_requireDistinctOperator`), 690–697 (`vestingValidatorCompleteOperatorTransfer`)

Description

`_requireDistinctOperator` (Vesting.sol:656–667) compares a proposed operator against each wallet’s current operator, and `vestingValidatorCompleteOperatorTransfer` (690–697) re-checks nothing. Two wallets in the same vesting can each initiate a transfer to the same new operator: each scan sees the other wallet’s current operator, not its pending one, so both pass, and completing both leaves the wallets sharing one operator. This defeats the uniqueness constraint that limits correlated slashing.

Impact

Staking rejects a duplicate operator at join time, but the rotation path slips past and both wallets end on one operator. The staking layer does not re-check uniqueness on rotation, so nothing else catches it.

Deciding question for the L2 source

Does the validator wallet’s `getOperator` return the current operator rather than a pending one? The reproduction assumes it does.

Proof of Concept

`test_V2_dualPendingTransfer_defeatsDistinctOperatorCheck` joins two vesting-owned wallets, shows staking rejecting a duplicate operator at join time (the control), then rotates both wallets to the same operator X through the two-step path.

```

1 // test/VerificationPass.t.sol :: test_V2_dualPendingTransfer_defeatsDistinctOperatorCheck
2 // setUp: proxy Vesting, vm.deal 100e18, beneficiary owns the escrow
3 vm.prank(beneficiary); v.vestingValidatorJoin(A1, 10e18);
4 vm.prank(beneficiary); v.vestingValidatorJoin(B1, 10e18);
5 address w1 = v.validatorWallets(0);
6 address w2 = v.validatorWallets(1);
7
8 // CONTROL: staking DOES reject a duplicate operator at join time.
9 vm.prank(beneficiary);
10 vm.expectRevert(bytes("operator taken"));
11 v.vestingValidatorJoin(A1, 10e18);
12
13 // But the two-step rotation slips past: each scan sees the OTHER wallet's CURRENT operator.
14 vm.prank(beneficiary); v.vestingValidatorInitiateOperatorTransfer(w1, X);
15 vm.prank(beneficiary); v.vestingValidatorInitiateOperatorTransfer(w2, X);
16
17 // complete() re-runs no check at all.
18 vm.prank(beneficiary); v.vestingValidatorCompleteOperatorTransfer(w1);
19 vm.prank(beneficiary); v.vestingValidatorCompleteOperatorTransfer(w2);
20
21 assertEq(OpWallet(payable(w1)).getOperator(), X, "w1 rotated to X");
22 assertEq(OpWallet(payable(w2)).getOperator(), X, "w2 rotated to X");

```

After the two rotations both `getOperator()` calls return X, so one operator runs both vesting-owned validators.

Recommendation

Compare against pending operators as well as current ones, re-run the distinctness check inside the complete step, and add it to the join path as defense in depth. Note the check still only spans one vesting’s wallets; cross-vesting reuse is covered in GTV-26.

Remediation Status

Remediated in 66af77611. `_requireDistinctOperator` compares against each other wallet's pending operator through `getPendingOperator` as well as its current one, and `vestingValidatorCompleteOperatorTransfer` re-runs the check at completion. The finding stays *Open* against the audited snapshot.

3.13 [GTV-14] A one-wei squat blocks a beneficiary's grant and the clear is not atomic

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/VestingFactory.sol`: 97 and 175–178 (permissionless create, set-once mapping), 285–294 (`clearBeneficiaryRegistration`)

Description

`createVesting` is permissionless (`VestingFactory.sol:97`), and a 1-wei amount passes the zero check and claims the set-once `beneficiaryToVesting[b]` (175–178), blocking the real grant. The owner's `clearBeneficiaryRegistration` (285–294) runs in a separate transaction, so the squatter re-squats by front-running the follow-up create.

Impact

The funded grant reverts with `VestingAlreadyExists`, and because the clear is not atomic the attacker re-squats before it lands again. Cleared squats are orphaned rather than removed, so `isVestingContract` stays true for them and `allVestings` grows unboundedly. One already-squatted slot reverts an entire `createBatchVesting`.

Proof of Concept

`test_P8_squatBlocksGrant_andClearIsNotAtomic` shows a 1-wei create claiming the victim's slot, the funded grant reverting, and the non-atomic clear letting the attacker re-squat before the grant lands again.

```

1 // test/AssessmentProbes.t.sol :: test_P8_squatBlocksGrant_andClearIsNotAtomic
2 // setUp: factory behind proxy, blueprint + beacon set; victim, attacker, foundation
3 vm.prank(attacker);
4 address squat = f.createVesting{value: 1}{
5     "squat", victim, attacker, block.timestamp, 0, 30 days, 10, 0, false, 1, IVesting.Category
6     .Others
7 };
8
9 assertEq(f.getVesting(victim), squat, "P8: attacker owns the slot");
10
11 vm.prank(foundation);
12 vm.expectRevert(IVesting.VestingAlreadyExists.selector);
13 f.createVesting{value: 10 ether}{
14     "grant", victim, foundation, block.timestamp, 0, 30 days, 10, 0, false, 10 ether, IVesting
15     .Category.Others
16 };
17
18 // Owner clears in a SEPARATE tx, so the attacker re-squats before the follow-up lands.
19 f.clearBeneficiaryRegistration(victim);
20 vm.prank(attacker);
21 f.createVesting{value: 1}("squat2", victim, attacker, block.timestamp, 0, 30 days, 10, 0,
22     false, 1, IVesting.Category.Others);
23
24 vm.prank(foundation);
25 vm.expectRevert(IVesting.VestingAlreadyExists.selector);
26 f.createVesting{value: 10 ether}{
27     "grant", victim, foundation, block.timestamp, 0, 30 days, 10, 0, false, 10 ether, IVesting
28     .Category.Others
29 };
30
31 // The cleared squat is orphaned, not cleaned up.
32 assertTrue(f.isVestingContract(squat), "P8: isVestingContract stays true for the orphan");
33 assertEq(f.getAllVestingsCount(), 2, "P8: allVestings grows unboundedly with squats");

```

The companion `test_batch_existingSlotBlocksBatch` confirms a single squatted slot reverts an entire `createBatchVesting`.

Recommendation

Make the clear atomic with re-creation, for example an owner-only `createVestingFor` that overwrites, or gate creation behind an allowlist or a minimum registration amount, and clean up `isVestingContract` and `allVestings` when a registration is cleared.

Remediation Status

Remediated in 66af77611. `createVesting` no longer reverts on an occupied beneficiary slot, so the first registration is only a default. Recovery is atomic through `claimBeneficiaryRegistration`, which the beneficiary calls to select any of their own grants, leaving no owner-mediated clear and no intermediate empty slot to re-squat. The finding stays *Open* against the audited snapshot.

3.14 [GTV-15] Role rotation grants without revoking the previous holder

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated in c382be97

Code Location

- `src/GEN.sol`: 475–480 (`setGenLayerStaking`), 482–487 (`setTimelockFactory`)

Description

Both setters grant `MINTER_ROLE` and `BURNER_ROLE` to the new address and never revoke the old one, and `GEN` contains no `_revokeRole` call. Rotating either address, the expected response to an L2 contract upgrade, leaves every previous holder with an unlimited mint.

Impact

After two `setGenLayerStaking` calls, the first address still minted 1,000,000e18. Severity is limited because these addresses are L2-resident and cannot be the L1 `msg.sender` absent an L1/L2 address collision, so any future L1-controllable grantee would reinstate the risk. This pairs with the handover problem in GTV-07.

Proof of Concept

`AssessmentProbes.t.sol::test_P7_rotationLeavesStaleMinter_andHandoverLeavesMintKey` rotates the staking endpoint and shows the outgoing address keeps an unlimited mint.

```
1 // test/AssessmentProbes.t.sol :: test_P7_rotationLeavesStaleMinter_andHandoverLeavesMintKey
2 // setUp: GEN deployed behind ERC1967Proxy, initialize()
3 address oldStaking = address(0x5741);
4 address newStaking = address(0x5742);
5
6 gen.setGenLayerStaking(oldStaking);
7 gen.setGenLayerStaking(newStaking); // rotation
8
9 // The OUTGOING address keeps unlimited mint forever, no _revokeRole anywhere in GEN.
10 assertTrue(gen.hasRole(gen.MINTER_ROLE(), oldStaking), "P7a: stale grant retained");
11 vm.prank(oldStaking);
12 gen.mint(oldStaking, 1_000_000e18);
13 assertEquals(gen.balanceOf(oldStaking), 1_000_000e18, "P7a: rotated-away address still mints");
```

The rotated-away `oldStaking` address still holds `MINTER_ROLE`; the same test's companion assertions show the deployer retaining `MINTER_ROLE` and `DEFAULT_ADMIN_ROLE` after `transferOwnership`, the handover gap noted in GTV-07.

Recommendation

Revoke from the outgoing address before granting to the new one, reject the zero address, and drop the L1 role grants entirely for addresses that only ever act on L2.

Remediation Status

Remediated in the reference tree `c382be97`, verified against `GEN.sol` SHA-256 `b02f1bfd...59e1`: rotation routes through `_revokeEndpointRoleIfUnused`, which revokes the outgoing address's minter and burner roles once it is no longer a configured endpoint. The finding stays *Open* against the audited snapshot.

3.15 [GTV-16] Rotating a role leaves the factory's registry describing the wrong parties

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/VestingFactory.sol`: 213–215 (indexes written at creation), 298–301 (`getVesting`), 306–312 (`getValidatorWallets`), 314–320 (`getMyVestings`, `getMyVestingCount`)
- `src/Vesting.sol`: 1180–1189 (`setBeneficiary`), 1194–1199 (`setCreator`)

Description

The factory keeps two indexes, `beneficiaryToVesting` and `creatorVestings`, and writes both once when a vesting is created. The escrow lets those same two roles be rotated afterwards, and neither `setBeneficiary` nor `setCreator` calls back into the factory, so the registry keeps describing the parties who held those roles at creation.

Impact

After the beneficiary rotates, `getVesting` still resolves the original beneficiary and returns nothing for the party the escrow now pays; `getValidatorWallets` reads the same mapping and drifts the same way. The original beneficiary's slot stays occupied, so a re-grant to them reverts with `VestingAlreadyExists`, though the owner can clear that. The new beneficiary's slot was never taken, so a second escrow can be created naming them: both escrows then pay that party and `getVesting` returns only the newer one, breaking the one-vesting-per-beneficiary property through a call the beneficiary makes alone. Creator rotation is less visible: `creatorVestings` still lists the vesting under the previous creator, so the new creator cannot enumerate what they control.

Proof of Concept

`test_setBeneficiary_breaksCanonicalRegistry` rotates the beneficiary with a call `bob` makes alone, then shows the registry drifts both ways and the freed slot lets a second escrow be created for the same party.

```

1 // test/FactoryRegistryDrift.t.sol :: test_setBeneficiary_breaksCanonicalRegistry
2 // setUp: factory + Vesting blueprint; _create mints a grant via createVesting
3 address v = _create(alice, bob, 10 ether);
4 assertEq(f.getVesting(bob), v, "registry knows Bob");
5
6 vm.prank(bob);
7 Vesting(payable(v)).setBeneficiary(carol);
8 assertEq(Vesting(payable(v)).beneficiary(), carol, "escrow now pays Carol");
9
10 // The registry still points Bob at a vesting he no longer benefits from...
11 assertEq(f.getVesting(bob), v, "STALE: Bob still mapped to the vesting");
12 // ...and does not know Carol has one at all.
13 assertEq(f.getVesting(carol), address(0), "MISSING: Carol not in the registry");
14
15 // Consequence 1: Carol's slot is free, so a SECOND vesting can be created for her.
16 address v2 = _create(address(0xF0), carol, 5 ether);
17 assertTrue(v2 != v, "a second escrow now pays Carol");
18 assertEq(Vesting(payable(v)).beneficiary(), carol);
19 assertEq(Vesting(payable(v2)).beneficiary(), carol);
20 assertEq(f.getVesting(carol), v2, "registry sees only the second one");

```

Recommendation

Have the escrow notify the factory when either role rotates, with the factory moving the registry entry and rejecting a move onto an address that already holds one. If a callback is unwanted, drop the ambition instead of leaving it half kept: remove the stale indexes and derive the same answers from events, so no on-chain value claims to be authoritative while drifting.

Separately, the two `getMy...` views read `msg.sender`, and `creatorVestings` is `private`, so no parameterised alternative exists. A contract calling on a user's behalf sees its own record rather than the user's, and an `eth_call` that does not set `from` presents the zero address and receives an empty list rather than an error. Both were reproduced. Add `getVestingsOf(address)` and keep the `msg.sender` forms only as sugar over it.

Remediation Status

Remediated in 66af77611. `setBeneficiary` and `setCreator` call back into the factory through `rotateBeneficiaryRegistration` and `rotateCreatorRegistration`, and the factory tracks `creatorVestingIndex` for the bucket move, so the registry no longer describes the parties that held the roles at creation. The finding stays *Open* against the audited snapshot.

3.16 [GTV-17] The creator can decouple the vested numerator from totalAmount and then dilute every ratio-denominated protection

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 259–263 (`_scheduledVestedAt` returns `cliffAmount` when `numberOfPeriods == 0`), 1152–1162 (`setNumberOfPeriods`),

1086–1094 (`topUp`), 820–821 (revocation snapshot)

- The four consumers of the ratio: `:468`, `:484`, `:967`, `:1037`

Description

Four beneficiary protections are denominated in `vestedAtRevocation / totalAmountAtRevocation`: the beneficiary's share of post-revocation staking rewards and losses, and the vested-share floors on the delegation and self-stake paths. On a live schedule that ratio is stable under `topUp` because both the cliff and linear terms scale with `totalAmount`. The creator can break the coupling: `setNumberOfPeriods(0)` ratchets `vestedFloor` to the amount accrued so far and makes `_scheduledVestedAt` return only `cliffAmount`, which is zero when `cliffUnlockBps` is zero, so `_vestedAmount` reads a fixed floor while a subsequent `topUp` raises the denominator. The `topUp` guard at line 1088 keys on `vestingStopped`, the revoker's freeze, so the creator's freeze, which reaches the same pinned state through a schedule setter and sets no flag, is never caught.

Impact

Reproduced on a 1,000e18 grant fully delegated at 50% vested: `setNumberOfPeriods(0)` followed by a 9,000e18 top-up records a 5% ratio instead of 50%, the share floor collapses from 500e18 to 50e18, the revoker exits 950e18 rather than 500e18, and the beneficiary's share of a 100e18 post-revocation reward falls from 50e18 to 5e18. The move is nearly free, since `revoke` refunds 8,500e18 of the 9,000e18 outlay in the same call, leaving a net cost near 5% and a single block of exposure. The beneficiary can neither prevent it, as `topUp` is creator-only, nor undo it, as both revocation snapshots are written once. Vested principal stays safe; what is lost is continued access to the staked position and its proportional earnings, and the same dilution occurs silently when a creator postpones a schedule and tops up in good faith.

Proof of Concept

`test_creatorDilutesTheBeneficiarysShareFloor` pins the vested numerator with `setNumberOfPeriods(0)`, inflates the denominator 10x with a creator top-up that the `vestingStopped` guard never sees, and shows the delegation share floor collapse from half the position to a twentieth.

```

1 // test/CreatorDilution.t.sol :: test_creatorDilutesTheBeneficiarysShareFloor
2 // setUp: 1000e18 grant, cliffUnlockBps==0, fully delegated to VAL
3 vm.prank(ben); v.vestingDelegatorJoin(VAL,TOTAL);
4 vm.warp(t0+150 days);
5 assertEq(v.vestedAmount(),500e18,"50% vested");
6
7 // 1) ratchet the floor, then detach _rawVested from totalAmount
8 vm.prank(cre); v.setNumberOfPeriods(0);
9 assertEq(v.vestedFloor(),500e18,"floor pinned at the accrued amount");
10
11 // 2) topUp inflates the denominator. The guard at 1088 does not fire: no flag was set.
12 uint256 outlay = 9000e18;
13 vm.deal(cre,outlay);
14 vm.prank(cre); v.topUp{value: outlay}("incentive");
15 assertEq(v.totalAmount(),10_000e18,"denominator inflated 10x");
16 assertEq(v.vestedAmount(),500e18,"numerator did NOT move");
17
18 // 3) revoke snapshots the diluted ratio and refunds most of the outlay
19 vm.prank(rev); v.revoke();
20 assertEq(v.totalAmountAtRevocation(),10_000e18,"ratio is now 5%, not 50%");
21
22 // 4) the beneficiary's staking share floor collapses from 50% to 5%
23 vm.prank(rev); v.revokerDelegatorExit(VAL,TOTAL);
24 assertEq(v.delegationVestedFloor(VAL),50e18,"floor collapsed to 50e18");
25 assertEq(stk.sharesOf(address(v),VAL),50e18,"revoker exited 950e18 instead of 500e18");

```

The floor falls from 500e18 to 50e18 so the revoker exits 950e18 rather than 500e18, while revoke refunds 8500e18 of the 9000e18 outlay in the same call; the companion `test_creatorDilutesTheBeneficiarysRewardShare` shows the same 5% ratio credit the beneficiary only 5e18 of a 100e18 reward, against `test_control_floorProtectsHalf` where the undiluted floor holds 500e18.

Recommendation

Extend the `topUp` guard to the state rather than the flag. Reject a top-up whenever `_rawVested()` is below `vestedFloor`, which is the condition the docstring actually describes and which covers the creator's freeze as well as the revoker's. Alternatively, recompute the floor as a fraction of `totalAmount` when `totalAmount` changes, so the ratio the four consumers read stays meaningful. Rejecting `setNumberOfPeriods(0)` while a cliff unlock of zero is configured would close the specific route used here but not the general one through `setStartDate` or `setCliffDuration`.

Remediation Status

Remediated in 66af77611. `topUp` reverts `VestingFloorActive` when `_rawVested()` is below `vestedFloor`, so the creator cannot inflate `totalAmount` while the floor is ratcheted above the raw schedule. The finding stays *Open* against the audited snapshot.

3.17 [GTV-18] Revoking a manual-unlock vesting before it is unlocked zeroes the beneficiary permanently

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in 66af77611

Code Location

- `src/Vesting.sol`: 217–224 (`_rawVested`), 812–820 (`revoke`)

Description

In `_rawVested` the manual-unlock gate is checked before the revoked branch, so it returns zero while the vesting is locked. If a `needsManualUnlock` vesting is revoked before the creator unlocks it, `vestedAtRevocation` snapshots zero, and a later `manualUnlock` cannot restore it because the revoked branch returns the frozen zero. Everything sweeps to the creator.

Impact

A fully-scheduled manual-unlock vesting revoked before unlock leaves the beneficiary with zero vested and zero withdrawable permanently, with no way to force the unlock. The Deepthought DAO category uses manual unlock.

Recommendation

Take the manual-unlock decision before the revocation snapshot, or compute `vestedAtRevocation` from the scheduled amount so an unlock after revocation still credits the beneficiary.

Remediation Status

Remediated in 66af77611. `revoke` computes the vested figure from `_scheduledVestedAt` directly, bypassing the manual-unlock gate, so revoking before the unlock preserves the beneficiary's accrued scheduled share. The finding stays *Open* against the audited snapshot.

3.18 [GTV-19] The GEN implementation contract is left initializable

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in c382be97

Code Location

- `src/GEN.sol`: 84–90 (no constructor), compare `src/Vesting.sol`:143 and `src/VestingFactory.sol`:67

Description

GEN has neither a constructor nor a `_disableInitializers` call, while both sibling contracts do. An attacker can call `initialize` on the logic contract and take its owner and role slots.

Impact

The attacker becomes owner of the implementation and can mint on that phantom instance, but `upgradeToAndCall` reverts on the implementation under

the OpenZeppelin UUPSUnauthorizedCallContext guard, so the real proxy, its owner, balances, and roles, is untouched. The residual risk is a squatted phantom instance and misleading verification state.

Recommendation

Add `constructor() { _disableInitializers(); }` to GEN, matching the sibling contracts, and confirm the proxy is initialized atomically in the deployment.

Remediation Status

Remediated in the reference tree c382be97, verified against GEN.sol SHA-256 b02f1bfd...59e1: GEN gains `constructor() { _disableInitializers(); }`, matching both sibling contracts. The finding stays *Open* against the audited snapshot. The separate deployment-atomicity question is unaffected and still needs confirming against the deploy script.

3.19 [GTV-20] The inflation curve reverts permanently past a finite epoch

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in 66af77611

Code Location

- `src/GEN.sol: 589–601 (rate math)`, `src/lib/LogExpMath.sol (MIN_NATURAL_EXPONENT)`

Description

The exponent handed to `LogExpMath.exp` grows with the epoch and eventually leaves the function's domain, after which `_calculateInflation` reverts. A fuzz sweep across epochs 1 to 25,000 found the first reverting epoch at 15,107, roughly 41 years of daily epochs. Once the chain reaches that range no inflation message can step over it, though the curve has effectively converged well before then.

Impact

Inflation halts permanently at that epoch, and the only escape is a UUPS upgrade, so GEN can never credibly renounce upgradeability. The existing curve fuzz test bounds its inputs below epoch 6040 and never reaches the boundary.

Recommendation

Clamp the exponent to the domain before calling `exp`, returning the converged rate beyond the point where the curve has flattened.

Remediation Status

Remediated in 66af77611. `INFLATION_CURVE_CONVERGED_EPOCH` is set to 15,107 and the rate calculation returns `INFLATION_FINAL` directly at or beyond that epoch, bypassing `LogExpMath.exp` and its domain limit. The finding stays *Open* against the audited snapshot.

3.20 [GTV-21] The compounding root uses a 360-day year while the decay horizon uses 365

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in f6230f7e8

Code Location

- `src/GEN.sol`: 604–607 (`1e18 / 360`), 99 (`INFLATION_DECAY_EPOCHS = 1460`)

Description

The per-epoch compounding raises the annual rate to the power `1e18 / 360`, which also truncates. The decay horizon is expressed as `1460 = 4 * 365`. The two day-counts disagree.

Impact

Year-one issuance measures 585,651,610 GEN, or 13.94% of the fixed base. Whether that is correct is a tokenomics question for the designers, not a defect on its own; the dead constants `INFLATION_DECAY_EPOCHS` and `INFLATION_DECAY_DELTA` suggest the formula was rewritten without reconciling them.

Recommendation

Confirm the intended day-count and make the two uses agree. The live curve is already 360-based, since `INFLATION_MID` is `1440e18`, which is four 360-day years, so if 360 is intended the fix is to delete the two 365-based constants rather than change the arithmetic:

```
1 // Live path, 360-based and self-consistent:
2 int256 public constant INFLATION_MID = 1440e18; // 4 * 360
3 uint256 _nominal = LogExpMath.pow(1e18 + _rate, uint256(1e18) / 360) - 1e18;
4
5 // Dead, 365-based, read nowhere. Remove, or restate on the 360 basis:
6 uint256 public constant INFLATION_DECAY_EPOCHS = 1460; // 4 * 365
7 uint256 public constant INFLATION_DECAY_DELTA =
8     (INFLATION_INITIAL - INFLATION_FINAL) / INFLATION_DECAY_EPOCHS;
```

Leaving both bases in one file makes the intended year length unknowable from the source.

Version note

The measured figure above describes the audited snapshot, where `INFLATION_INITIAL` is 15%. The newer reference tree sets it to 9%, matching the L2 staking contract, so year-one issuance there is lower than the number quoted.

The day-count inconsistency itself is unchanged: both trees compound over 360 while the decay horizon is expressed in 365-day years. Re-measure against the deployed constant before treating the figure as current.

Remediation Status

Remediated in f6230f7e8. `INFLATION_DECAY_EPOCHS` is removed, so the 365-day base no longer appears in the file. The per-epoch root stays `1e18 / 360` and `INFLATION_MID` stays `1440e18`, both expressed on the same 360-day year, and the convention is pinned into the policy hash the L2 verifies (`GEN_INFLATION_LOGISTIC_9_TO_4_PERCENT_360_DAY_V1`). The finding stays *Open* against the audited snapshot.

3.21 [GTV-22] Inflation has no L1 clock check, and the inputs for one are present but unused

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in f6230f7e8

Code Location

- `src/GEN.sol`: 409–415 (`_inflation`, where the only guards are the monotonic cursor and `maxInflationEpochs`, and where an elapsed-time check belongs)
- `src/GEN.sol`: 100 (`INFLATION_EPOCH_DURATION`, declared and never read), 121 and 155 (`INFLATION_INCEPTION`, written once and never read)

Description

L1 mints inflation on the epoch target the L2 asserts, with no check that those epochs have elapsed in wall-clock time. The only forward bound is `maxInflationEpochs`, which caps how far a single call may advance the cursor, not where the cursor may end up: consecutive calls each advance another step, so the total can run arbitrarily far ahead of real time. The contract stores an inception timestamp and an epoch duration, the two values such a check needs, yet reads neither.

Impact

An L2 bug or a compromised L2 sender that reports a future epoch mints inflation that has not yet accrued. This does not by itself close the GTV-01 rewind, whose re-minted epochs have already elapsed and so still pass an elapsed-time check. It is also distinct from GTV-31: that finding concerns a correct epoch whose L2 delivery fails, leaving the two cursors desynchronized, while this one concerns a target that has not yet accrued and would be delivered and applied on both sides without complaint. The per-epoch loop costs about

3,428 gas, so a 4,000-epoch catch-up costs 13.7M gas, which bounds what any `setMaxInflationEpochs` recovery achieves within a block.

Recommendation

Add the L1 clock check using the constants already stored, as defense in depth against a future-epoch target. It belongs beside the existing forward-progress guards in `_inflation`:

```

1  if (_epochTo > 0) {
2      if (_lastInflationEpoch >= _epochTo) revert InflationNotReadyToBeRealized();
3
4      if (maxInflationEpochs < _epochTo - _lastInflationEpoch)
5          revert CanOnlyTriggerInflationMaxEpochsInFuture();
6
7      // NEW: L1 cannot mint more epochs than have actually elapsed.
8      uint256 elapsedEpochs = (block.timestamp - INFLATION_INCEPTION) /
9          INFLATION_EPOCH_DURATION;
10     if (_epochTo > elapsedEpochs) revert InflationEpochNotElapsed();
11 }

```

`INFLATION_INCEPTION` and `INFLATION_EPOCH_DURATION` are already set, so this reads state the contract keeps and never consults. Declare `InflationEpochNotElapsed` alongside the existing inflation errors. Pair it with the GTV-01 write guard rather than treating either as sufficient alone.

Remediation Status

Remediated in f6230f7e8. `_inflation` now reads both stored constants and reverts `InflationEpochNotElapsed` when the target runs ahead of the L1 clock. The bound is `elapsedEpochs + INFLATION_PREFUND_EPOCHS` rather than `elapsedEpochs` alone, a documented 15-epoch allowance matching the runway L2 pre-funds, so the frontier is finite instead of unbounded. `INFLATION_INCEPTION` is written before any reachable call, so the check cannot pass vacuously. The finding stays *Open* against the audited snapshot.

3.22 [GTV-23] An unguarded signed cast in `divInt` can wrap negative

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in 66af77611

Code Location

- `src/GEN.sol: 630–637 (divInt)`

Description

`divInt` calls `divDown`, which computes `a * 1e18 / b`. For a small divisor the quotient can exceed the signed maximum, and the following `int256` cast wraps negative, so a positive divided by a positive returns a negative result. It is not reachable through `_calculateInflation`, whose only call site passes a fixed numerator of `11e16` over a divisor of at least `1e18`, capping the quotient at

11e16. The sibling `mulInt` is unaffected, since its `mulUp` divides by `1e18` first and cannot overflow the cast.

Impact

Latent. The in-range path the curve uses stays exact and positive; the concern is reuse of the helper in a future context with different bounds.

Recommendation

Bound-check before the cast, or return the result as `uint256` where the sign is known, so a future caller cannot trip the wrap.

Remediation Status

Remediated in 66af77611. `divInt` guards the cast with `if (quotient_ > uint256(type(int256).max)) revert SignedConversionOverflow();`.

The finding stays *Open* against the audited snapshot.

3.23 [GTV-24] A superseded inflation message is indistinguishable from a live one

Risk Level

Severity: Low, Likelihood: High, Status: Remediated in 66af77611

Code Location

- `src/GEN.sol`: 410–411 (the `revert`), 378–381 (`isL2MessagePending` early returns)
- The error itself is declared in the `IGEN` interface, which was not part of the delivery (Section 1.1)

Description

`_inflation` computes its mint range from the stored cursor `lastInflationEpoch`, so an epoch target is an absolute high-water mark, not a delta. Executing a higher target sweeps up every epoch below it, which permanently supersedes any lower pending message: absent the testnet-only `resetInflation` lever (GTV-02), `lastInflationEpoch` only moves forward, so once it reaches or passes a message's `epochTo` the guard `if (_lastInflationEpoch >= _epochTo) revert InflationNotReadyToBeRealized()` fires forever. The error name implies the message is early, but here the condition is terminal. `isL2MessagePending` reports such a dead message as (`pending = true`, `rewards = 0`), the identical signal a live BURN or TEST message carrying no reward produces, so neither the `revert` nor the view distinguishes a dead message from a live one.

Impact

Reproduced: no value is lost and no epoch is minted twice, but a caller retrying a superseded message pays for the full Merkle proof verification before the guard reverts, since the cursor check runs after `proveL2MessageInclusion`. A watcher that keys off the `pending` boolean rather than the zero reward will retry a dead message indefinitely, and because the reward is a percentage of the whole swept range, executing the highest pending target always pays most, so superseded messages are the normal outcome rather than an edge case.

Recommendation

Split the error and give the terminal case its own name, for example `InflationAlreadyRealized`. In the audited snapshot the error has exactly one use site, so this is a two-line change: the declaration in the interface and the single `revert`. The forward-progress case and the `maxInflationEpochs` case are already separate errors, so no other call site is affected.

Return `(false, 0)` from `isL2MessagePending` when `lastInflationEpoch >= epochTo`, so a watcher can retire the message instead of polling it. Of the two changes this is the one that matters economically: a caller who can see the message is dead never submits, which avoids the whole cost rather than part of it.

A cheap early-out inside `executeL2Message` is also available and worth taking, since the cursor guard runs after `proveL2MessageInclusion` and a doomed call pays for the Merkle verification before it fails. Measured against a bridge mock performing real proof hashing, that verification is roughly 8.0k gas at proof depth 8, 9.1k at depth 16, and 16.2k at depth 32, against about 38.5k for the whole doomed transaction at depth 16 once the 21k base cost and the roughly 8.5k of unavoidable proof calldata are counted. Checking the cursor first therefore saves on the order of a quarter of a wasted call, and the proof bytes must still be transmitted either way.

Two caveats on how to implement it. The guard inside `_inflation` must stay, because `inflationInit` reaches that function by a different path and the early test is a pre-flight duplicate rather than a relocation. And rather than decoding the message before it has been authenticated, prefer binding the target epoch as an explicit argument the way `_l2GasPrice` and `_l2GasLimit` already are: compare the supplied epoch against the cursor before verifying, then after verifying assert that the decoded epoch matches what the caller supplied. A caller who overstates the epoch to slip past the early test is caught by that equality check, so no unverified field is acted on.

Version note

Present in both trees, and the newer one widens the overload rather than narrowing it. The reference tree adds `if (epochTo == 0) revert InflationNotReadyToBeRealized();` as part of its GTV-01 fix, so the same error now covers three distinct terminal conditions: an invalid zero target, a target already realized, and a target the cursor has passed. Its `isL2MessagePending`

still returns `(true, 0)` for both the zero-target and already-passed cases. The recommendation applies to the reference version with more force than to the audited one.

Remediation Status

Remediated in 66af77611. The terminal case has its own error, `InflationAlreadyRealized`, and `isL2MessagePending` returns `(false, 0)` once the cursor has passed the target, so a watcher can retire a superseded message. The finding stays *Open* against the audited snapshot.

3.24 [GTV-25] The validator address sync is permissionless, so a third party picks the moment a registry migration takes effect

Risk Level

Severity: Low, Likelihood: Medium, Status: Remediated in f6230f7e8

Code Location

- `src/Vesting.sol`: 751–775 (the sync), 752–758 (the registry reads and the fees fallback), 211 (the escrow’s own staking resolution)
- `test/PermissionlessSync.t.sol` (ten tests, including the front-running proof of concept)

Description

`vestingValidatorUpdateExternalAddresses` takes no arguments and re-reads all three addresses from the same `AddressManager` the escrow uses, so a caller cannot inject addresses and the permissionless design is sound. What it hands out is timing: each wallet caches `Staking` while the escrow reads it fresh (line 211), and this is the only function that reconciles them, so any address chooses when the caches move to a new registry value. That enables four things: a staking migration can be front-run, stranding an in-flight unbonding claim on the old contract; the `getAddress("ConsensusMainWithFees")` zero-fallback lets anyone drop every wallet to non-fee routing; unlike the operator setters it has no `revoked` guard; and its loop over `validatorWallets` is unbounded with no per-wallet variant.

Impact

Reproduced: with 100e18 queued on the old staking, an unrelated address flipping the registry key and calling the sync leaves the claim returning nothing until the registry is repointed, so the principal is stranded, not lost. The timing is what operations cannot control. The first sync costs about 92k gas per wallet, so roughly 325 wallets exceed a 30M block; the theory that a bloated `validatorWallets` bricks the withdrawal gate is refuted, since `_outstandingValidatorPrincipal` costs about 586 gas per entry.

Recommendation

Keep the function permissionless with registry-only inputs, and add a per-wallet form `vestingValidatorUpdateExternalAddressesFor(address wallet)` so operations can sync safe wallets while leaving one with an in-flight claim alone; this also removes the fleet-size ceiling. Add the `revoked` guard the sibling functions carry, and move the migration ordering, drain pending wallet claims before repointing the registry, into the runbook.

Version note

The audited snapshot already documents this hazard. A developer note above the function describes the same sequencing requirement and attributes it to an earlier internal review. The gap this finding records is enforcement: the ordering the note prescribes is advisory, and the function that can violate it is open to everyone.

Remediation Status

Partially Remediated in 66af77611. Fixed: the migration front-run, since `_updateValidatorExternalAddresses` keeps the old staking address whenever that position still holds shares, a deposit commit or a withdrawal tranche, so an in-flight claim can no longer be stranded; and the fleet-size ceiling, through the new per-wallet `vestingValidatorUpdateExternalAddressesFor`. Outstanding: neither entrypoint carries a `revoked` or `vestingStopped` guard, and the `ConsensusMainWithFees` zero-fallback is unchanged, so any address can still collapse every wallet to non-fee routing. The finding stays *Open* against the audited snapshot.

Remediated in f6230f7e8, which closes both. Each entrypoint now opens with `if (revoked) revert AlreadyRevoked();` and `if (vestingStopped) revert VestingAlreadyStopped();`, and `consensusWithFees` is read through `getAddressNonZero("ConsensusMainWithFees")`, so a missing fee endpoint reverts instead of silently routing the whole fleet without fees.

3.25 [GTV-26] Informational suggestions

Risk Level

Severity: Informational, Likelihood: N/A, Status: Acknowledged in f6230f7e8

These are suggestions only. Each item below is a hygiene or robustness observation; none affects security, and fixing them or not does not change the risk profile. They are grouped by surface.

Cross-chain surface: stuck ETH, stray-transfer burns, and a misleading init comment

- **ETH is trapped in GEN** (GEN.sol:471). The payable `receive` accepts ETH, but GEN is the L2 base token and the bridge is called with no value, so GEN never spends ETH and exposes no sweep. Anything sent is locked until an upgrade.
- **BURN consumes stray GEN** (GEN.sol:303–306). A BURN burns the amount it names from the token’s own balance (`_burn(address(this), value)`), which is commingled, so GEN mistakenly sent to the token address is indistinguishable from the protocol’s and is consumed by a later BURN with no recovery. Confirmed by test.
- **Misleading inflationInit NatSpec** (GEN.sol:205–209). “0 = handshake only” reads as a first step, but passing zero consumes the one-time guard and makes the seeding half unreachable. The 20-epoch seeding cap is undocumented, and the zero value is the GTV-01 trigger.

Suggestion. Add a rescue path or remove the payable `receive`; document the stray-transfer behavior; and correct the `inflationInit` comment to describe the mode selector and the cap.

Staking accounting robustness: a hard panic, dust drift, and a discarded return value

- **Division-by-zero panic** (Vesting.sol:407–410). `vestingDelegatorExit` divides by `totalShares` with no zero guard and panics, where its revoker twin reverts cleanly with `ZeroAmount`. Add the same guard.
- **Rounding dust** (Vesting.sol:409–419). `proRataDeposited` can round to zero at a large share ratio, but the detached principal is sub-wei and a later full exit self-corrects. Cosmetic.
- **Discarded return value** (Vesting.sol:443 and 999). `delegatorClaim` returns the amount claimed, and both call sites ignore it in favor of a balance-delta inference. Using the returned figure would remove the fragility that several other findings turn on.

Suggestion. Add the zero guard, and prefer the returned claim amount over balance-delta inference.

Operator and authority hygiene

- **Set-time-only operator EOA check** (Vesting.sol:516, 682, 722). The no-code check passes for a CREATE2 address not yet deployed, which can gain code later. Re-check at completion time.
- **Operator rotation emits nothing** (Vesting.sol:672–726). All four rotation entry points forward to the wallet without emitting, while every sibling emits (`ValidatorJoined`, `ValidatorExited`, and the rest). `IVesting` declares no operator event, so the fix is to add one and emit

it from all four. Operator identity governs what gets slashed and cannot be followed from this contract's logs.

- **The operator check admits the zero address** (`Vesting.sol:516, 682, 722`). All three sites test `operator == address(this) || operator.code.length != 0`, which the zero address passes. A validator registered to it is selected for duties it cannot perform and slashed, a loss borne by the beneficiary and, after revocation, the creator. Add a zero check at all three sites. The newer staking derives the operator from a validated key (Section 1.1).
- **Cross-vesting operator reuse** (`Vesting.sol:656–667`). `_requireDistinctOperator` scans only the current vesting's wallets, so an actor can reuse an operator across two vestings and raise correlated-slashing exposure. The cross-vesting companion to GTV-13.
- **Dual authority roots** (`GEN.sol`). GEN runs Ownable and AccessControl as unrelated roots, the structural cause behind GTV-07.

Suggestion. Re-check operator code at completion, reject the zero address at all three sites, declare an operator-rotation event in `IVesting` and emit it from all four functions, extend the distinctness check across vestings one actor controls, and resolve the dual-root design per GTV-07.

Setter, factory, and configuration hygiene

- **Configuration not followable from events** (`VestingFactory.sol:249–258, 263–269; IVesting.sol:110–115`). `vestingBlueprint` has two writers but only `setVestingBlueprint` emits `VestingBlueprintSet`; `upgradeBeacon` writes it at line 255 emitting only `BeaconUpgraded`. `addressManager` has two writers and no event, because `IVesting` declares none. `setAddressManager` also lacks the zero-address check `setVestingBlueprint` carries, and a zero manager bricks staking for later vestings; several `GEN.sol` setters (`:475–502`) share the gap.
- **vestingBlueprint can disagree with the beacon** (`VestingFactory.sol:263–269`). Calling `setVestingBlueprint` after `deployNewBeacon` rewrites the variable without touching the beacon, so it stops describing the running implementation. Only `upgradeBeacon` changes behavior.
- **Vacuous per-item funding guard** (`VestingFactory.sol:179`). The per-item check compares `msg.value`, constant across a batch, so it never fires; only the exact-sum check protects the batch. Replace it with a running remainder.
- **Dead reentrancy guard** (`VestingFactory.sol`). The factory initializes `ReentrancyGuardUpgradeable` but applies `nonReentrant` nowhere.
- **Donations swept to the creator** (`Vesting.sol:843, 920`). Value sent to a vesting's payable fallback is counted as idle and swept to the creator

on revocation, misattributing third-party funds.

- **Permissionlessly-mintable predicate** (`VestingFactory.sol`, `isVestingContract`). Anyone can create a 1-wei vesting to set this flag for an address they control. Harmless here, but an escalation if an off-chain system grants privilege on it. See the deciding question below.
- **Hardcoded initializer encoding** (`VestingFactory.sol:30`). `INITIALIZE_SELECTOR` hardcodes a 13-argument encoding, so a beacon upgrade that changes `initialize` breaks new creations until the factory is upgraded in lockstep.

Deciding question for the L2 source. Does any contract outside this repository grant privilege based on `isVestingContract`? If so, the mintable predicate is a High-severity escalation rather than a cosmetic note.

Suggestion. Emit `VestingBlueprintSet` from `upgradeBeacon` as well; declare an address-manager event in `IVesting` and emit it from both writers; give `setAddressManager` the zero-address check its neighbour has; treat `vestingBlueprint` as a cache of the beacon or read it through the beacon; replace the per-item funding guard with a running remainder; and remove or apply the reentrancy guard.

Reentrancy on value-sending functions was investigated and is not exploitable **Code Location:** `src/Vesting.sol: 355` (`vestingWithdraw`), `812` (`revoke`), `917` (`foundationSweep`).

These three functions send native value without a `nonReentrant` modifier, which both passes examined and cleared. `vestingWithdraw` follows checks-effects-interactions, incrementing `totalWithdrawn` before the transfer. A reentrant withdrawal inside a claim removes value the claim already counted into its balance-before, so the subtraction underflows and the transaction reverts. A partial grab understates the measured return by exactly the amount grabbed, so the attacker ends strictly worse off, 130e18 against 160e18 on the honest path. The protection here is the arithmetic itself.

Suggestion. Add `nonReentrant` to the three functions anyway, so safety does not rest on an arithmetic accident that refactoring could remove.

Remediation Status

Partially Remediated in 66af77611. Fixed: the division-by-zero panic on delegation exit, now guarded by `if (totalShares == 0 || shares > totalShares) revert ZeroAmount()`; the missing zero-address check on `setAddressManager`; the operator zero-address admission, since the operator is now derived from `PubKeyUtils.validate` rather than supplied directly; and the dead reentrancy guard, now applied to `claimBeneficiaryRegistration`. Outstanding: trapped ETH in GEN, stray-GEN burns, the `inflationInit NatSpec`, the discarded `delegatorClaim` return value, cross-vesting operator

reuse, the absent operator-rotation event, the vacuous per-item funding guard, donations swept to the creator, and the dual authority roots. The finding stays *Open* against the audited snapshot.

Acknowledged in f6230f7e8. Applied: the trapped ETH, through `rescueEth`; the stray-GEN burn, since burns now target an isolated `GENBurnCustody` rather than the token's own commingled balance; the `inflationInit` `NatSpec`, rewritten with the seeding cap as a named constant; the discarded `delegatorClaim` return, now used in place of the balance delta; cross-vesting operator reuse, through the new `VestingOperatorRegistry`; the missing rotation event, `ValidatorOperatorAction` from all four paths; and the vacuous per-item funding guard, removed in favour of the callers' exact-sum checks. `nonReentrant` is also applied to all three value-sending functions.

Two items are not applied and Arcadia does not ask for them. Donations are still swept to the creator, which is a defensible reading of a vesting's balance as escrow. GEN still carries two authority roots, but `transferOwnership` now moves ownership and the token roles together, so the consequence this item pointed at, GTV-07, is closed. Neither affects security, and the finding is closed as acknowledged.

3.26 [GTV-27] A delegation exit taken while a same-epoch deposit is still unpriced over-attributes principal, books a phantom loss and strands the deposit

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `contracts/vesting/Vesting.sol`: 387 (`vestingDelegatorJoin` credits `depositedPerValidator` at once), 402–432 (`vestingDelegatorExit`, the `deposited * shares / totalShares` pro-rata), 991–1029 (`revokerDelegatorExit`, same arithmetic plus the floor), 434–468 (`vestingDelegatorClaim` early return at `deposited == 0`)
- `contracts/utils/DelegatorUtils.sol`: 726 (`_claimShares` returns zero for an unpriced non-genesis commit), 709–710 (`_sharesOf` zeroes the same commit)
- `contracts/utils/ValidatorPrimeUtils.sol`: the deposit-pricing hop, which advances only to commits with `linkToNextCommit <= _epoch - 1` before setting `deposit.priced = true`
- `contracts/staking/StakingValidators.sol`: 411 (`validatorPrime(address)`, public and permissionless), 420 (prime requires the prior epoch finalized)

Description

`vestingDelegatorJoin` adds the full deposit to `depositedPerValidator[v]` immediately, but the staking ledger does not price it then. `delegatorJoin` records the deposit as a commit stamped with the current epoch E and leaves it unpriced. Pricing happens only inside `validatorPrime`, whose deposit cursor advances solely to commits from epochs at most $_epoch - 1$. A deposit made in epoch E therefore cannot be priced before a prime runs in epoch $E+1$ or later, and that prime is itself a transaction, either `validatorPrime(address)` or a validator's own deposit, gated on the prior epoch being finalized. For the whole of that window `sharesOf(vesting, v)` excludes the deposit: `\_claimShares` returns zero for any commit with `!priced && epoch != 0`, and `DelegatorUtils.\_sharesOf` sets the same commit to zero shares outright. The only commits that count as shares before pricing are genesis (epoch 0) commits, at 1:1.

Both exit paths compute the principal leaving the pool as

```
1 uint256 totalShares = staking.sharesOf(address(this), validator); // priced shares only
2 uint256 deposited = depositedPerValidator[validator]; // includes the unpriced deposit
3 uint256 proRataDeposited = (deposited * shares) / totalShares;
```

so the numerator counts principal the denominator does not. Exiting the priced shares during the priming window moves the unpriced principal into `pendingExitDeposited` as well, and `depositedPerValidator` drops to zero while the deposit is still staked. When the deposit later prices, its exit computes `proRataDeposited = 0`, adds nothing to `pendingExitDeposited`, and the claim path short-circuits at `deposited == 0` before ever calling `staking.delegatorClaim`. `revokerDelegatorExit` uses the same denominator for both its pro-rata and its floor.

Impact

Join 100 at epoch 2 (priced by epoch 4), join 100 more at epoch 10, exit the 100 priced shares in the same epoch: `proRataDeposited = 200 * 100 / 100 = 200`, so `pendingExitDeposited = 200` against a real return of 100. The claim books `accumulatedLosses += 100` that never happened. The second deposit prices at epoch 12 with zero tracked principal; exiting it adds 0 to `pendingExitDeposited`, and every later claim returns at `deposited == 0` without touching staking, so 100 GEN sits in the staking withdrawal queue with no authorized caller (`delegatorClaim` requires `msg.sender == delegator`). Pre-revocation the only recovery is a fresh join at or above `delegatorMinStake` plus an exit and claim on the same validator; after revocation `vestingDelegatorJoin` reverts `AlreadyRevoked` and the principal is unrecoverable. The revoker can trigger the same over-attribution post-revocation against a beneficiary deposit made shortly before revoke, docking the beneficiary `loss * V/T`. With only a pending deposit outstanding, `sharesOf == 0` and the exit reverts with a division-by-zero panic rather than a named error.

Relation to prior findings

GTV-08 covers the partial-maturity claim and the priced-zero payout; this is a

third way into the same stranded state, with a different root cause (the pro-rata base, not the claim reconciliation) and a different fix.

Proof of Concept

Six independent passes traced the same sequence with concrete numbers; the trace below is the epoch-by-epoch state.

```

1 // epoch 5: vestingDelegatorJoin(V, 100) -> depositedPerValidator = 100, sharesOf = 0 (unpriced)
2 // epoch 6: prime -> sharesOf = 100
3 // epoch 8: vestingDelegatorJoin(V, 100) -> depositedPerValidator = 200, sharesOf = 100,
  pendingDeposits = 100
4 // epoch 8: vestingDelegatorExit(V, 100) -> proRata = 200*100/100 = 200
5 // depositedPerValidator = 0, pendingExitDeposited = 200
6 // staking exits only the 100 priced shares
7 // epoch 9: prime -> second deposit priced: 100 live shares, 0 tracked principal
8 // epoch 15: vestingDelegatorClaim(V) -> returned = 100 -> pendingExitDeposited = 0
9 // _bookProfitOrLoss(200, 100): accumulatedLosses += 100 (phantom)
10 // epoch 16: vestingDelegatorExit(V, 100) -> proRata = 0*100/100 = 0, pendingExitDeposited
  stays 0
11 // epoch 23: vestingDelegatorClaim(V) -> deposited == 0 -> _reconcileWipedDelegation -> return
12 // staking.delegatorClaim never called: 100 GEN stranded

```

Recommendation

Pro-rate against priced principal only in both exit paths: `priced = deposited - staking.getPendingDelegatorDeposits(address(this), validator)` and `proRataDeposited = priced * shares / totalShares`, with `if (totalShares == 0) revert ZeroAmount()`. Alternatively refuse exits while `getPendingDelegatorDeposits != 0`. Pair with the GTV-08 fix so a zero-principal exit can still be settled through `staking.delegatorClaim`.

Remediation Status

Remediated in 66af77611. `_delegationPrincipalForExit` subtracts `getPendingDelegatorDeposits` from `depositedPerValidator` and pro-rates only the remaining active principal, so numerator and denominator are both priced-only, and it opens with `if (totalShares == 0 || shares > totalShares) revert ZeroAmount();`. Both exit paths share the fix through `_exitDelegation`. The finding stays *Open* against c382be97.

3.27 [GTV-28] The revoker's capped partial exit is rejected by staking's minimum-stake rule, so the clawback can never start

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `contracts/vesting/Vesting.sol`: 991–1029 (`revokerDelegatorExit`, shares capped to `totalShares - floor`)

- `contracts/utils/DelegatorUtils.sol`: `delegatorExit (DelegatorMustExitAllWhenBel` and `DelegatorExitWouldBeBelowMinimum)`
- `contracts/Staking.sol`: `DEFAULT_DELEGATOR_MIN_STAKE = 42e18`

Description

`revokerDelegatorExit` always leaves the vested fraction of the shares staked, so whenever the vested amount is non-zero the exit it submits to staking is a partial one. `DelegatorUtils.delegatorExit` rejects a partial exit in two cases: if the remainder would be below `delegatorMinStake` it reverts `DelegatorExitWouldBeBelowMinimum`, and if the delegation is already below the minimum (for instance after a slash) it reverts `DelegatorMustExitAllWhenBelowMinimum`. The vesting never asks staking for the minimum and never falls back to a full exit, so the revoker's only recovery path reverts by construction whenever the beneficiary's vested share of that delegation is worth less than 42 GEN, or whenever the delegation has been slashed below it.

Impact

A grant of 1,000 GEN with a 1% cliff unlock (vested 10 GEN) and 500 GEN delegated: the floor is 5 shares, the revoker asks for 495, and staking reverts because the 5 GEN remainder is under the 42 GEN minimum. The revoker can work around the first case by leaving exactly 42 GEN behind (a hostage band of up to `delegatorMinStake` per validator), but a delegation slashed below the minimum is fully un-exitable by the revoker because the cap forbids the exit-all that staking demands. Only the beneficiary, whose `vestingDelegatorExit` carries no cap, can release the creator's principal, and they keep V/T of its rewards while they wait.

Relation to prior findings

GTV-05 covers the floor stranding the clawback through the vesting's own accounting; this is an independent staking-side rejection of the same capped exit that fires even when the floor arithmetic would have allowed it.

Proof of Concept

Executed against the real staking diamond in a Hardhat probe (deleted after the run).

```
1 // vesting: totalAmount 1000e18, cliff 0, cliffUnlockBps 100 -> vestedAtRevocation = 10e18
2 // beneficiary delegates 500e18 to V; delegatorMinStake = 10e18 in the fixture
3 revoke();
4 // floor = 500e18 * 10 / 1000 = 5e18 shares; unvestedShares = 495e18
5 Staking.delegatorExit(V, 495e18) // called from the vesting directly
6 // -> revert DelegatorExitWouldBeBelowMinimum()
7 revokerDelegatorExit(V, 500e18)
8 // -> revert DelegatorExitWouldBeBelowMinimum()
9 // state after: pendingExitDeposited = 0, depositedPerValidator = 500e18, delegationVestedFloor
  = 0
```

With the production minimum of 42 GEN any delegation whose vested-fraction stake is under 42 GEN reproduces this.

Recommendation

In `revokerDelegatorExit`, read `delegatorMinStake` and, when the capped exit would leave less than the minimum or the delegation is already below it, exit all shares instead. Post-revocation the beneficiary's entitlement is already reserved out of idle balance by `_withdrawableAmount` and `foundationSweep`, so the vested share needs no share-level floor once the recovered principal is back in the escrow.

Remediation Status

Remediated in 66af77611. `_recoveryExitShares` escalates to a full exit when a capped partial one would leave the remaining active stake below the staking minimum, so the revoker's recovery is no longer rejected. The finding stays *Open* against c382be97.

3.28 [GTV-29] A pending operator transfer survives revocation and the incoming operator completes it on the wallet directly

Risk Level

Severity: Medium, Likelihood: Medium, Status: Remediated in 66af77611

Code Location

- `contracts/vesting/Vesting.sol`: 725–735 (`vestingValidatorCompleteOperatorTransfer` the revoked guard), 736–742 (`vestingValidatorCancelOperatorTransfer`, `onlyBeneficiary`), 853–905 (`revoke`, no cancellation of pending transfers)
- `contracts/validatorsWallet/ValidatorWalletBlueprint.sol`: 166–196 (`completeOperatorTransfer`: `if (msg.sender != owner() && msg.sender != pendingOperator) revert`)

Description

The post-revocation operator freeze is enforced only on the vesting's forwarder: `vestingValidatorCompleteOperatorTransfer` reverts `AlreadyRevoked`, but the wallet's own `completeOperatorTransfer` accepts the pending operator as a caller. `revoke` does not cancel pending transfers, pending transfers never expire, and the only cancel path is `onlyBeneficiary`. A beneficiary who stages a transfer to an address they control at any time before revocation can therefore have that address finish the rotation after revocation, while the creator's unvested self-stake is still in the wallet waiting out the seven-epoch unbonding.

Impact

The freeze was added specifically so a revoked beneficiary cannot hand the creator's stake to an operator who will get it slashed. It is bypassable by

planning ahead: stage `initiate(W, X)` on day one and never complete it. After `revoke`, `X` calls `W.completeOperatorTransfer()` directly; the factory's `onOperatorChanged` does only list maintenance. The creator's exposure is bounded by the slash rate on its unvested self-stake during the unbonding window. The same out-of-band completion also lets two wallets converge on one operator when combined with GTV-13.

Relation to prior findings

GTV-13 covers the distinct-operator check ignoring pending transfers. This is the post-revocation freeze, a separate guard with a separate fix in `revoke`.

Proof of Concept

```
1 // t=0 beneficiary: vestingValidatorInitiateOperatorTransfer(W, pk(X), proof)
2 // wallet: pendingOperator = X, operatorTransferInitiated = now (never expires)
3 // t=T revoker: revoke() // W still holds 100,000 GEN self-stake, 25% vested
4 // -> 75,000 GEN is creator-owned and stays staked until revokerValidatorExit + 7 epochs
5 // t=T+1 X: W.completeOperatorTransfer()
6 // pendingOperator != 0 ok, delay elapsed ok, msg.sender == pendingOperator ok
7 // -> operator = X; factory.onOperatorChanged(X) (map maintenance only)
8 // The vesting path would have reverted AlreadyRevoked; the wallet path did not.
9 // test/vesting/VestingCon796ValidatorWalletOps.test.ts:195 exercises only the vesting
10 // passthrough
11 // against a mock wallet and relies on the beneficiary cancelling.
```

Recommendation

In `revoke`, iterate `validatorWallets` and call `cancelOperatorTransfer()` on each (owner-only, a no-op when nothing is pending), and add a revoker-callable `cancel`. Alternatively have the wallet restrict completion to `owner()` when the owner is a vesting contract.

Remediation Status

Remediated in 66af77611. `vestingValidatorCompleteOperatorTransfer` reverts `AlreadyRevoked` once the vesting is revoked, and `ValidatorWallet-Blueprint` independently blocks completion through `_ownerIsRevoked` and `OwnerRevoked`. The finding stays *Open* against c382be97.

3.29 [GTV-30] Self-stake settlement books returns principal-first, so realized profit or loss can be deferred across the revocation boundary

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated in 66af77611

Code Location

- contracts/vesting/Vesting.sol: 620–660 (`_claimValidator`, `principalReturned = min(returned, outstanding)`), compare 402–432 (`vestingDelegatorExit`, pro-rata booking at exit)

Description

The delegation path attributes principal at exit time, pro-rata to the shares leaving, and books profit or loss on the very next claim. The self-stake path does not read `vShares` before and after an exit; instead `_claimValidator` consumes every returned wei as principal until `validatorDeposited` is exhausted, and recognizes all reward or loss only on the tranche that fully settles the wallet. Whoever times the last tranche therefore decides on which side of `revoke` the wallet's entire realized result is booked, and the attribution rule changes at that boundary: 100% to the beneficiary before, V/T split after.

Impact

Self-stake 100 in wallet W, $V/T = 0.5$, W slashed 20% (worth 80). A pre-revocation exit of half the shares returns 40, all booked as principal, no loss recorded although 10 was realized. After `revoke` the remaining half returns 40 and settles the wallet: loss 20 finalized and split, so the beneficiary bears 10. Through the delegation path the same events book 10 before revoke (100% beneficiary) and 10 after (split), leaving the beneficiary 485 and the creator 495 instead of 490/490. The shift to the creator is $(1 - V/T)$ of every loss realized before revocation, scaling with stake and slash size; the mirror case (a gain realized before revocation, settled after) shifts $(1 - V/T)$ of the beneficiary's pre-revocation reward to the creator, which the beneficiary avoids by always exiting in full.

Relation to prior findings

GTV-11 covers the asymmetry in how pre- and post-revocation results are migrated. This is a separate timing lever in the self-stake path that lets one party choose which regime applies to a result that was already realized.

Proof of Concept

```

1 // T = 1000, V = 500 at revoke; self-stake 100 in W, idle 900; W slashed 20% (worth 80)
2 // pre-revoke: vestingValidatorExit(W, 50%) -> returned 40
3 // _claimValidator: principalReturned = min(40, 100) = 40, validatorDeposited = 60, no loss
  booked
4 // revoke(): benefReserve 500, idle 940 -> creator receives 440
5 // post-revoke: revokeValidatorExit(W, max) exits half of remaining -> returned 20 ->
  outstanding 40
6 // beneficiary exits the rest -> returned 20 -> outstanding 20 -> _validatorFullySettled
7 // lossFinalized = 20; _bookProfitOrLoss(40, 20): loss 20 -> beneficiaryLoss = 20 * 500/1000 =
  10
8 // final: beneficiary 490, creator 490
9 // same events through vestingDelegatorExit: loss 10 booked pre-revoke (100% beneficiary), 10
  post
10 // final: beneficiary 485, creator 495

```

Recommendation

Book self-stake exits pro-rata like delegation: read `validatorView(wallet).vShares` before and after `validatorExit`, move `validatorDeposited * sharesExited / sharesBefore` into a per-wallet pending bucket, and reconcile that bucket, not `min(returned, outstanding)`, against the returned amount at

claim.

Remediation Status

Remediated in 66af77611. `_claimValidator` matches withdrawal tranches by epoch, sums only the settled principal from `validatorExitPrincipalByEpoch`, and passes that figure to `_settleValidator`, so profit is booked as it is realized rather than deferred behind principal. The finding stays *Open* against c382be97.

3.30 [GTV-31] The L1 inflation cursor advances before the L2 delivery is known to succeed, and a single failed delivery deadlocks inflation with no recovery path

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated in f6230f7e8

Code Location

- `contracts/GEN.sol`: 443–500 (`_inflation`: `lastInflationEpoch = _epochTo` before `_sendToL2`; fee quote `l2TransactionBaseCost(..., _l2GasPrice, ...)` * 16 / 10), 594–620 (`_sendToL2`), 248–280 (`inflationInit`)
- `contracts/staking/StakingInflation.sol`: 230–257 (`inflationReceive`: `InflationNotInitialized`, `_requireInflationFunds`), 258–275 (`_requireInflationFunds`)

Description

`_inflation` commits `lastInflationEpoch` and mints before the bridge call, and the bridge call only enqueues an L1 to L2 transaction; nothing on L1 ever reads the L2 outcome. On L2, `inflationReceive` requires `!inflationInitDone == false` and `msg.value >= sum(epochInflation(e))` over its *own* cursor range, and `inflationEpoch` is written only inside `inflationInit` and `inflationReceive`; there is no admin resync on either side and production GEN has no reset. If the L2 execution fails for any reason, the bridged value is refunded to the staking contract as untracked balance and the L2 cursor stays behind while the L1 cursor has moved. Every later delivery covers `(L1.last, to]` but L2 demands `[L2.prev, to)`, so `inflationReceive` reverts `InsufficientInflationFunds` forever. If the failing leg is the `inflationInit` handshake, L1 is permanently initialized while L2 stays uninitialized and rejects every later receive.

Three ordinary causes can make the L2 leg fail: the L2-configured gas limit is too low for the `_requireInflationFunds` loop over up to `maxInflationEpochs` epochs of `LogExpMath`; the L2 `incentivePercentage` is set lower than L1's (the two are independent owner-set values that a comment says

are “kept in sync”); or, on the L1 side, the fee quote is computed from the message-supplied `_l2GasPrice` while the zkSync Mailbox prices the request from the executor’s live `tx.gasprice`, so when the L1 base fee exceeds about 1.6 times the price baked into the message the request reverts and the message is unexecutable until gas falls, and a later message with a higher `epochTo` executed first permanently kills it.

Impact

With L1 at epoch 5 and a message for epoch 20, L1 mints roughly 15 epochs of inflation and sets its cursor to 20; if the L2 transaction runs out of gas, those tokens sit in the staking contract’s balance unaccounted. The next message for epoch 35 delivers 15 epochs while L2 requires 30, and reverts; every subsequent message likewise. Validator, staker and developer inflation halts network-wide until a GEN or Staking upgrade. The condition needs no attacker, only a configuration drift or a gas-limit miss, and the executor incentive keeps being paid on L1 for messages that never land.

Relation to prior findings

GTV-22 (no L1 clock check) and GTV-24 (superseded messages) touch the same pipeline; neither covers the L1/L2 cursor desynchronization or its irreversibility. The L1/L2 rounding-order asymmetry in the incentive carve was checked numerically by three independent passes (exact integer replay over 800 epochs, widths 1–20, seven incentive values) and found safe only because `INFLATION_BASE` is a multiple of `1e18`; it is recorded as a lead.

Proof of Concept

```
1 // L1: lastInflationEpoch = 5. L2: inflationEpoch = 5.
2 // executeL2Message(INFLATION, epochTo = 20):
3 // _inflation: lastInflationEpoch = 20; mints sum(6..20) ~ 15M GEN (+ 60% gas buffer)
4 // _sendToL2: requestL2TransactionDirect(l2Value = amountToStaking, gas = _l2GasLimit)
5 // L2: inflationReceive(20) runs out of the 500k gas configured on L2
6 // -> l2Value refunded to genLayerStaking as plain balance; inflationEpoch still 5
7 // executeL2Message(INFLATION, epochTo = 35):
8 // L1 delivers sum(21..35) ~ 15M; L2 _requireInflationFunds(5, 35) needs sum(6..35) ~ 30M
9 // -> revert InsufficientInflationFunds. Repeats forever.
```

Recommendation

Advance `lastInflationEpoch` only after the L2 transaction is proven successful (zkSync `proveL1ToL2TransactionStatus`), or add an owner-only `resend-Inflation(epochFrom)` that re-delivers a proven-failed range. The cheapest structural change is to stop treating the enqueue as delivery:

```
1 // Today: the cursor commits before the bridge call, and nothing reads the L2 outcome.
2 if (_epochTo > 0) lastInflationEpoch = _epochTo;
3 _sendToL2(totalMint, amountToStaking, _l2Calldata, _l2GasLimit);
4
5 // Instead: record the range as in flight, and commit it only once proven.
6 pendingInflation[canonicalTxHash] = Range(_lastInflationEpoch, _epochTo);
7
8 function confirmInflation(bytes32 txHash, ...proof) external {
9     Range memory r = pendingInflation[txHash];
10    if (r.to == 0) revert NoSuchInflation();
11    // Succeeded: commit the cursor. Failed: clear it so the range can be resent.
12    if (ZKSYNC_BRIDGE_HUB.proveL1ToL2TransactionStatus(...)) {
13        if (r.to > lastInflationEpoch) lastInflationEpoch = r.to;
14    }
15    delete pendingInflation[txHash];
16 }
```

Alternatively let L2 `inflationReceive(epochFrom, epochTo)` accept the L1-supplied range and check funds only for it, which removes the dependence on the two cursors agreeing. Quote the L1 fee from `tx.gasprice` rather than the message price, and read `incentivePercentage` from one source.

Remediation Status

Remediated in f6230f7e8. Delivery is now a proof-driven phase machine. `_inflation` records the request and blocks further inflation until it clears, `proveInflationRequestStatus` proves the canonical transaction succeeded or failed, and `submitInflationReconciliation` re-delivers or returns the principal from an attested L2 escrow; only a proven reconciliation advances `confirmedEpoch` and reopens the machine. The cursor still advances inside `_inflation`, but a failed delivery is recoverable rather than terminal, the retry is permissionless, and `confirmedInflationEpoch()` exposes the settled frontier separately from `lastInflationEpoch`. The finding stays *Open* against c382be97.

3.31 [GTV-32] The registry rotation added in c382be97 can be blocked by a one-wei squat on the target address

Risk Level

Severity: Low, Likelihood: Medium, Status: Remediated in 66af77611 (introduced by the GTV-16 remediation in c382be97)

Code Location

- `contracts/vesting/VestingFactory.sol`: 283–310 (`rotateBeneficiaryRegistration`, reverts `VestingAlreadyExists` when `beneficiaryToVesting[_newBeneficiary] != 0`), 98 (`createVesting`, permissionless), 311 (`clearBeneficiaryRegistration`, owner-only)
- `contracts/vesting/Vesting.sol`: 1228–1240 (`setBeneficiary` calls the factory rotation; the revert propagates)

Description

The newer tree fixes GTV-16 by having `setBeneficiary` rotate the factory's `beneficiaryToVesting` entry, and the rotation refuses when the new address already has an entry. Because `createVesting` is still permissionless with a one-wei minimum (GTV-14), anyone watching for a `setBeneficiary(newB)` can front-run it with `createVesting(beneficiary = newB, 1 wei)`, making the rotation revert and planting an attacker-created vesting as `newB`'s canonical registry entry, which also poisons `getValidatorWallets(newB)`. Only the owner's `clearBeneficiaryRegistration` unblocks it, and the squatter re-squats between the clear and the retry unless both are batched.

Impact

A beneficiary's only key-rotation path is grievable for one wei per attempt and needs owner intervention each time. No fund loss: the seven passes that flagged it found no on-chain consumer that trusts `getVesting` or `getValidatorWallets` for authorization, so the damage is denial of rotation and off-chain misdirection.

Relation to prior findings

GTV-14 (creation-time squat) and GTV-16 (stale registry on rotation). The rotation-time squat did not exist in the audited snapshot and is a side effect of the GTV-16 fix.

Recommendation

Make the rotation succeed when the existing entry for `_newBeneficiary` is a vesting that has never been funded above a threshold (or is owned by the same creator), or gate `createVesting` behind an allowlist or minimum amount as recommended in GTV-14, and make the owner clear atomic with the rotation.

Remediation Status

Remediated in 66af77611. `rotateBeneficiaryRegistration` no longer reverts on an occupied target; it emits `BeneficiaryRegistrationRotationDeferred` and returns, so the local role transfer always succeeds and the beneficiary resolves the lookup atomically with `claimBeneficiaryRegistration`. The finding stays *Open* against c382be97.

4 Additional Findings in Later Builds

Build 66af77611 rewrites most of `Vesting.sol`. The balance-delta settlement of the audited snapshot is replaced by a per-tranche ledger keyed on the staking withdrawal epoch, creator payouts become pull credits, pre-revocation profit and loss is netted into a signed carry at revoke, and every proxy deployed before the upgrade has to pass a two-party migration before any staking or revocation path opens. Arcadia reviewed the changed code for defects it introduces. GTV-33 to GTV-38 are new to 66af77611; none exists in the audited snapshot.

Line numbers refer to `contracts/vesting/Vesting.sol` in that build unless stated. The staking contracts are unchanged and remain out of scope, but are read where the new vesting logic depends on them. All six were re-checked against `f6230f7e8` and were closed there. That build introduced GTV-39. The September 16 review of recovery snapshot `0760dd53e850` confirmed that it remediates GTV-39 but reintroduces the forced-unstaking impact of GTV-36. GenLayer acknowledged and accepted that Low-severity impact as a deliberate design trade-off. The September 24 differential review and the September 25 and 26 audit of all 13 agreed TGE implementation files used `404cd5ef8fa4`. GTV-39 remains remediated and GTV-36 remains acknowledged and accepted. That audit raised GTV-40, a token finding: after the move from zkSync to an Arbitrum Orbit rollout, an L2 address the token trusts can be occupied on L1. GenLayer fixed it in `cb764fc5c`. The latest test results are recorded in Sections 5.9 and 5.10. The historical findings and remediation records below retain their original snapshot references. Severities and current statuses are in the summary table of Section 2.

4.1 [GTV-33] A permissionless staking claim desynchronises the self-stake ledger and books principal as reward

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in `f6230f7e8`

Code Location

- `contracts/vesting/Vesting.sol`: 939–989 (`_claimValidator`, the before/after tranche diff), 1004–1015 (`_settleValidator`)
- `contracts/staking/StakingValidators.sol`: 125–135, `validatorClaim(address)` has no caller check
- `contracts/utils/ValidatorUtils.sol`: 306 (returns 0 when nothing is claimable), 338 (pays the wallet)
- `contracts/validatorsWallet/ValidatorWalletBlueprint.sol`: 347–357 (`validatorClaim` forwards the whole wallet balance), 615 (`receive` accepts staking only)

Description

`_claimValidator` reads the wallet’s withdrawal list, calls the wallet, reads the list again, and settles principal only for the epochs that were present before and are gone after. That is its only source of principal; the audited snapshot’s clamp of the returned amount against `validatorDeposited` is gone.

Staking’s `validatorClaim(address _validator)` can be called by anyone. `delegatorClaim` requires `msg.sender == _delegator`; the validator variant does not. A third party calling it on a vesting-owned wallet pays the matured

tranche into the wallet and purges it from the list. When the vesting claims afterwards: `beforeLen` is 0, so nothing is matched; the wallet's inner staking call returns 0 without reverting; the wallet forwards its balance, so `returned` is the full principal plus yield; the wallet still holds shares, so `_validatorFullySettled` is false and no residual is added; `_settleValidator(wallet, 0, returned)` books everything as reward. `validatorDeposited`, `pendingValidatorExitDeposited` and `validatorExitPrincipalByEpoch[wallet][e]` keep the principal on the books.

Impact

Pre-revocation the reward lands in `accumulatedRewards` and is withdrawable at once, so the beneficiary can turn unvested self-stake principal into cash: `exit`, wait for unbonding, call `staking.validatorClaim(wallet)` from any address, then `vestingValidatorClaim` and `vestingWithdraw`. The stale principal surfaces as a loss only when the wallet is fully settled, by which time the funds are gone and the entitlement floors at zero. If `revoke` intervenes, the loss is charged to the carry, but the beneficiary has already withdrawn. The creator loses up to the exited principal. Any generic claim keeper produces the same desync by accident.

Proof of Concept

The repository's `MockStaking` does not expose the permissionless claim, so the sequence is traced against the real staking code.

```
1 // beneficiary: vestingValidatorJoin(pk, proof, 300) validatorDeposited = 300
2 // beneficiary: vestingValidatorExit(wallet, sharesFor(100)) pending = 100, byEpoch[e] = 100
3 // ... unbonding period passes, validatorPrime runs ...
4 // anyone: staking.validatorClaim(wallet) 100 -> wallet, tranche purged
5 // beneficiary: vestingValidatorClaim(wallet)
6 // beforeLen = 0, principalSettled = 0, returned = 100
7 // accumulatedRewards = 100; validatorDeposited still 300, pending still 100
8 // beneficiary: vestingWithdraw(100) succeeds before the cliff
```

Recommendation

Settle from the vesting's own record rather than from what this call happened to observe. Keep the exit epochs per wallet and settle every recorded epoch that staking no longer lists:

```
1 mapping(address => uint256[]) private validatorExitEpochs; // push in _exitValidator
2
3 // in _claimValidator, replacing the before/after diff
4 uint256[] storage epochs = validatorExitEpochs[wallet];
5 for (uint256 i = epochs.length; i > 0; i--) {
6     uint256 e = epochs[i - 1];
7     if (!stakingListsWithdrawal(staking, wallet, e)) {
8         principalSettled += validatorExitPrincipalByEpoch[wallet][e];
9         delete validatorExitPrincipalByEpoch[wallet][e];
10        epochs[i - 1] = epochs[epochs.length - 1];
11        epochs.pop();
12    }
13 }
```

As defence in depth, `clamp` returned against `pendingValidatorExitDe-`

posited[wallet] when no tranche settled, and add if (msg.sender != _validator) revert to StakingValidators.validatorClaim, mirroring delegatorClaim.

Remediation Status

Remediated in f6230f7e8. `_claimValidator` settles from the durable ledger instead of the before/after diff: it sums the principal still referenced by live tranches and books the remainder as `pendingValidatorExitDeposited - remainingPrincipal`. A third party's claim purges the tranche but not the vesting's own record, so the forwarded balance settles against principal and nothing is booked as reward. `_exitValidator` additionally clears a stale same-epoch entry before reusing it. Staking's `validatorClaim(address)` still carries no caller check, but the vesting no longer depends on one. The finding stays *Open* against 66af77611.

4.2 [GTV-34] Legacy proxies cannot be revoked until the beneficiary co-signs the migration

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated in f6230f7e8

Code Location

- `contracts/vesting/Vesting.sol`: 1289 (revoke opens with `_requireDelegationAccounting`), 1420, 1456, 1470, 1506 (the four revoker recovery paths, same gate)
- `contracts/vesting/Vesting.sol`: 1712–1727 (`_approveAccountingMigration`: creator approves, beneficiary executes), 232 and 1804 (the only two writes of `delegationAccountingInitialized`)

Description

`delegationAccountingInitialized` is set in `initialize`, which existing proxies never run again, and in `migrateAccounting`, which executes only after the creator has approved the exact payload and the beneficiary has repeated the call. Until then `revoke` and every revoker recovery path revert with `DelegationAccountingMigrationRequired`. `vestingWithdraw` is not gated before revocation, so the beneficiary keeps withdrawing on schedule. Each delegation target must also be pinned first through `migrateDelegationRouter`, another two-party round per validator, because `migrateAccounting` calls `_getDelegationStaking`.

For every proxy deployed before the upgrade, revocation therefore depends on the consent of the party being revoked. `stopVesting` still works and freezes accrual, but returns nothing to the creator.

Impact

A beneficiary who expects to be revoked does not sign. Unvested idle balance and staked principal stay under that beneficiary's control for as long as the standoff lasts, and the creator has no on-chain move.

Proof of Concept

```
1 // legacy proxy upgraded to 66af77611, delegationAccountingInitialized == false
2 // creator: migrateAccounting(validators, data) -> stores approval, returns
3 // beneficiary: (never calls)
4 // revoker: revoke() -> DelegationAccountingMigrationRequired
5 // revoker: stopVesting() -> ok, but no clawback
6 // beneficiary: vestingWithdraw(vestedSoFar) -> ok
```

Recommendation

`revoke` touches no delegation ledger; it snapshots the schedule, nets profit and loss, and credits idle balance. The code already handles the revoked-but-unmigrated state: `_withdrawableAmount` returns 0 for it (line 448) and a test covers migrating an already revoked proxy. Drop the gate from `revoke` and keep it on the recovery paths. That also gives the beneficiary the incentive to complete the migration, since post-revocation withdrawals stay at zero until it is done.

```
1 function revoke() external onlyRevoker {
2     // no _requireDelegationAccounting(): revocation stays unilateral
3     if (revoked) revert AlreadyRevoked();
4     ...
5 }
```

If the gate is kept, let the revoker execute a creator-approved payload after a fixed delay when the beneficiary has not.

Remediation Status

Remediated in f6230f7e8. `revoke` no longer calls `_requireDelegationAccounting`. The gate is retained on withdrawals, staking and the four revoker recovery paths, and `_withdrawableAmount` still returns zero for a revoked but unmigrated proxy, which leaves the beneficiary with the incentive to complete the migration. The finding stays *Open* against 66af77611.

4.3 [GTV-35] The accounting migration cannot detect a missing validator, and a miss is permanent

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated in f6230f7e8

Code Location

- `contracts/vesting/Vesting.sol`: 1736–1806 (`migrateAccounting`: sorted-and-unique check, no coverage check, one-shot flag), 601 and 721 (`totalDelegationOutstanding` `==`, checked arithmetic)

Description

The vesting keeps no list of its delegation targets, so `delegationValidators` is rebuilt off-chain from events. The contract checks that the list is sorted and that each entry is consistent with staking, not that it is complete. Once `delegationAccountingInitialized` is set the function cannot run again.

A validator with principal left out of the list has two effects. `totalDelegationOutstanding` is set below the true total, so the first claim on that validator underflows at line 721 and reverts; the position can still be exited (the `epoch > 0` path does not touch the total) but never claimed through the vesting. A legacy pending exit on it has neither `legacyPendingExitPrincipal` nor a per-epoch record, so its return is booked as pure reward and `pendingExitDeposited` never clears.

Impact

Principal on the omitted validator is stuck in staking's withdrawal queue with no authorised caller, or is misbooked as beneficiary reward, with no correction path. Both signers must get the payload wrong, which the contract can neither prevent nor repair.

Proof of Concept

```
1 // legacy proxy: depositedPerValidator[A] = 100, depositedPerValidator[B] = 100
2 // creator + beneficiary: migrateAccounting([A], "0x") -> totalDelegationOutstanding = 100
3 // beneficiary: vestingDelegatorExit(B, all) -> ok (epoch > 0 path)
4 // ... unbonding ...
5 // beneficiary: vestingDelegatorClaim(B) -> 100 - 100 ok, then
6 // totalDelegationOutstanding -= 100 on a total of 0 -> panic, forever
```

Recommendation

Make the subtraction saturating so a bookkeeping gap cannot brick a claim, and allow a single validator to be added after initialisation under the same two-party approval, guarded by a per-validator flag that `migrateAccounting` also sets:

```
1 mapping(address => bool) private delegationMigrated;
2
3 function migrateDelegationValidator(address validator) external {
4     if (delegationMigrated[validator]) revert DelegationMigrationDataMismatch();
5     if (!_approveAccountingMigration()) return;
6     uint256 active = depositedPerValidator[validator];
7     uint256 pending = pendingExitDeposited[validator];
8     if (active + pending != 0)
9         _validatePendingDelegation(validator, _getDelegationStaking(validator));
10    legacyPendingExitPrincipal[validator] = pending;
11    totalDelegationOutstanding += active + pending;
12    delegationMigrated[validator] = true;
13 }
```

Remediation Status

Remediated in f6230f7e8. `migrateDelegationValidator` adds an omitted target after initialization under the same two-party approval, guarded by a per-validator `DELEGATION_ACCOUNTED` flag so it cannot run twice, and `_requireDelegationTargetAccounting` blocks an unaccounted target from exit-

ing, claiming or being recovered. An omission is now a named revert rather than an underflow that strands the position. The finding stays *Open* against 66af77611.

4.4 [GTV-36] Revoker recovery can exit the beneficiary's vested share

Risk Level

Severity: Low, Likelihood: Medium

Status: Acknowledged and accepted in 0760dd53e850

Code Location

- `contracts/vesting/Vesting.sol`: 438–441 (`_recoveryExitShares`, the minimum-stake escalation), 409–413 (`_requiredTargetBacking`, other positions counted at nominal principal), 92–94 (`delegationVestedFloor`, `selfStakeVestedFloor`, now written and never read)
- Recovery snapshot 0760dd53e850, `contracts/vesting/Vesting.sol`: 1218–1234 (revoker delegation exit), 1309–1328 (revoker validator exit); neither applies the recovery lens's vested-stake floor

Description

The audited snapshot capped a revoker exit strictly at the unvested shares. Two changes loosen that. `_recoveryExitShares` sets `shares = totalShares` whenever the remainder would fall under the staking minimum, overriding the floor it just computed, so any position under roughly twice the minimum stake is exited in full. `_requiredTargetBacking` counts every other unsettled position at nominal principal as backing for the beneficiary's entitlement; a slashed position elsewhere is over-counted, the backing required here is understated, and the floor drops below the vested share.

Impact

The beneficiary's entitlement is intact, since the principal returns to idle balance and is reserved, but the beneficiary loses the post-revocation yield share on stake that was meant to stay staked.

Original Recommendation

Revert with a dedicated error instead of escalating, or escalate only when the floor is zero. Value other positions at `stakeOf` or `vStake` rather than nominal principal in `otherBacking`. Remove the two floor mappings or mark them informational.

Remediation Status

Remediated in `f6230f7e8`. In `VestingRecoveryLens.recoveryExitShares` the minimum-stake escalation is conditional on `floor == 0`, and a nonzero floor is instead raised to at least the minimum stake. Other positions are valued at live custody, `stakeOf`, `vStake`, wallet balance and priced commit outputs, rather than nominal principal, so a slash elsewhere no longer understates what this position must retain. The two floor mappings are still written and never read. The original issue remains reproducible in `66af77611`.

Reopened during review of recovery snapshot `0760dd53e850`, as a consequence of the GTV-39 remediation. Both revoker exit paths now cap the request only at the selected position's available shares and clear its stored floor. They no longer call `VestingRecoveryLens.recoveryExitShares`. This reintroduces the reported ability to exit vested stake through removal of the protection, rather than through the original minimum-stake escalation or nominal-backing errors.

For example, with a fully vested 1,000 GEN grant, all funds staked and no other backing, `f6230f7e8` rejects a revoker exit because the floor protects the entire position. The new snapshot permits the revoker to request all available shares, subject to staking's exit and maturity rules. The proceeds still belong to the beneficiary under the existing profit-and-loss accounting, but the revoker can end continued staking and its future yield without the beneficiary's consent. The earlier floor was dynamic and could fall as idle funds or pending withdrawals supplied other backing; it was not a permanent guarantee that a fixed quantity would remain staked. The fully vested case demonstrates the additional authority introduced by this change even where that earlier protection would bind.

The beneficiary's net entitlement remains reserved by the withdrawal and foundation-sweep checks. Earned rewards and applicable slashing remain subject to staking settlement, and an exit does not give immediate immunity from slashing. This is a loss of control over continued staking and future yield, not a demonstrated diversion of vested principal; severity remains **Low**.

Client acceptance, recorded September 16, 2026. GTV-36 is **Acknowledged and accepted** by GenLayer for recovery snapshot `0760dd53e850`. GenLayer provided the following statement:

We accept GTV-36 as a deliberate design trade-off. Following revocation, full unstaking—including the vested portion—is permitted so recovery can complete under the existing settlement accounting. Unsettled stake remains exposed to slashing, so treating it as equivalent to liquid funds could allow payouts that leave the beneficiary's entitlement underfunded. The beneficiary retains their financial entitlement under the existing rewards/loss accounting; the revoker does not acquire the vested portion. We accept the resulting staking interruption and potential foregone rewards during unbonding and re-entry. Please record this finding as acknowledged

and accepted.

This disposition records GenLayer's explicit acceptance of the retained behavior and its associated risk. GTV-36 is counted as acknowledged in the summary; its technical impact and Low severity remain documented.

4.5 [GTV-37] A router pin is irrevocable and a wrong one writes the position off

Risk Level

Severity: Low, Likelihood: Low, Status: Remediated in f6230f7e8

Code Location

- `contracts/vesting/Vesting.sol`: 1684–1698, `migrateDelegationRouter`, where a code-length check is the only test on an active-only position and the zero-address test makes the pin final; 734–753, `_reconcileWipedDelegation`

Description

For a position with no pending exit the only check on `staking` is that it has code. Once written, `delegationStaking[validator]` can never change. If the pin is wrong, the pinned router reports zero shares, zero pending deposits and zero withdrawals, and the next claim runs `_reconcileWipedDelegation`, which books all of `depositedPerValidator` as a loss and zeroes it. The real position on the correct router is unreachable from the vesting from then on.

Impact

Both parties must sign the wrong address, but the result is a full write-off of that principal with no correction path.

Recommendation

For an active-only position require that `sharesOf` plus `getPendingDelegatorDeposits` on the pinned router is non-zero for this vesting, and allow re-pinning while the pinned router shows zero custody in all three views.

Remediation Status

Remediated in f6230f7e8. A pin can now be corrected while the previous router holds no custody, and a candidate without observable custody requires an evidence digest both parties attest. `_reconcileWipedDelegation` reverts `DelegationHistoricalLossConfirmationRequired` unless the position carries `DELEGATION_CUSTODY_OBSERVED`, so a mistaken pin can no longer write the principal off on its own; `confirmDelegationLoss` is the deliberate path to that. The finding stays *Open* against 66af77611.

4.6 [GTV-38] Design notes on the new settlement model

Risk Level

Severity: Informational, Likelihood: N/A, Status: Remediated in f6230f7e8

Description

Suggestions only; fixing or not does not change the security posture.

- **Carry policy.** `postRevocationNetCarry` (lines 754–790) attributes post-revocation profit or loss that offsets pre-revocation net profit or loss wholly to the beneficiary before the V/T split. That is a policy choice rather than a consequence of the V/T rule and moves value in either direction relative to a plain split. State it in user-facing documentation.
- **Migration sequencing.** A legacy proxy with N delegation targets needs $2N+2$ strictly interleaved transactions through the single `accountingMigrationApproval` slot; a creator re-approval overwrites a pending one. Let `migrateDelegationRouter` take arrays.
- **Legacy pro-rata.** `legacyPendingExitPrincipal` is released pro rata by share quantity across tranches priced at different epochs (lines 704–709). Acceptable for a one-off migration; worth a comment.
- **Unpriced deposit on a banned validator.** `getPendingDelegatorDeposits` counts a deposit as pending until a prime prices it. If a validator is banned before that happens, `_reconcileWipedDelegation` never fires and `totalDelegationOutstanding` keeps reserving the amount after revocation. Whether staking guarantees pricing for banned validators was not verified (staking is out of scope).

Verified and not reported

Storage: eleven slots appended, `__gap` reduced from 49 to 38, no reordering, inheritance unchanged. `revoke` reads the raw balance rather than `_availableBalance`, safe because `creatorClaimable` is always zero before revocation. `foundationSweep` reserves `creatorClaimable`, so repeated sweeps do not double count. `delegatorClaim` is restricted to the delegator, so the delegation ledger is not exposed to GTV-33; staking keeps one withdrawal entry per epoch, so the per-epoch keying and ordered diff are consistent. Genesis-epoch deposits are neither pending nor excluded from `sharesOf`, so `_delegationPrincipalForExit` prices them correctly. `setBeneficiary` and `setCreator` clear the migration approval, and `keccak256(msg.data)` binds selector and arguments.

Remediation Status

Remediated in f6230f7e8. The carry policy is documented with a worked table, `migrateDelegationRouters` takes arrays of up to 64 targets so migration is no longer a long interleaved sequence, and the legacy pro-rata pool

is residual only, since `_seedDelegationAccounting` subtracts principal that already carries a per-epoch basis. The banned-validator note is answered by staking, which gives a permabanned validator a settlement-only prime so its commits still price. The finding stays *Open* against 66af77611.

4.7 [GTV-39] Revocation walks every validator wallet, so the beneficiary controls whether it fits in a block

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated in 0760dd53e850

Code Location

Locations below refer to the affected build f6230f7e8.

- `contracts/vesting/Vesting.sol`: 1118–1122, `revoke` iterates `validatorWallets` with one external call per entry
- `contracts/vesting/VestingOperatorModule.sol`: 56, where `validatorWallets` is appended with no cap
- `contracts/validatorsWallet/ValidatorWalletBlueprint.sol`: 281–288 (`cancelOperatorTransfer`), 219–232 (`_syncVestingOperatorReservations`, which reaches the factory and the operator registry)
- `contracts/vesting/VestingRecoveryLens.sol`: 224 (`_validatorBacking`, a second walk of every wallet and every withdrawal)
- `contracts/staking/StakingValidators.sol`: 767–772 (the minimum-stake gate is skipped at epoch zero)

Description

`revoke` in f6230f7e8 cancels any pending operator transfer on each wallet the vesting owns:

```
1 address[] memory wallets = validatorWallets;  
2 for (uint256 i = 0; i < wallets.length; i++) {  
3     IVestingValidatorWallet(wallets[i]).cancelOperatorTransfer();  
4 }
```

Each iteration writes three wallet slots and calls on to the factory and the operator registry, which do their own reads and writes. `validatorWallets` is append-only, uncapped, and grows only through `vestingValidatorJoin`. `revoke` was constant cost in the audited snapshot and in 66af77611; here the party being revoked sets its cost.

Staking waives the minimum stake at epoch zero (`epoch() > 0 && msg.value < validatorMinStake()`), so genesis joins cost a wei of principal each. After epoch zero each wallet costs `validatorMinStake`, locked and slashable.

Impact

Past the block gas limit `revoke` can never succeed and the creator loses the clawback permanently. There is no second revocation path and no pagination; `stopVesting` freezes accrual but returns nothing. `VestingRecoveryLens._-validatorBacking` walks the same roster with a call per wallet and per withdrawal tranche, so the revoker exit paths fail earlier still. Same end state as GTV-04, reached by exhausting gas rather than by rejecting a transfer. Likelihood is Low because it takes a few hundred wallets: conspicuous, and outside epoch zero expensive.

Recommendation

Take the unbounded walk off the revocation path. Cancelling pending transfers is defensive: the `_checkVestingOwnerRestrictions` guard inside `completeOperatorTransfer` already prevents a delayed transfer from landing after revocation, so the sweep can be a separate paginated call that revocation does not depend on.

```
1 // revoke() keeps no loop.
2 // Cancel in bounded batches afterwards; revocation never depends on this.
3 function revokeCancelOperatorTransfers(uint256 from, uint256 to) external onlyRevoker {
4     if (!revoked) revert NotRevoked();
5     if (to > validatorWallets.length) to = validatorWallets.length;
6     for (uint256 i = from; i < to; i++)
7         IVestingValidatorWallet(validatorWallets[i]).cancelOperatorTransfer();
8 }
```

Independently, cap the roster at join so no code path that must succeed is unbounded:

```
1 uint256 public constant MAX_VALIDATOR_WALLETS = 32;
2 // in vestingValidatorJoin, before the roster grows
3 if (validatorWallets.length >= MAX_VALIDATOR_WALLETS) revert ValidatorWalletLimit();
```

A cap also bounds the distinctness scan and the recovery lens, both of which walk the same array.

Remediation Status

Remediated in recovery snapshot 0760dd53e850. In that snapshot:

- `Vesting.revoke` (1066–1125) records revocation and local accounting without iterating the wallet roster or calling a wallet. A failing wallet or operator-reservation callback can no longer block this transition.
- Pending operator-transfer cancellation is a separate, single-wallet call that becomes permissionless after revocation (`Vesting.sol`: 901–904; `VestingOperatorModule.sol`: 109–116). The supplied wallet’s completion guard rejects a transfer after its recognized vesting owner is revoked (`ValidatorWalletBlueprint.sol`: 173–213). Older wallets must receive that guard before the vesting upgrade removes automatic cancellation.

- New wallet admission is capped at 32 before external work (`VestingOperatorModule.sol`: 42–43). Retired wallets still count toward the cap.
- Revoker exits inspect only the selected delegation or wallet (`Vesting.sol`: 1218–1234, 1309–1328), removing the recovery lens's portfolio walk from these paths. The new finalizers (1253–1292, 1346–1371) require all active shares and pending deposits to be gone before claiming, and all withdrawals to be cleared afterwards. They verify received value, settle the remaining principal basis once, and retire the position against replay.

The source fix closes the reported revocation blocker and removes the related vesting-side portfolio scan from revoker recovery. It does not establish a universal gas bound for staking: staking's own history scans, activation, unbonding, and slash-safe exit conditions still apply. Ordinary partial-claim paths retain their scans, and the new admission cap does not shorten pre-existing rosters; beneficiary withdrawal still iterates those wallets. The finding remains historically open against `f6230f7e8`.

Related accepted trade-off: GTV-36. Removing the recovery lens from revoker exits also removes its vested-stake protection. GenLayer has acknowledged and accepted that separate Low-severity impact, as recorded under GTV-36 above. GTV-39 remains remediated.

4.8 [GTV-40] The L2 staking address can mint on the L1 token, and on Orbit that address can be occupied on L1

Risk Level

Severity: High, Likelihood: Low, Status: Remediated in `cb764fc5c`

Code Location

- `contracts/bridge/GENCore.sol`: 251–261, `setGenLayerStaking` grants `MINTER_ROLE` and `BURNER_ROLE` to the staking address
- `contracts/GENOrbit.sol`: 148–162, 188 and 439, where the same address is required as the L2 sender of configuration, inflation and burn messages
- `contracts/GENOrbit.sol`: 163–165 and 401–428, where the escrow and fee vault are fixed at activation and named as refund recipients on every delivery and retry
- `contracts/bridge/ArbitrumGENBridge.sol`: 63–68, a dependency outside the TGE scope, where `send` reverts if a refund recipient has code on L1

Description

`setGenLayerStaking` grants the staking address both token roles. The function is unchanged from the previous `GEN.GenOrbit` then uses the same variable to decide which L2 contract it listens to:

```
1 // GENCore.setGenLayerStaking, unchanged from GEN
2 genLayerStaking = _genLayerStaking;
3 _grantRole(MINTER_ROLE, _genLayerStaking);
4 _grantRole(BURNER_ROLE, _genLayerStaking);
5
6 // GENOrbit.onArbitrumMessage: the same address must be the L2 sender
7 if (sender != genLayerStaking || staking != genLayerStaking /* ... */)
8     revert InflationDeliveryConfigurationMismatch();
```

The configuration message, inflation requests and burns are only accepted when their L2 sender is `genLayerStaking`. Activation therefore only works once `genLayerStaking` holds the address of the staking contract on the L2, and at that point the L1 token has given unlimited mint authority to an L2 address.

Under `zkSync` this grant was harmless. `zkSync` computes contract addresses with its own formula, so in practice nothing on L1 can sit at an L2 contract's address. Arbitrum Orbit computes them exactly as Ethereum does, and neither formula includes the chain:

```
CREATE: address = keccak256(rlp(deployer, nonce))
CREATE2: address = keccak256(0xff, factory, salt, keccak256(initCode))
```

The same inputs give the same address on both chains, so the staking address can be occupied on L1. There are two ways this happens, shown in Figure 2(a).

The same deployer key. If the key that deployed staking on the L2 is also used on L1, its nonce there keeps rising. The L1 transaction that reaches the same nonce puts whatever it deploys at the staking address, and that contract holds the minter role. It can mint if it can make arbitrary calls, as a Safe, a timelock or an upgradeable proxy can.

A public CREATE2 factory. If staking is deployed through a public deterministic factory, anyone can deploy the same code with the same salt on L1. That copy can mint if it can be taken over, for example when its proxy is initialized in a separate transaction.

The role does nothing useful on Orbit. `GENOrbit` mints inflation itself in `_send`, and nothing in the staking system calls `GEN.mint`.

The escrow and fee vault have the same exposure with a different result, shown in Figure 2(b). The adapter refuses to send when a refund recipient has code on L1, because Arbitrum would alias that address and route refunds elsewhere. Both are fixed at activation and named on every delivery and retry. If code appears at the escrow's address on L1, all later inflation stops, and the escrow cannot be replaced without upgrading the token. This needs no control of the address: replaying the escrow's deterministic deployment on L1 is enough.

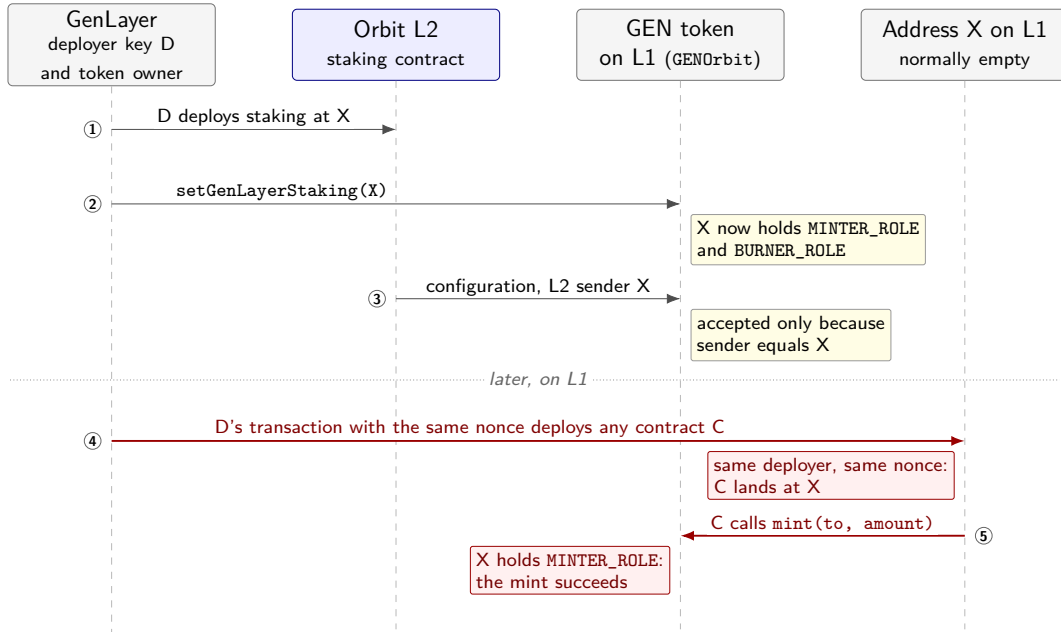
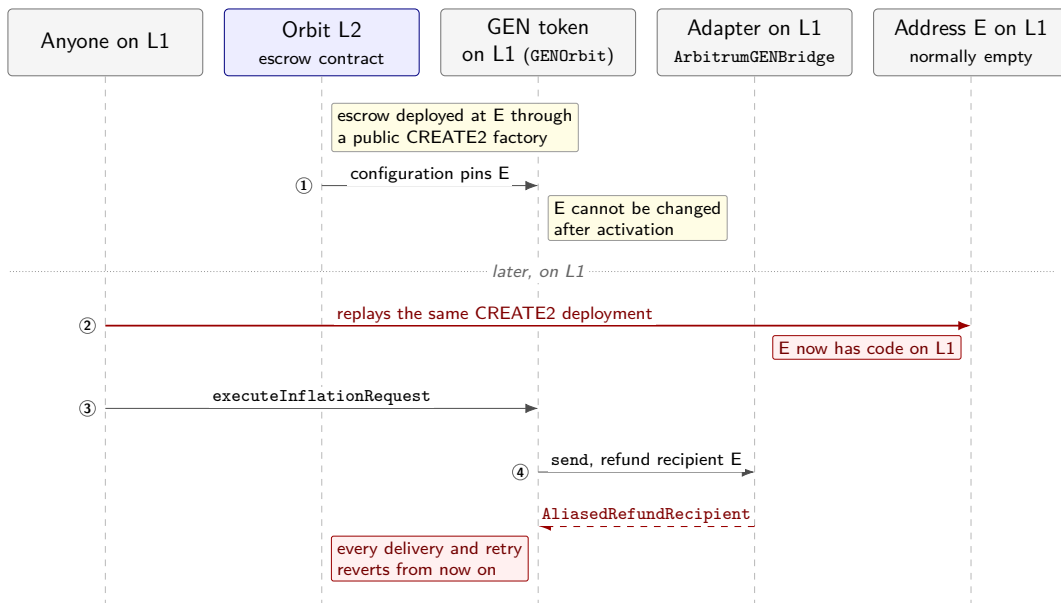
(a) The staking address mints on L1

(b) An occupied escrow address halts inflation


Figure 2: How GTV-40 works. (a) Staking lives at X on the L2, and pinning X on the L1 token grants it the minter role. Orbit computes addresses exactly as Ethereum does, so the deployer's transaction with the same nonce on L1 puts any contract C at X, and C can mint. Deploying staking through a public CREATE2 factory has the same effect for an exact copy of its code. (b) The escrow is fixed at activation. Replaying its deterministic deployment on L1 puts code at E, and the adapter then refuses every delivery that names E as a refund recipient.

Impact

Whatever occupies the staking address on L1 can mint any amount of GEN. That breaks the supply for every holder and for everything bridged to the L2. If the escrow or fee vault address is occupied instead, inflation delivery stops until the token is upgraded. One address meaning different things on two chains is the same mistake that let 20 million OP be taken on Optimism in 2022.

The likelihood is Low because either outcome needs code at one specific L1 address, and that depends on how GenLayer deploys the L2 contracts. Staking is deployed at a later stage, so that choice is still open. The package's deployment note has the owner pin `setGenLayerStaking` before inflation and does not revoke the roles afterwards.

Proof of Concept

Five Foundry tests reproduce both outcomes (Section 5.10). The core one, for the same deployer key:

```
1 address deployer = makeAddr("genlayerDeployer");
2 address l2Staking = vm.computeCreateAddress(deployer, 7); // staking's address on the L2
3 _activate(l2Staking); // setGenLayerStaking and the L2 configuration message
4
5 vm.setNonce(deployer, 7); // the same key reaches nonce 7 on L1
6 vm.prank(deployer);
7 L1Occupant c = new L1Occupant(); // an ordinary contract that can call mint
8 assertEq(address(c), l2Staking); // it lands at the staking address
9
10 c.mintTo(gen, attacker, 1e30); // passes onlyRole(MINTER_ROLE)
11 assertEq(gen.balanceOf(attacker), 1e30);
```

Recommendation

Stop granting token roles to the L2 staking address on Orbit. `setGenLayerStaking` is not virtual in `GENCore`; make it virtual, as was already done for `setIncentivePercentage`, and override it in `GENOrbit` so it records the address without granting anything:

```
1 // GENOrbit: the staking address is an L2 sender to check, not an L1 minter.
2 function setGenLayerStaking(address staking) external override onlyOwner {
3     if (staking == address(0)) revert InvalidContractAddress();
4     if (_deliveryStorage().activated && staking != genLayerStaking)
5         revert InflationDeliveryConfigurationLocked();
6     address previous = genLayerStaking;
7     genLayerStaking = staking;
8     _revokeEndpointRolesIfUnused(previous);
9     emit GenLayerStakingSet(staking);
10 }
```

Until that ships, the owner can revoke both roles right after pinning the address. One of the tests revokes them and shows that activation, inflation and acknowledgements still work.

Deploy the staking contract, escrow and fee vault on the L2 from a key that is never used on L1, and not through a public deterministic factory. If deterministic deployment is needed, use a factory that mixes the chain id and the caller

into the salt, so the same call on L1 lands somewhere else. For the escrow, a fallback is to let the owner replace the escrow and fee vault through a fresh configuration message from the L2.

`setTimelockFactory` grants the same two roles. Nothing in the codebase uses that address, so its chain is unknown; if it ever names an L2 contract, the same applies.

Remediation Status

Remediated in `cb764fc5c`, a single commit directly above the audited `62e309242f61`. `setGenLayerStaking` is now virtual in `GENCore`, and `GENOrbit` overrides it to record the L2 staking address without granting either token role. The owner check, zero-address check, activation lock and cleanup of the previous address are kept. Only the two token files changed; vesting is byte-identical. Run against this build, the same-deployer and `CREATE2` reproductions now revert with `AccessControlUnauthorizedAccount`, and activation, delivery and acknowledgement still work without the roles (Section 5.10). The finding stays *Open* against `404cd5ef8fa4`.

Deployment notice. Two steps are left to deployment, and GenLayer's deployment guide in this build records both:

- **Escrow and fee vault.** The adapter still refuses a refund recipient with code on L1, and both addresses are fixed at activation. Deploy the escrow on the L2 with ordinary `CREATE` from a key that is never used on L1. The escrow creates the fee vault itself, so that one key covers both. Do not use a public deterministic factory unless it enforces chain and caller separation on chain; a chain id placed in the salt is not enough.
- **Existing roles.** A token configured under the earlier implementation keeps any roles the old setter granted. Upgrading does not clear them, and calling the setter again does not either. Revoke both roles for the staking address and confirm neither is held before activation.

5 Testing and Verification

This section distinguishes the **original audit suite**, the **September 16 recovery probes**, the **September 24 delta-review tests**, the **September 25 TGE audit tests**, and the **GTV-40 reproduction and fix verification**. The thirty-one properties from the original threat model (Section 1.6) were the original suite's targets; Section 5.1 records their results. Every finding of Medium severity or above in the original audited snapshot has an executed reproduction. The original harness lives in `test/`, and its five source files remain byte-identical to that delivery. The later tests are recorded in Sections 5.7, 5.8, 5.9, and 5.10.

The original suite contains **125 tests**, all passing against the original de-

livery. Sixteen are stateful invariant tests that drive the contracts through randomized sequences; the rest are property fuzz tests over single functions and differential reproductions that walk one scenario through concrete values. The register, suite composition and campaign descriptions below record that original work; they are not current failures or coverage measurements for snapshot 404cd5ef8fa4.

5.1 Original audit invariant register

The original keeper-percentage and absolute floor-monotonicity properties are retained as historical test definitions. In 404cd5ef8fa4, keeper reimbursement is separately minted and gas-based, while the vesting floor scales with staking rewards and losses. The September 24 and 25 tests address the changed behavior separately.

The threat model ended in thirty-one properties stated precisely enough for a machine to check. Each one is reproduced below with the test that exercises it and the result. Arcadia records four outcomes rather than two, because that difference is what the exercise is for.

holds means the property was argued from the source and exercised without a counterexample. **VIOLATED** means the code breaks the property, and the finding it produced is named. **partial** means one clause of a compound property holds while another does not, or the covering test exercises only part of it. **uncovered** means the threat model asserted the property and nothing in the suite tests it. Those rows are listed rather than dropped without mention.

Of the thirty-one, thirteen hold, seven are partial, five are uncovered, and **six are violated**. The six violations are the spine of this report: every one became a finding, named in the register's last column.

ID	Property	Result	Evidence	Finding
G-INV-1	Every supply change occurs at one of five enumerated mint/burn sites	partial	test_GINV1_supplyConse rvation	GTV-09
G-INV-2	initialMint succeeds at most once per arming and mints exactly the genesis supply	partial	test_S1_resetInflation ReArmsSupply	GTV-02
G-INV-3	lastInflationEpoch never de- creases	VIOLATED	test_CRITICAL_epochZer oRewindsCounter_enabli ngDoubleMint	GTV-01
G-INV-4	A replay coordinate executes at most once	holds	test_GINV4_exactReplay Blocked	GTV-01, GTV-24
G-INV-5	Executor reward equals the exact carve and never exceeds the 10% cap	holds	testFuzz_caller_reward IsExactCarve	—
G-INV-6	incentivePercentage is always at most 1000 bps	uncovered	none	—
G-INV-7	Supply moves only through role-gated or internal bridge paths	uncovered	test_P7_rotationLeaves StaleMinter_andHandove rLeavesMintKey	GTV-07, GTV-15
G-INV-8	The inflation curve is monotonic and never reverts within the permitted range	VIOLATED	test_V4_inflationCurve RevertsPermanentlyPast FiniteEpoch	GTV-20
V-INV-1	vestedFloor never decreases	holds	invariant_floorMonoton e	—

ID	Property	Result	Evidence	Finding
V-INV-2	Vested amount is at least the floor and never decreases with time while live	partial	test_creatorFreeze_isPermanentAndUnreversible	–
V-INV-3	Withdrawable never exceeds the contract balance	holds	invariant_withdrawableLeBalance	–
V-INV-4	Cumulative withdrawals never exceed the entitlement ever accrued	VIOLATED	test_pricedZeroTranche_lossNeverBooked_creatorAbsorbsIt	GTV-06, GTV-08
V-INV-5	totalWithdrawn never decreases	holds	test_C2_partialGrabInWindow_isSelfPunishing	–
V-INV-6	Vested amount never exceeds totalAmount, including across top-ups	holds	invariant_vestedNeverExceedsTotal	–
V-INV-7	No join, deposit or top-up succeeds while revoked or stopped	uncovered	none	–
V-INV-8	State transitions are legal and no state is a dead end	VIOLATED	test_P5_stopThenRevoke_permanentlyDisablesBeneficiarySettlement	GTV-12
K-INV-1	Tracked principal never exceeds what was sent minus what returned	partial	test_Q5_sharesReading_roundTripIsExact	GTV-08
K-INV-2	pendingExitDeposited is non-zero only for exits taken past epoch zero	holds	test_P3_epochZeroExitBooksNoPnL_creatorAbsorbsTheLoss	GTV-10
K-INV-3	A claim returning zero books no profit or loss and preserves the tracker	partial	test_pricedZeroTranche_lossNeverBooked_creatorAbsorbsIt	GTV-08
K-INV-4	Contract balance equals every inflow minus every outflow	holds	invariant_conserved	–
K-INV-5	Post-revocation, unsettled staked principal is reserved against withdrawal	holds	test_poc_beneficiaryDeducesSlashShare	GTV-06
K-INV-6	The profit and loss split conserves value and cannot underflow	holds	invariant_creatorShareNeverUnderflows	GTV-04, GTV-11
A-INV-1	No two live wallets of one vesting share an operator	VIOLATED	test_V2_dualPendingTransfer_defeatsDistinctOperatorCheck	GTV-13
A-INV-2	Every operator set was a codeless address other than the vesting itself	uncovered	none	–
A-INV-3	After revocation, operator rotation and beneficiary rotation revert	uncovered	none	–
A-INV-4	Each role reaches only its own functions	partial	invariant_valueConserved	–
F-INV-1	A registered beneficiary maps to exactly one vesting	VIOLATED	test_setBeneficiary_requestsCanonicalRegistry	GTV-14, GTV-16
F-INV-2	A created vesting holds exactly its stated total immediately after creation	holds	test_batch_happyPath_fundsAndRegistersEach	–
F-INV-3	A batch spends exactly its value and never touches pre-existing balance	holds	test_batch_fundingMustMatchExactly	–
F-INV-4	isVestingContract is true precisely for factory-deployed vestings	holds	test_batch_happyPath_fundsAndRegistersEach	–
F-INV-5	The beacon is deployed once and only the owner mutates factory configuration	partial	test_V1_REFUTES_beacon_OwnershipIsRecoverableViaFactoryOwner	–

5.1.1 The six violated properties

G-INV-3, that the inflation epoch counter never decreases, is the most consequential. It is the only control that stops an already-minted epoch range from being paid twice, because the replay map keys on coordinates a second message can vary freely. A zero epoch target rewinds it (GTV-01).

G-INV-8, that the curve never reverts inside the permitted range, fails past

epoch 15,107 where the fixed-point exponent leaves its domain (GTV-20). The pre-existing curve fuzzing bounded its inputs below epoch 6040 and could not reach the boundary.

V-INV-4, that cumulative withdrawals never exceed the entitlement ever accrued, fails in two independent ways: the post-revocation reserve omits delegation principal (GTV-06), and a priced-zero tranche leaves a realised loss unbooked so the beneficiary withdraws against an entitlement that should have been reduced (GTV-08).

V-INV-8, that no state is a dead end, fails because stopping and then revoking leaves a flag no transition can clear, and both beneficiary settlement paths are gated on it (GTV-12).

A-INV-1, that no two live wallets of one vesting share an operator, fails through the two-step rotation: the distinctness check reads current operators and the completion step re-checks nothing (GTV-13).

F-INV-1, that a registered beneficiary maps to exactly one vesting, fails once a beneficiary rotates, because the factory is never told and the new beneficiary's slot stays free for a second escrow (GTV-16).

5.1.2 Where a passing property is not reassurance

Two rows deserve to be read with care, because a green result there means less than it appears.

K-INV-3 states that a claim returning zero books no profit or loss and preserves the tracker. The code does exactly that, so the property passes. But the reference staking treats a priced zero payout as final, which means a fully slashed tranche also returns zero and its loss must be booked. The property faithfully encodes the author's assumption that zero means not yet matured, and that assumption is the defect. This is the second half of GTV-08.

G-INV-2 holds only because its own wording exempts `resetInflation`. A property written around a lever is not evidence the lever is safe.

Both cases make the same point. A suite of passing invariants bounds correctness only as far as the invariants are themselves correct, and two of these thirty-one were written around the behaviour rather than against it.

5.1.3 The uncovered properties

Five properties have no test. G-INV-6 (the incentive cap) and V-INV-7 (no joins or top-ups while revoked or stopped) are enforced by visible guards in the source and are low risk, but neither is exercised. G-INV-7 (supply moves only through role-gated paths) is not tested as worded, and the role-hygiene failures in GTV-07 and GTV-15 sit adjacent to it. A-INV-2 and A-INV-3, covering operator code checks and post-revocation rotation freezes, are untested because

the operator surface was the largest coverage hole in the engagement. Closing these five is the cheapest available increase in assurance.

5.2 Original audit test suite composition

File	Tests	Kind	Primary target
GenSupplyBridge	11	unit	Supply conservation, replay, BURN sourcing (GTV-09)
IndependentGenCritical	6	unit	Zero-epoch counter rewind (GTV-01)
GenInflationMath	5	fuzz	Inflation curve shape
InflationMathFuzz	14	fuzz	Curve over/underflow and caller (GTV-20, GTV-22, GTV-23)
VestingEntitlement	4	invariant	Schedule, floor, withdrawal ceiling
VestingStaking	3	invariant	Delegation round-trip conservation
VestingValidator	5	invariant	Self-stake, ground-truth no-free-money
VestingRevokeReserve	3	differential	Delegation reserve gap (GTV-06)
VestingConditional	4	conditional	Staggered maturity, dust (GTV-08)
VestingEdgeCases	3	unit	Manual-unlock zeroing (GTV-18)
VestingAdversarial	7	adversarial	Reentrancy, lying staking, repointed registry (GTV-26)
IndependentReview	10	PoC	Rejecting creator, overflow brick, floor stranding (GTV-03, GTV-04, GTV-05)
AssessmentProbes	8	differential	Severity calibration (GTV-05, GTV-10)
BatchAndUnits	11	unit	Batch funding and the share-vs-token detector (GTV-14)
VerificationPass	5	unit	Confirm or refute candidate findings
FactoryRegistryDrift	4	unit	Registry drift on role rotation (GTV-16)
PricedZeroTranche	2	unit	Unbooked loss on a priced-zero tranche (GTV-08)
CreatorDilution	3	differential	Ratio dilution and share-floor collapse (GTV-17)

File	Tests	Kind	Primary target
CreatorSchedulePower	3	differential	Creator delay and gift powers (GTV-17)
VestedRatioInvariant	4	invariant	Vested ratio can never invert (GTV-11 denominator)
PermissionlessSync	10	PoC	Permissionless wallet sync and migration front-run (GTV-25)
Total	125		

5.3 Original audit invariant campaigns

The invariant campaigns drive the vesting contract through a handler that only makes calls a real actor could make: the beneficiary withdraws and stakes, the creator funds and edits the schedule, the revoker revokes and sweeps, and a fraction of calls come from a random address to test that access control holds. Between calls the handler warps time to move the schedule, and it maintains ghost variables that track every unit of value in and out so conservation can be checked exactly. Each invariant runs over 256 sequences of 500 calls, which is 128,000 calls per property. Sixteen invariant functions are now in place across four campaigns, so the stateful suite executes 2,048,000 randomized calls in total, with no reverts and no counterexample.

The properties checked include: withdrawable value never exceeds the contract's actual balance; the vested floor never decreases across any operation; cumulative withdrawals never exceed the entitlement the schedule ever granted; and total value is conserved as balance equals inflow minus outflow.

5.4 Original audit inflation fuzzing

The inflation curve is the only heavy arithmetic in the codebase, so it received dedicated fuzzing for overflow, underflow, truncation, and domain violations. An earlier property test bounded its inputs below epoch 6040, which sits entirely inside the safe region and cannot reach the point where the curve fails. Arcadia added a sweep across epochs 1 to 25,000 that asserts the curve returns a positive amount below its domain boundary and reverts at or above it, and this located the boundary at epoch 15,107 (GTV-20). The signed helper functions were fuzzed for sign correctness, which surfaced the latent cast in `divInt` (GTV-23). The caller, `executeL2Message`, had never been fuzzed at all; new tests drive it over randomized epoch targets and gas parameters and assert that every minted token lands on the bridge or with the executor, that the executor's carve equals the exact percentage rather than merely staying under the cap, and that the epoch throttle holds for every over-range request.

5.5 Original audit reproduction and verification tests

Each finding above the informational tier is anchored to a test that proves its numbers, the overflow brick shows a single call rendering a 1,000e18 escrow permanently unreadable, with the exact arithmetic panic asserted rather than any revert, so the test cannot pass for an unrelated reason. The rejecting-creator tests show a vesting that cannot be revoked and a principal that neither the beneficiary nor the revoker can claim. The zero-epoch tests mint the same twenty epochs twice and print the doubled supply. A final pass re-examined candidate findings against the source: it confirmed the operator-transfer bypass and the fee scaling that no earlier mock could express, and it refuted one candidate outright, showing that beacon ownership transfers with factory ownership and is not orphaned, which is why that candidate does not appear in this report.

5.6 Limits of the original testing

Two limits bound what the suite establishes. The first is mock fidelity. The L2 staking contract, validator wallet, and address manager were modeled, not audited, so a finding that depends on their behavior is proven against a mock and could be out of line with the reality. One modeling choice matters enough to name: every staking mock in the repository uses a one-to-one ratio between shares and tokens, which makes two readings of the exit interface indistinguishable. Arcadia added a detector that breaks that tie at a two-to-one ratio and showed that if the real staking treats the exit argument as a token amount rather than a share count, the delegation path books phantom rewards and strands principal.

The second limit is coverage. A handful of functions are still not exercised by any test, most of them metadata setters and registry syncs of lower risk. The operator two-step transfer and the batch creation path, both previously untested, are now covered. The remaining gaps are listed in Arcadia's working notes and are worth closing before a final sign-off.

5.7 September 16 recovery follow-up: GTV-39 and GTV-36

The September 16, 2026 review of 0760dd53e850 used two separate Foundry suites: **10 GTV-39 tests and 6 GTV-36 tests, all passing**. Each suite includes a fuzz test run over 64 cases. These 16 tests are additional to the original 125-test audit suite and do not represent a rerun of that suite against the new package.

The GTV-39 probes cover revocation despite a failing wallet callback, the 32-wallet admission cap, the supplied wallet's post-revocation operator-transfer guard, retryable cancellation, recovery without querying unrelated positions, and both finalizers' receipt, maturity, and replay checks. A synthetic roster comparison measured the same 140,219 execution gas for revocation with one

and 10,000 wallet entries. This isolates the removed roster dependency; it is not a production gas estimate or a bound on staking settlement.

The GTV-36 probes confirm that a fully vested position protected in `f6230f7e8` can be forcibly exited in the new snapshot, while full-exit proceeds remain subject to the existing beneficiary entitlement and creator allocation. They also exercise immature-finalization rollback, rejection of restaking through a revoked grant, the earlier floor's reduction as pending exits supply other backing, and the existing signed-carry rule's asymmetric allocation of some losses. These probes reproduce the behavior GenLayer subsequently acknowledged and accepted as a design trade-off. That acceptance changes the finding's disposition while the tested behavior remains in place.

The probes use the supplied Vesting implementations and modules behind ERC1967 proxies, the supplied validator-wallet implementation for its transfer guard, mocked staking and factory counterparties, and locally available OpenZeppelin libraries. Staking reward and slash timing were assessed from source rather than reproduced end to end. The package's Hardhat suite was not rerun because its installed dependencies were unavailable. This verification supports the targeted remediation conclusions and does not certify beacon deployment, upgrade sequencing, or all staking behavior.

5.8 September 24 token and vesting delta review

The September 24 follow-up compared snapshot `404cd5ef8fa4` with the September 14 final-recovery snapshot `0760dd53e850`. It recorded **30 passing local tests, zero failures and zero skipped tests**. The saved results are separate from the original 125-test suite and the September 16 set of 16 probes; those suites were not all rerun against the latest package.

Suite	Tests	Coverage
Vesting delta	9	Scheduled rewards/losses, withdrawals, floor scaling, revocation, zero base, standalone grants and delayed staking connection
Orbit token	8	Bootstrap roles and supply, fee/keeper bounds, authentication, retry, acknowledgement, expiry and replay
Orbit escrow	3	Retained principal after failed delivery, zero-value registration, alias and payload checks; additional dependency evidence outside the TGE scope
GTV-39 regressions	10	Wallet cap, revocation without roster calls, cleanup, finalization receipts, maturity, retirement and allocation
Total	30	All passed

Two fuzz tests ran 128 cases each, covering claim/revocation allocation and val-

idator finalization allocation/replay. The GTV-39 harness retains `f6230f7e8` as a historical broken control in one regression; the differential review's baseline remains `0760dd53e850`. The ten recovery regression tests passed against the updated code.

These tests found no defect in the behavior they exercised, with GTV-39 still remediated and GTV-36 still acknowledged and accepted. They did not model the L1 and L2 address overlap behind GTV-40 (Section 5.10). Token tests exercised later activation paths using mocked counterparties. The escrow probes are retained in the historical test record; they do not expand the confirmed TGE scope to the separate bridge deployments.

Build and dependency limits. The local harness used the supplied first-party implementations with OpenZeppelin **5.4.0**, solc **0.8.28**, IR compilation, 100 optimizer runs and Cancun. The package locks OpenZeppelin Contracts and Contracts Upgradeable to **5.1.0** and uses Paris for general compilation, with a separate Cancun Orbit job. The local probes therefore do not reproduce the package's exact locked build. The supplier's reported 265 passing Hardhat tests were not independently rerun.

The harness uses mocked staking, bridge and pricing counterparties, raises the local code-size limit, and uses an ERC1967 factory proxy for behavioral tests. It is not a rehearsal of the documented transparent-proxy factory deployment. Native Nitro execution, the complete staking/consensus system, live deployment or upgrade sequencing, and fresh formal verification were not validated. The later bridge/staking activation requires separate integration work; these limits do not represent new findings in the reviewed TGE changes.

5.9 September 25 audit of the agreed TGE scope

This audit used the same `404cd5ef8fa4` delivery and the verified `0760dd53e850` baseline. It manually reviewed all 13 implementation files in Section 1.1.2, including unchanged dependencies, and recorded **47 distinct passing tests, zero failures and zero skipped tests**. Twenty tests were new; 27 were rerun from the earlier regression suites.

Suite	Tests	Coverage
New vesting scope tests	12	Factory and beacon deployment, delayed connection, operator ownership and cleanup, module call restrictions, shared reentrancy protection and settlement allocation
New token and utility tests	8	Initialization, privileged calls, ownership and endpoint roles, initial minting, unconfigured transport, registry seals and possession proofs
Vesting delta regressions	9	Scheduled rewards/losses, floor scaling, revocation, zero base, standalone grants and delayed connection

Suite	Tests	Coverage
Orbit token regressions	8	Supply, fee/keeper bounds, authentication, retry, acknowledgement, expiry and replay
GTV-39 regressions	10	Wallet cap, bounded recovery, cleanup, finalization receipts, maturity, retirement and allocation
Total	47	All passed

All 47 tests passed under Cancun. The 31 vesting/recovery tests were also run under **Paris**, matching the package's general compilation target, and all passed. These 31 are a subset of the 47 tests, not additional distinct tests. Three fuzz tests ran **256 cases each in each EVM run**. They covered two-settlement allocation with variable claim order and prior withdrawals, claim/revocation allocation, and validator finalization allocation/replay.

The new factory tests used the actual `TransparentUpgradeableProxy`, factory, `UpgradeableBeacon`, beacon-proxy grants, helper modules and operator registry. They exercised deployment without staking, creator and beneficiary changes, atomic batch funding, operator reservation collisions and cleanup, and callback reentrancy across the vesting and operator module. Token tests checked unauthorized minting, upgrades and callbacks, initialization guards, role retirement, and operation before transport setup. Registry and public-key tests checked malformed responses and proof binding.

Build and dependencies. The harness used solc **0.8.28**, IR compilation and **100 optimizer runs**, with local copies of OpenZeppelin Contracts and Contracts Upgradeable declaring version **5.1.0**. Hashes of all 324 Solidity dependency files were recorded. This replaces the September 24 harness's 5.4.0 dependency for the tests in this subsection; it does not change that earlier test record. The local copies' npm archive integrity was not independently verified, and the full dependency-locked Hardhat build and supplier test suite were not reproduced.

The harness increased the code-size limit to accommodate large test contracts and historical controls. Production runtime sizes were checked separately under the intended targets: `GENOrbit`, including inherited `GENCore`, was **23,722 bytes** under Cancun; `Vesting` was **23,855 bytes** under Paris. Both are below 24,576 bytes, as are the factory and all four separately deployed vesting helpers. These compiler measurements do not establish deployment gas requirements.

Coverage limits. Some retained regressions use ERC1967 factory fixtures and synthetic state. New operator tests use staking and wallet mocks. The token regressions exercise the actual adapter with mocked canonical bridge and pricing contracts; the adapter remains an external dependency, not an additional audit target. The three separate escrow tests recorded on September 24 were not rerun in this TGE audit. Native Nitro execution, the complete

staking/consensus system, a live deployment or upgrade rehearsal, and fresh formal verification were not covered. The original 125-test suite and its full stateful invariant campaign were not rerun.

GTV-39 remains remediated in the exercised recovery paths. GTV-36 remains the acknowledged and accepted full-unstaking trade-off; that behavior was not changed by this review. A second pass over the same scope on September 26 raised GTV-40, which these tests did not exercise. Its reproduction follows.

5.10 GTV-40 reproduction and fix verification

Five Foundry tests reproduce GTV-40 against 404cd5ef8fa4. They use the compiler settings and dependency copies of Section 5.9 in a separate harness, so the 47-test record above is unchanged. **All five pass.**

Test	What it shows
Activation grants the roles	Activation only works with the L2 staking address pinned, and that address then holds both token roles
Same deployer, same nonce	The deployer reaches the same nonce on L1, deploys an ordinary contract at the staking address and mints one trillion GEN
CREATE2 on L1	A CREATE2 address does not depend on the chain; a contract deployed the same way on L1 mints
Orbit without the roles	With both roles revoked, activation, a full inflation delivery and its acknowledgement still complete
Escrow replay	Delivery works; a third party replays the escrow's CREATE2 deployment on L1; the next inflation reverts with <code>AliasedRefundRecipient</code> , and reconfiguration is rejected
Total: 5	All passed

The canonical bridge and inbox are the package's own test doubles, as in the Orbit regressions above. The tests show the address overlap and its effect on the token; they do not deploy on a live chain.

Against the fix in cb764fc5c, the three mint reproductions revert with `AccessControlUnauthorizedAccount`, as intended. A separate suite of six tests checks the fixed behavior: pinning the staking address grants no role, contracts placed at that address on L1 by either route cannot mint, and activation, delivery and acknowledgement work without the roles. It also confirms two points behind the deployment notice. Roles granted by the earlier setter survive an upgrade to the fixed implementation, and calling the setter again does not clear them; only an explicit revoke does. An escrow address occupied on L1 still blocks delivery, because the code leaves that half to deployment. **All six pass.**

6 Contact Information

For further inquiries or to schedule a follow-up audit, please contact:

Arcadia Agency

Email: audits@arcdiamgroup.com

Website: <https://arcadia.agency/>

Telegram: <https://t.me/thearcadiagroup>

7 Disclaimer

While best efforts and precautions have been taken in preparing this document, Arcadia and the Authors assume no responsibility for errors, omissions, or damages resulting from the use of the provided information. Arcadia also emphasizes that using its services does not guarantee the security of a smart contract or set of smart contracts and does not guarantee against attacks. One audit on its own is not enough for a project to be considered secure; that categorization can only be earned through extensive peer review and battle testing over an extended period.